cadence®

ASIC Lab Manual

Covering - IES, RC, ET, Conformal, EDI, Tempus, Voltus

Developed By
University Support Team
Cadence Design Systems, Bangalore

Table of Contents

Get started

Directory Structure

Let us understand the directory structure of the counter_database.

- Constraints – Contains SDC file
- lef – Contains lef files
- lib – Contains lib files
- rtl – Contains rtl design files
- QRC_Tech – Contains QRC tech file
- Captable – Contains Cap table
- Gate_level_simulation – Contains Simulation library

Please save your design (RTL files) inside the RTL directory and keep both RTL as well as test bench inside the simulation directory. Libraries, LEF and Constraint files (SDC files) are kept in respective directories. Simulation, Synthesis, gate_level_simulation, Equivalence_checking, Physical_design and STA directories are used to run simulation, synthesis, gate level simulation, logic equivalence checking, physical design and timing analysis so that all log files, command files and other tool generated files won't get mixed up.

Steps to Invoke Tools

- Before invoking any tool, invoke C shell be typing 'csh' in terminal.
- Source the cshrc file by typing 'source <cshrc file>'
  e.g.:- source cshrc

Verification

IES (Incisive Enterprise Simulator) is the tool used for verification. Navigate to Simulation directory where you have kept your RTL and test bench (simulation directory).

- Invoke the tool by typing '*nclaunch -new*' in the terminal.

NCLaunch window will appear like in the below screen shot and in the NCLaunch window, select 'Multiple Step' option.

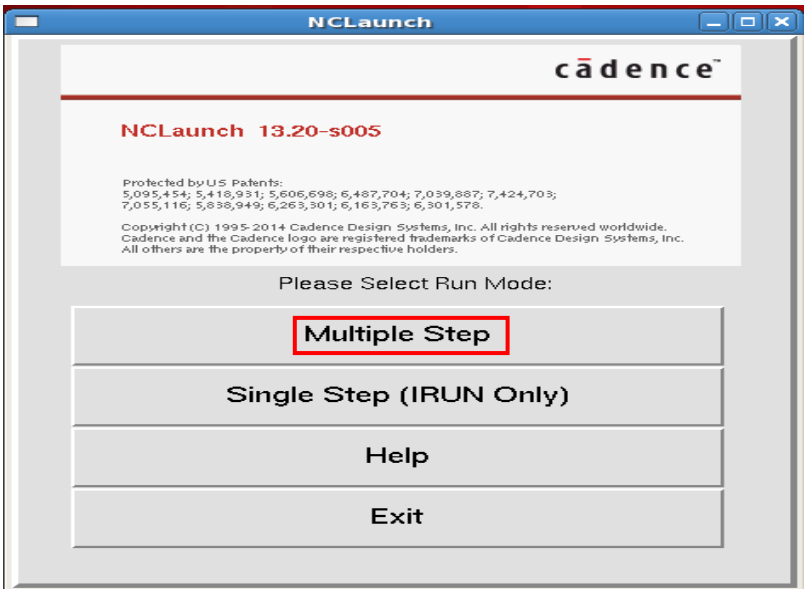
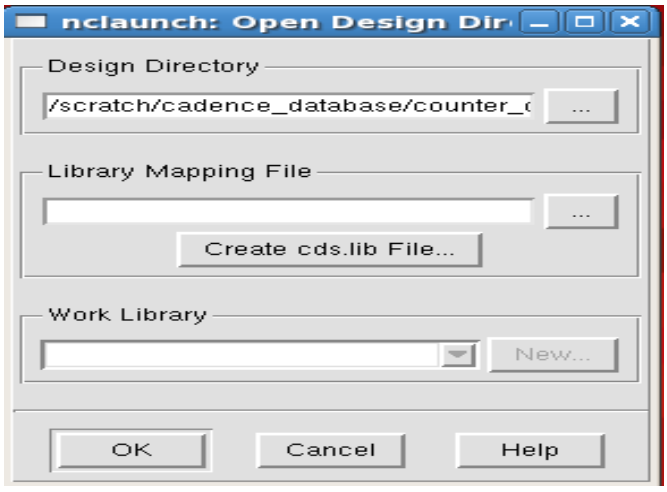
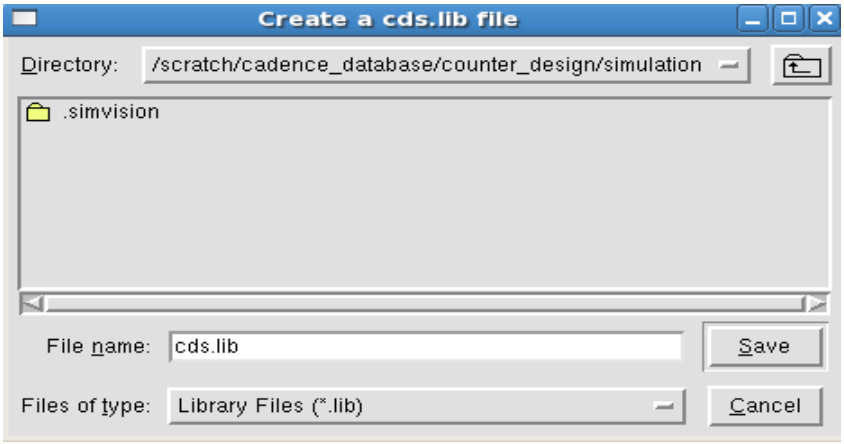


Fig 3

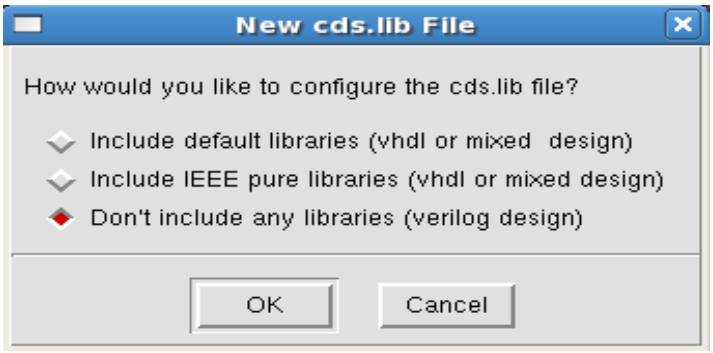
- On clicking the '*Multiple Step*' option, '*nclaunch: Open Design Directory*' window will appear as shown below



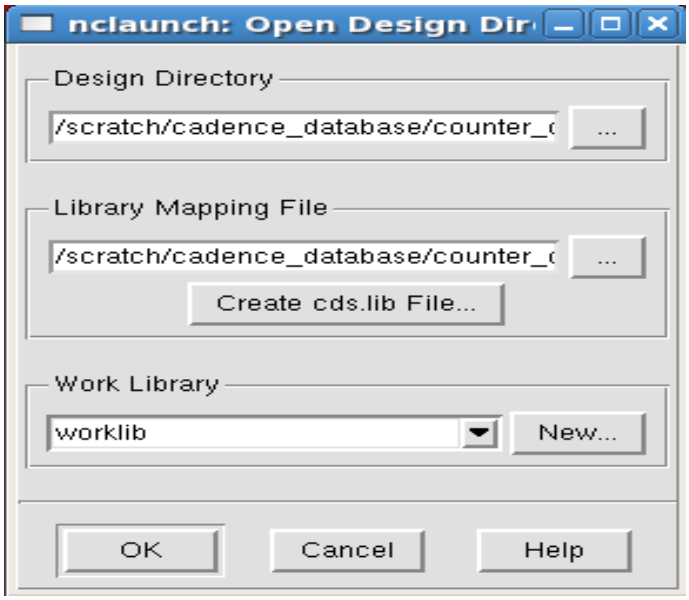
Click on '*Create cds.lib File'* option and a '*Create a cds.lib file'* window will open. Click '*Save'* option.



- A '*New cds.lib File'* window will appear. Click any of the three options available depending on your RTL and click '*OK*'. As the counter design is in verilog, the third option is selected.



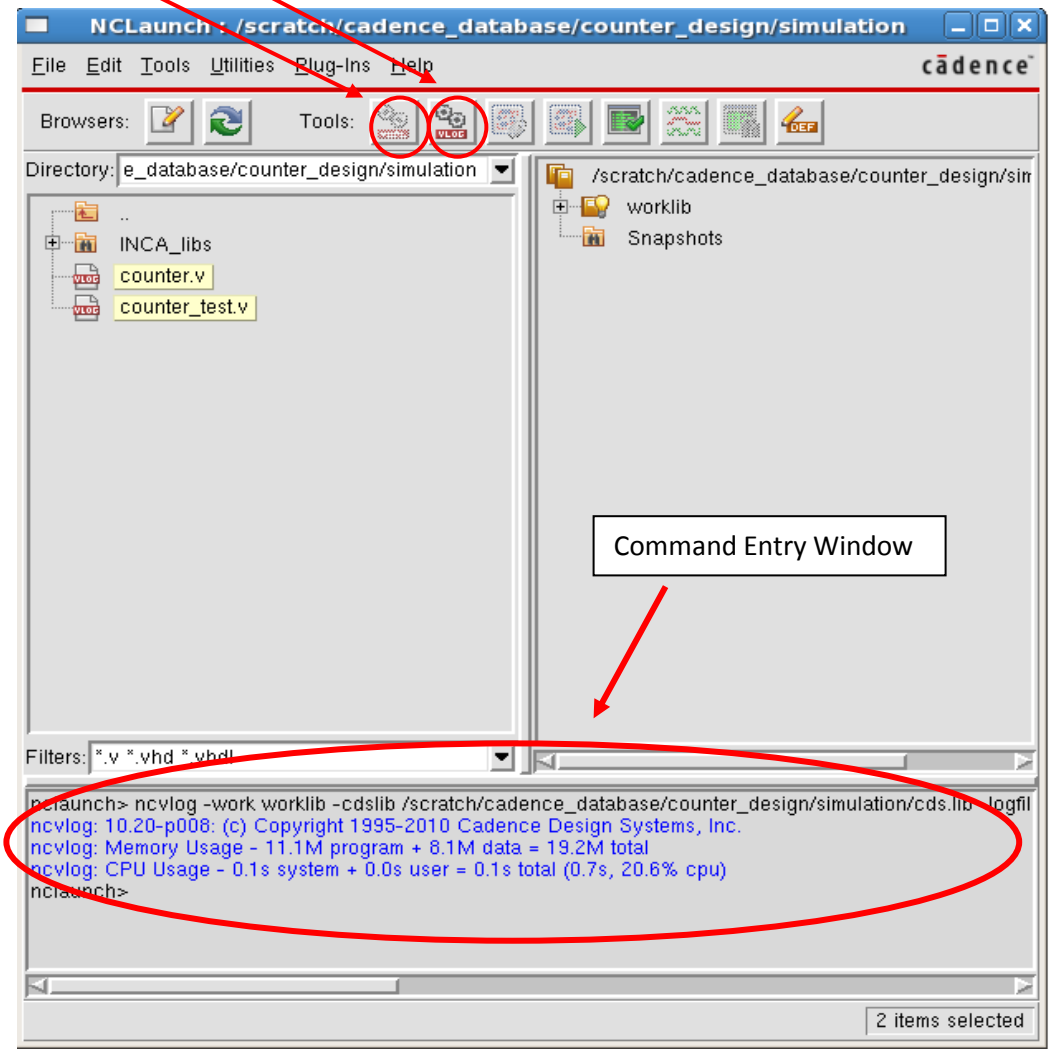
- Click '*OK'* in the '*nclaunch: Open Design Directory'* window.



- In the NCLaunch window, we will be able to see the design as well as the testbench that we kept inside the simulation directory.

Compilation

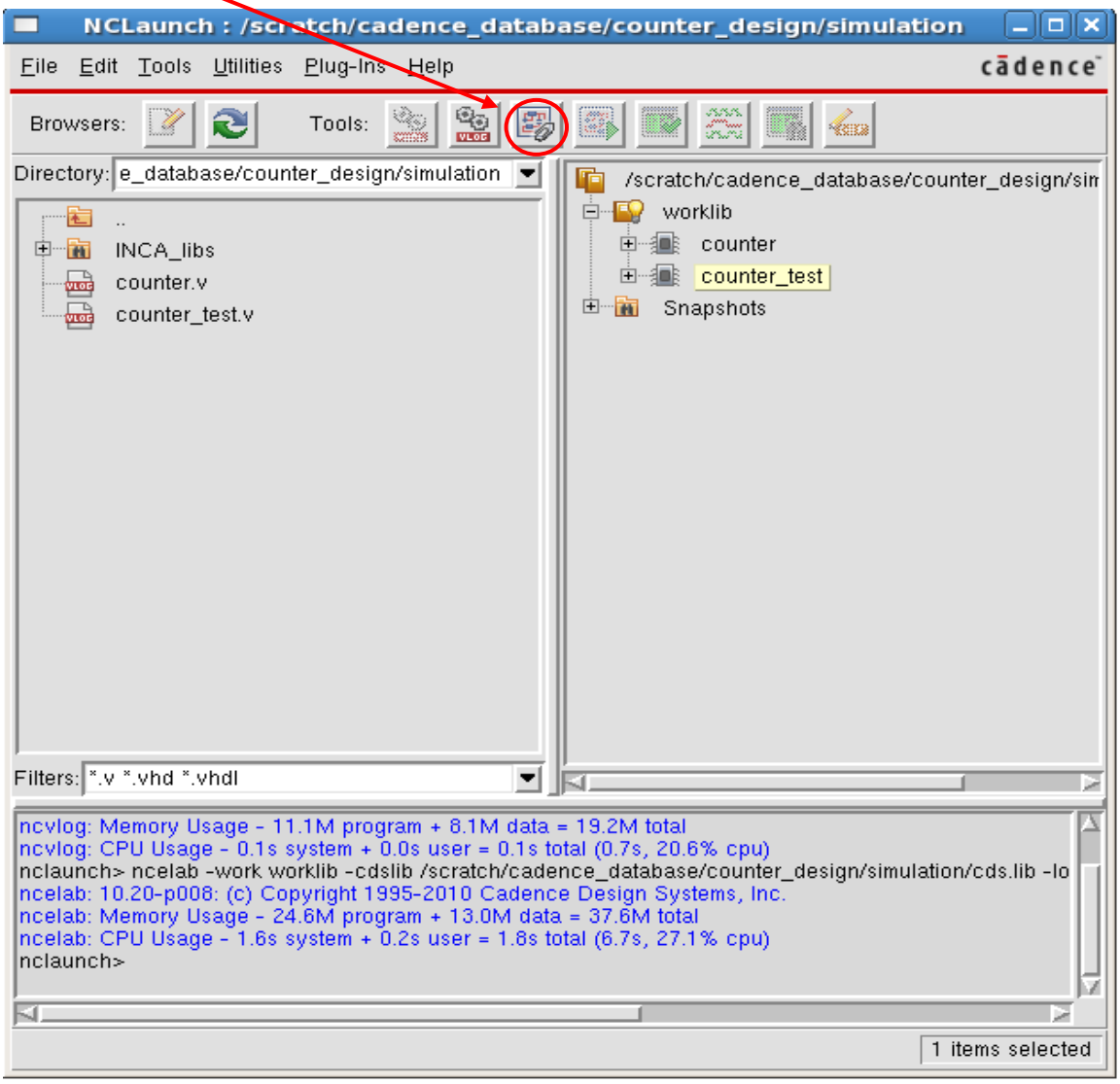
- The next step is to compile (Checks syntax and semantics) the code. For this, select both the design and testbench and choose the appropriate compilers.
  - *ncvlog* for Verilog designs. (choose ncvlog for counter design as the design is in verilog)
  - *ncvhdl* for VHDL designs.



Any error in the code will be reported in the *'Command Entry Window'*.

Elaboration

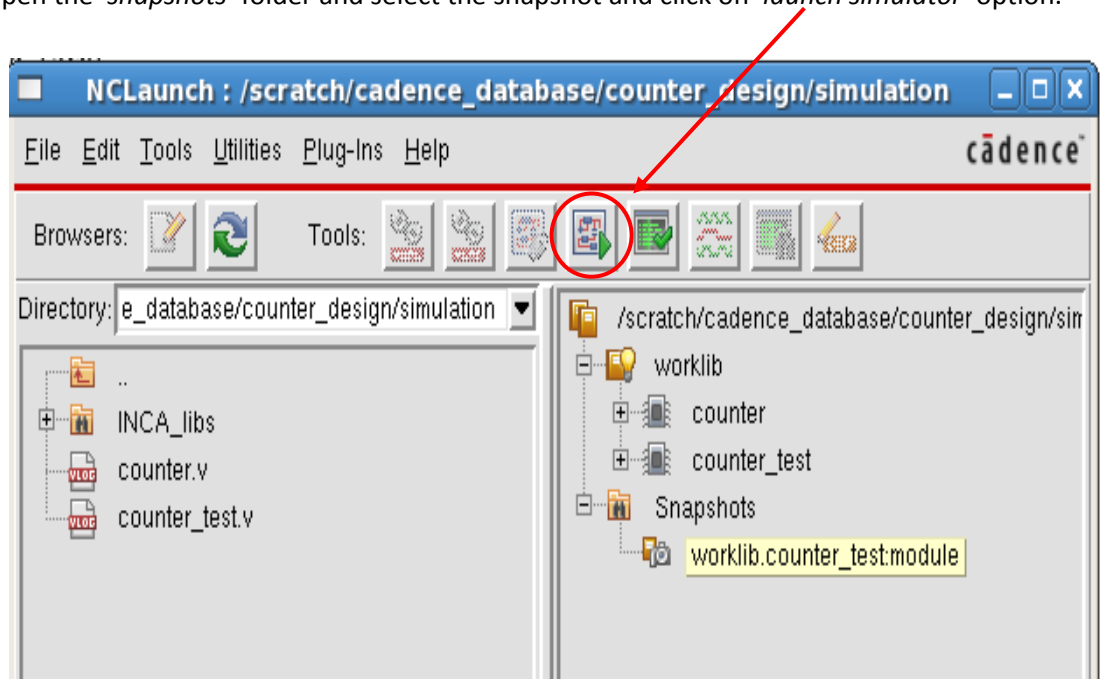
- After rectifying the errors in the code, the next step is elaboration (constructs design hierarchy and connects signals). Once the compilation is successfully completed, open the *'worklib'* directory on the right side of the window and we can see the design objects created inside. Elaboration should be performed on the testbench as testbench is the top module at this stage and design is instantiated inside the testbench. Select the testbench module and select the '*launch elaborator'* (ncelab) key.



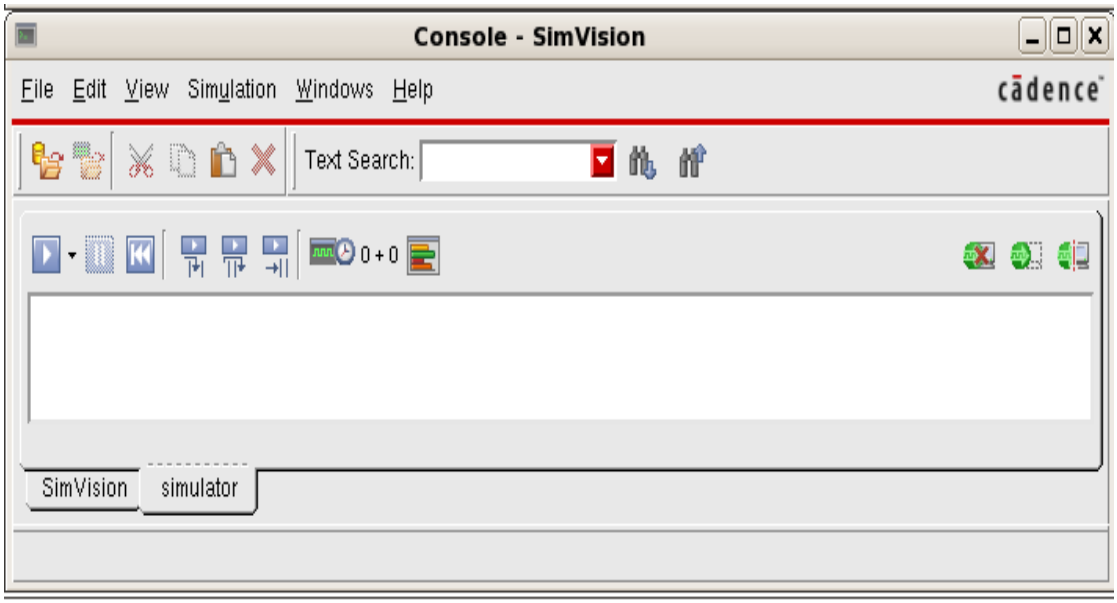
Simulation

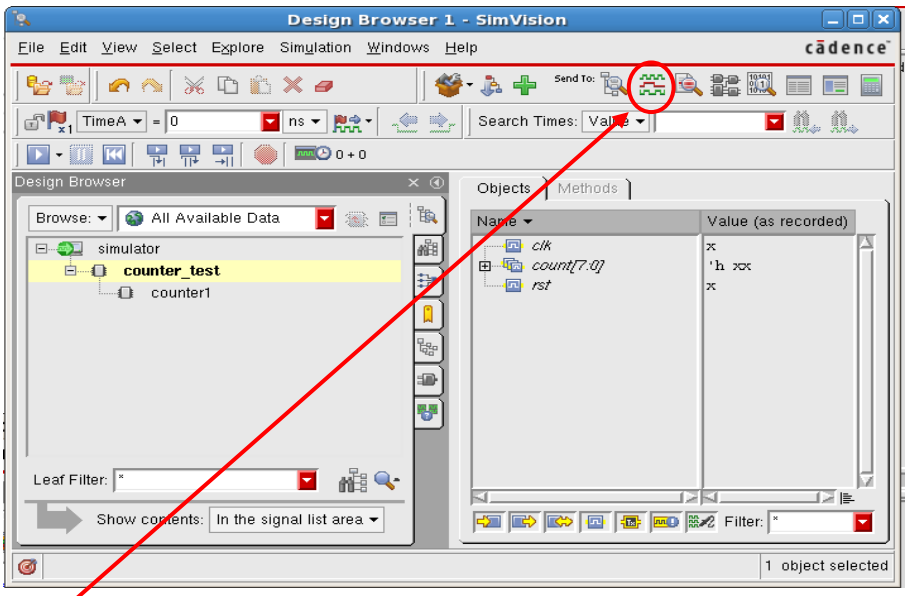
- After elaboration, the next step is simulation (executes simulation code). For this we have to send the snapshot generated during elaboration to the simulator.

- Open the *'snapshots'* folder and select the snapshot and click on '*launch simulator*' option.

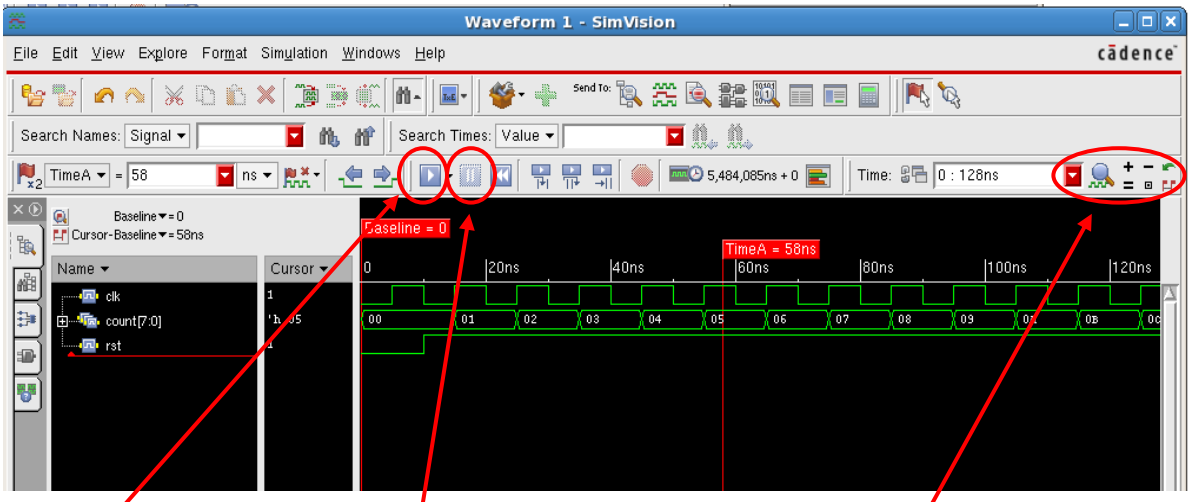


- 'Launch simulator' will open *'Design Browser'* and *'console'* windows. 'Console –SimVision' window can be used to perform simulation in command mode and hence can be minimized while using *'Design Browser –SimVision'* window to run simulation in GUI mode.





- In the Design Browser window, select the testbench module (counter_test) and select the 'waveform' option. A 'waveform –SimVison' window will appear.
- In the waveform window, we can see different ports in the design. Now click on the Run simulation key to start the simulation. Use the 'pause' key to interrupt or stop the simulation. Use different options like zoom in, zoom out etc to analyze the plot. There are many different options available in the 'Design Browser' as well as in the 'Waveform' window to analyze the design and debugging.



Run Simulation Interrupt Simulation Zoom in, Zoom out etc..

- After verifying the design, close the tools. We can now proceed for synthesis.

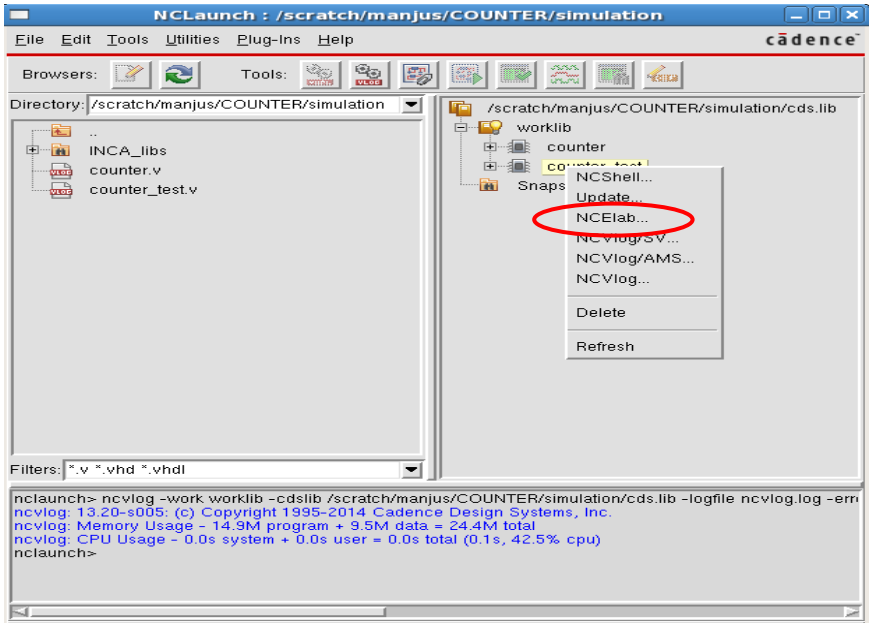
To know more about tool options, use help or pdfs present inside the doc directory of the tool.

Code coverage Flow

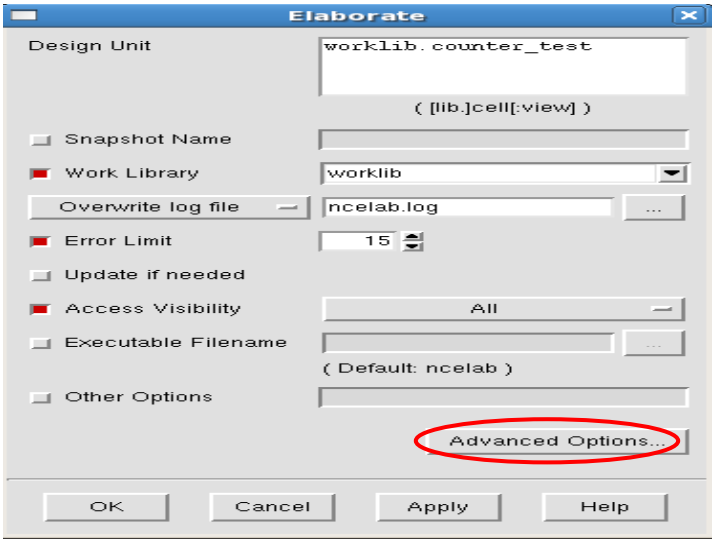
Code coverage will tell you how well your HDL code has been exercised by your test bench. In other words, how thoroughly the design has been executed by the simulator using the test stimulus you have provided in the test-bench.

For code coverage we will use IES (Incisive Enterprise Simulator) tool and ICCR (Incisive Comprehensive Coverage Reporting) tool. *ICCR* is a coverage reporting tool to merge coverage data, display textual and graphical reports for coverage data, mark coverage items, and analyze coverage data.

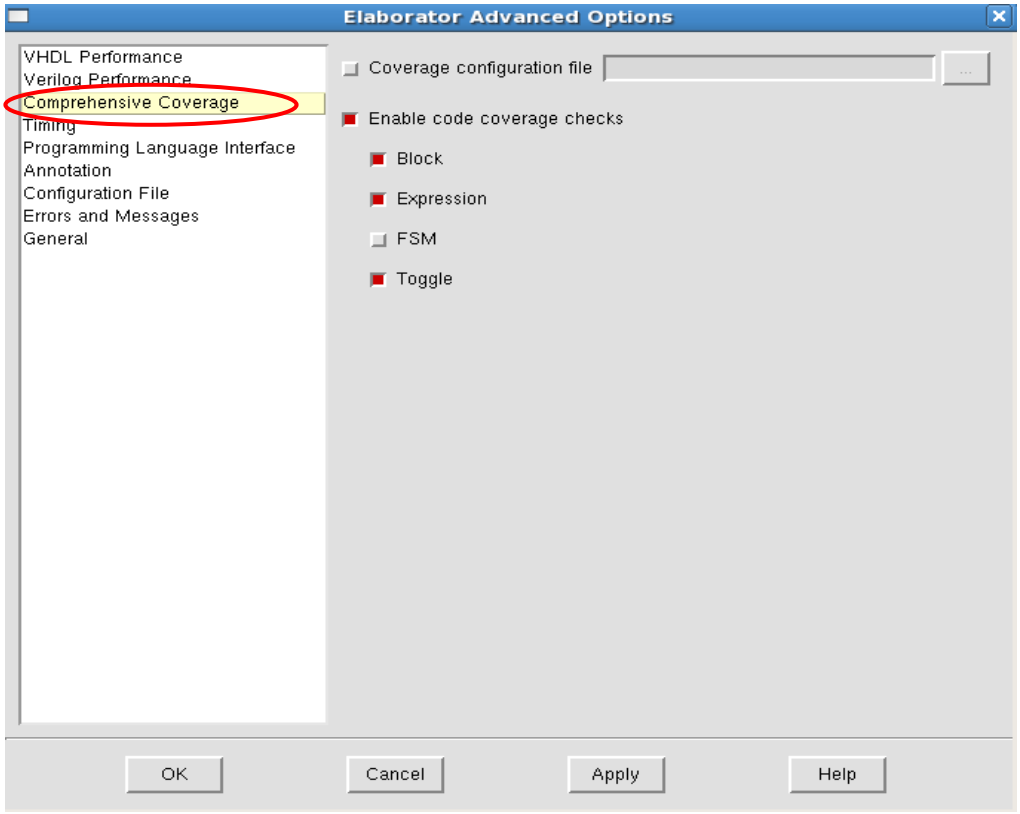
- Repeat the same steps of verification (functional simulation) up-to elaboration.
- Select the testbench module and right click on it and click on ncelab option.



- In the elaborate window click on advanced options.



- Click on the comprehensive coverage option in the elaborator advanced options window and choose all the coverage options as like below. Then click on OK and again click on OK in the elaborate window.

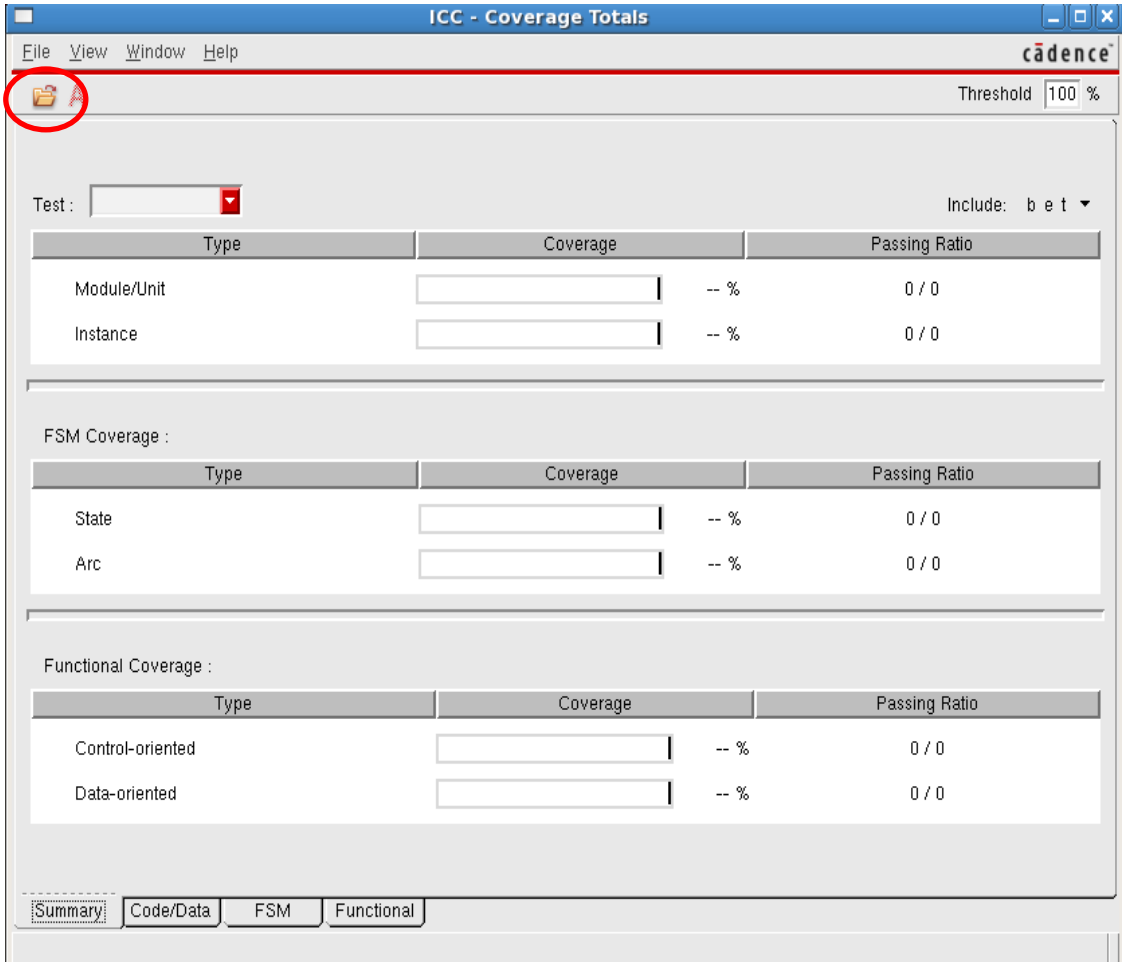


- Then repeat the simulation steps. After simulation you will get cov_work directory which results from enabling coverage during elaboration.

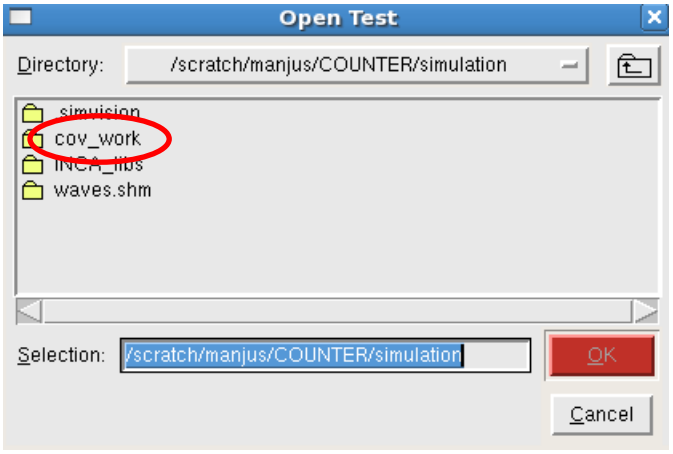
- Now invoke the ICCR tool for coverage analysis with the command: **iccr –gui**

```
[manjus@lnx-manjus simulation]$ iccr -gui
iccr: 13.20-s005: (c) Copyright 1995-2014 Cadence Design Systems, Inc.
  Using work directory: ./cov_work
  Using design: scope
  Using log file: ./iccr.log

################################################################################
iccr: *W,ICCREL: ICCR, the coverage analysis tool, has now been moved to
ntenance mode, and is no more under active development.  It will go out
roduction mode in a future release. So, the Incisive Metrics Center (IMC)
hould be used for coverage analysis.
```

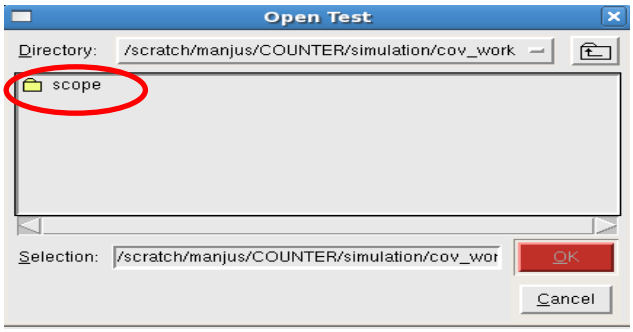
- You will get the below window. Then click on load test icon  and load the coverage result which is stored in the cov_work directory.



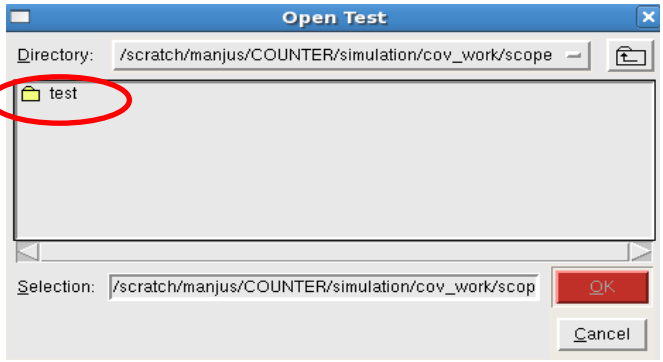
- Double click on cov_work.



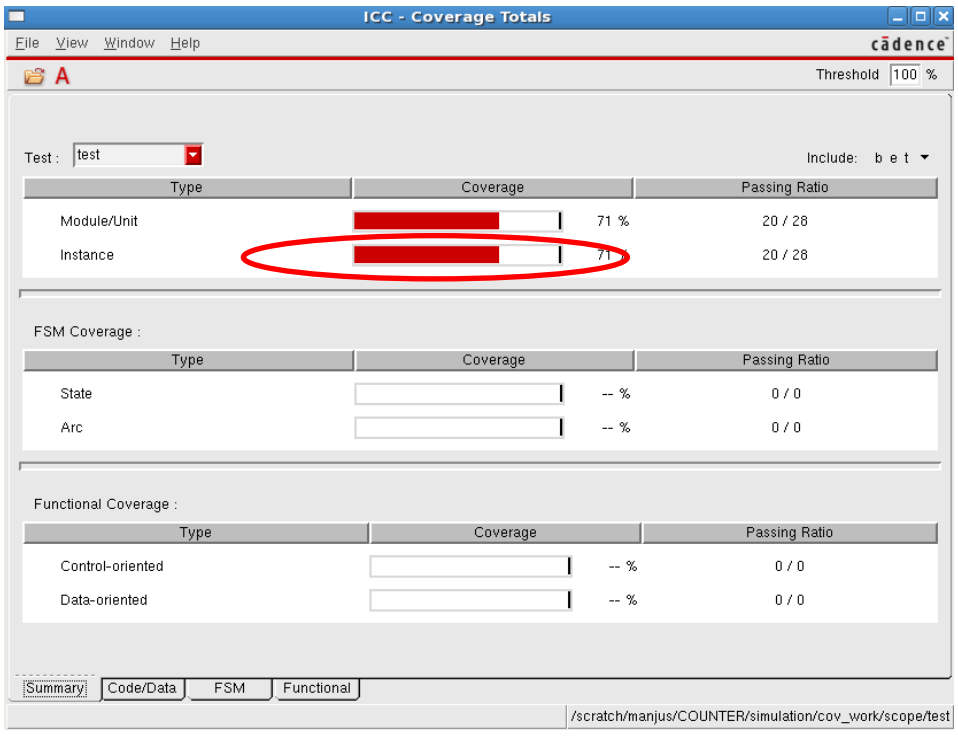
- Double click on scope.



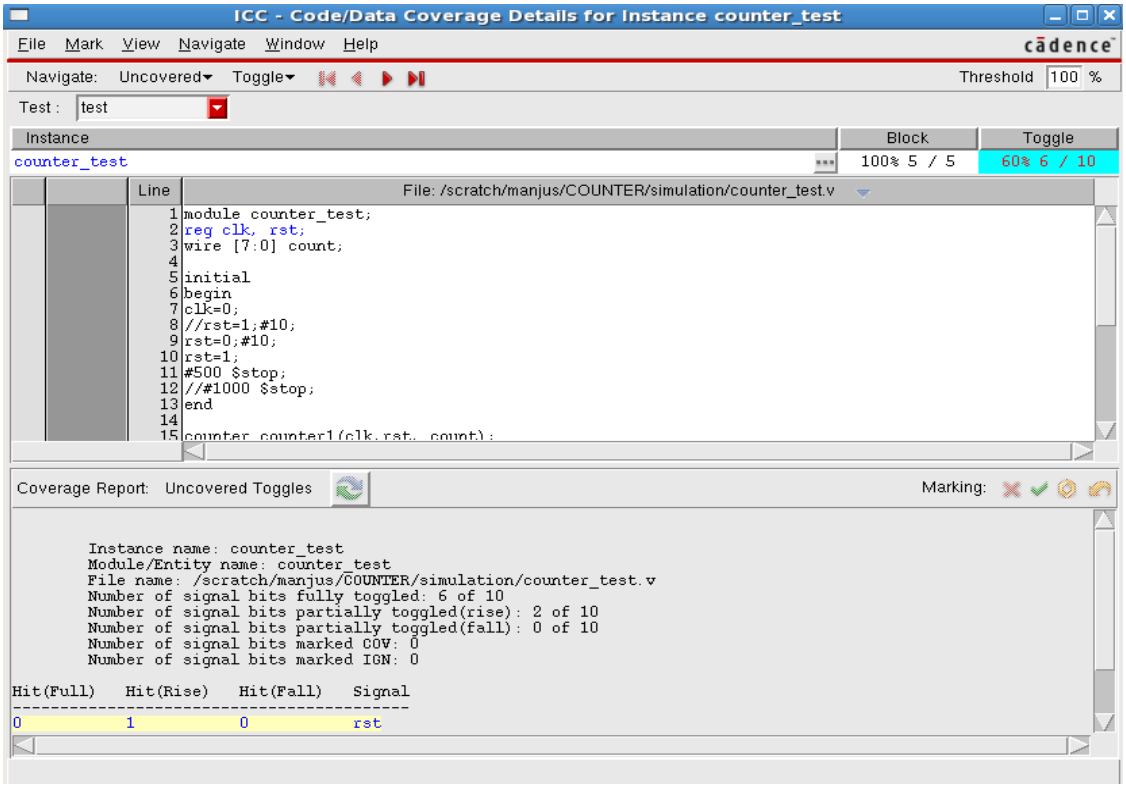
- Double click on test to load the coverage results.



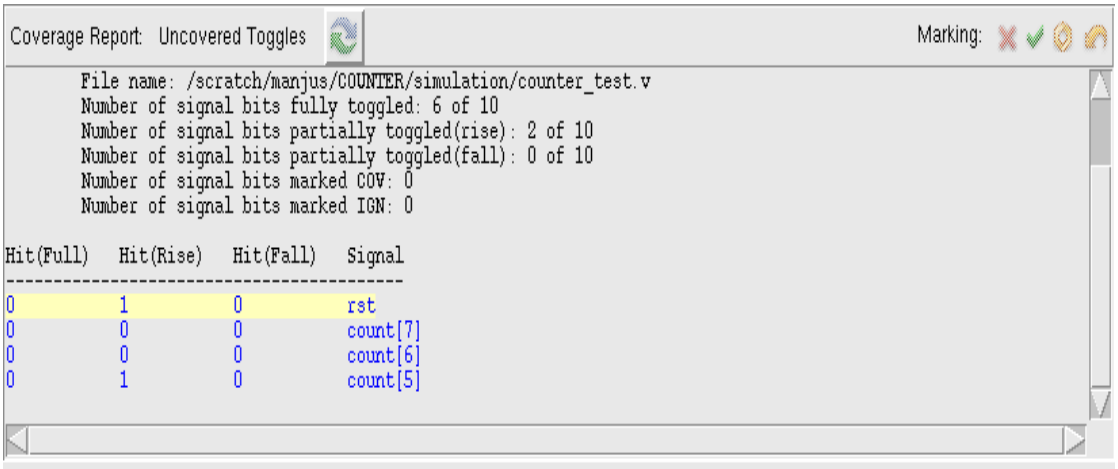
- Double click on the module or instance to get the detailed view of the individual code coverage results.



- Click on toggle options to see uncovered areas.



- Scroll down and see the uncovered toggles.



- Reset is not toggled 100% and also counter higher bits are not completely toggled. So, to cover these uncovered areas just we have to re-write the test-bench (write directed test cases/un-comment the commented lines) and re-run the simulation by enabling coverage during elaboration and analyze the coverage results with the ICCR tool.

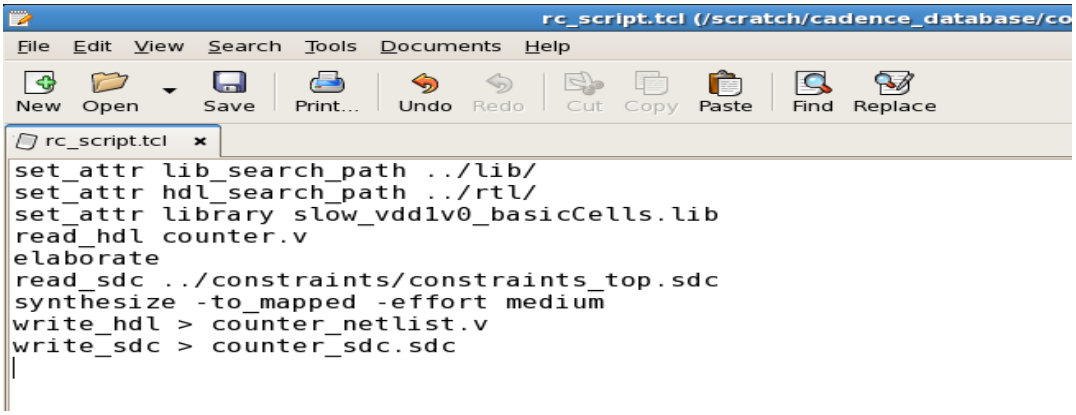
Synthesis

The tool used for synthesis (converting RTL to gate level netlist) is RTL Compiler (RC).

Running Synthesis (without DFT)

Change the directory to synthesis and write a script file for synthesis.

Below is an example of a script file for synthesis.



- The necessary inputs to perform synthesis are RTL, standard cell library and constraints.
- Let us see the usage and purpose of each command.
    - **set_attribute lib_search_path <library path>**
      This command will set the path for the standard cell library.
    - **set_attribute hdl_search_path <rtl path>**
      This command will set the path for rtl files.
    - **set_attribute library <library name>**
      This command will read the specified standard cell library from the specified library path.
    - **read_hdl <rtl design>**
      This command will read the rtl design.

      Note: - If the design is hierarchical or has multiple modules instantiated inside the top module, use curly braces '{ }' to mention all modules including the top design.

      E.g.:- read_hdl {top.v sub1.v sub2.v}
      Here top.v is the top module and sub1.v and sub2.v are the sub modules that are instantiated inside the top module.
    - **elaborate**
      The elaborate command constructs design hierarchy and connects signals.

- **read_sdc < sdc file name with path>**
  This command reads in the timing constraints file. Here we have to provide the constraints file name along with the path. Explanation on constraints file is provided

- **Synthesize –to_mapped –effort medium**
  This command will perform synthesis by combining the generic, mapped and incremental synthesis and – effort medium command specifies the synthesis effort. The effort can be set to 'low', 'medium' or 'high' depending upon the scenario.

➢ Include all the above commands in the script file.

Note: - In counter design, you can see a script file 'rc_script.tcl' inside the synthesis directory. Please open the script file for further understanding.

Timing Constraints or SDC file

Now let us understand the content of the Constraints or SDC file.

➢ Using SDC, we define clock period, pulse width, rise and fall time, uncertainty and also input and output delays for different signals. Below is the constraints file used in counter design.

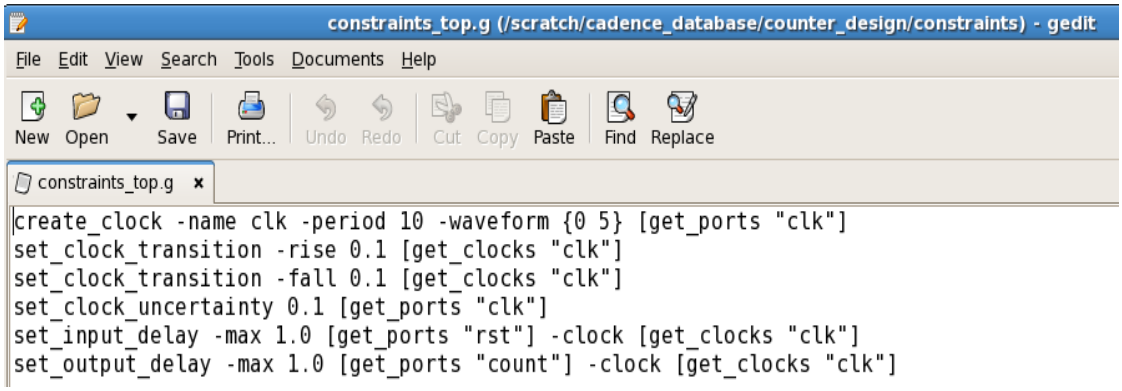


Fig 14

➢ Let us see the usage and purpose of each command.

- Create_clock –name –period 10 –waveform {0 5} {get_port "clk"}
  This command will define clock with period 10ns and 50% duty cycle and signal is high in the first half.
- 'Set_clock_transition –rise/fall' command defines the transition delay for clock.
- 'Set_clock_uncertainty' command will set the uncertainty due to (clock skew and jitter).
- 'Set_input/output_delay' command will specify the input and output delay used for timing slack calculations.

➢ Keep the constraints file inside the constraints directory.

Note: - To know more about writing timing constraints, please refer the rc_ta.pdf available inside the doc/rc_ta directory of the tool.
I.e. - <RC_tool directory>/doc/rc_ta/rc_ta.pdf

Once the script file to run synthesis and the constraints file are ready, we can initiate synthesis.

- Use the below command to invoke RTL compiler along with the script file.

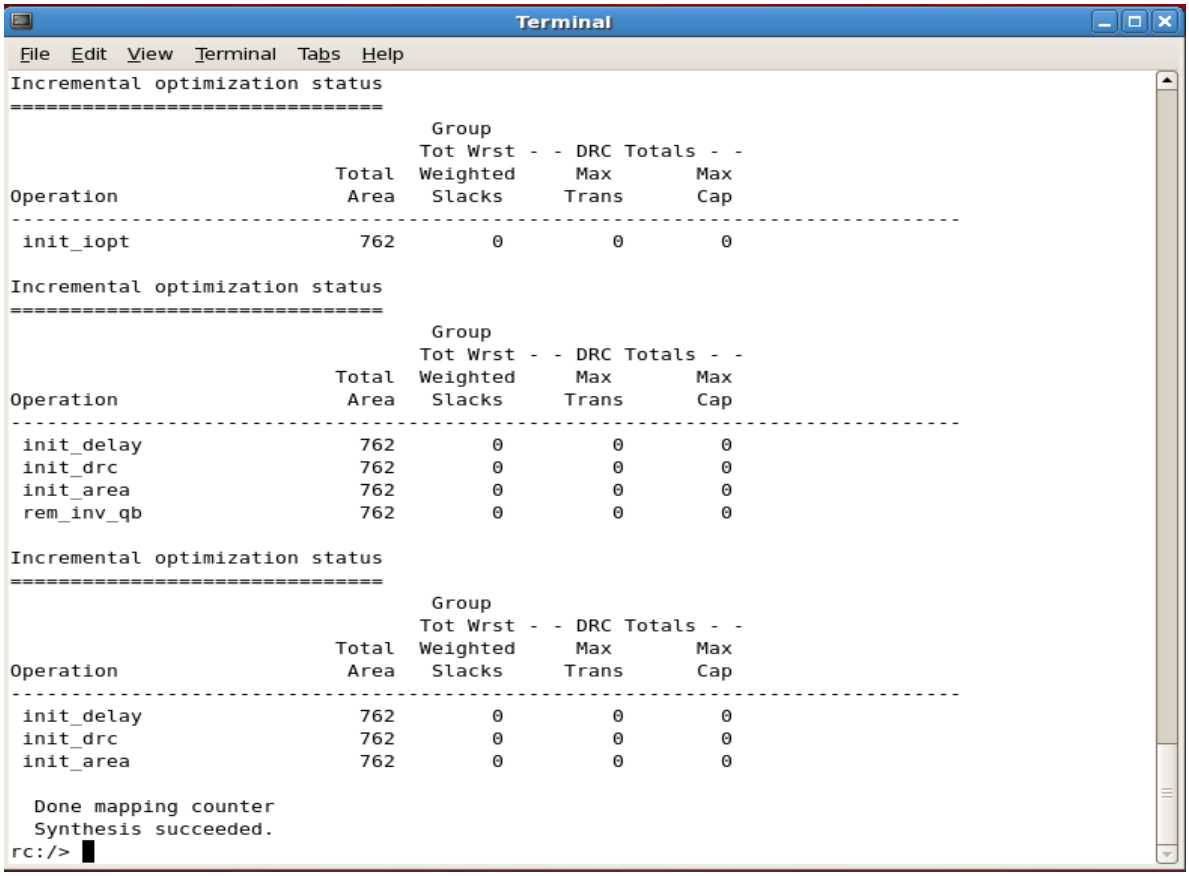
  rc -f <script file name with path>

  'rc' is the command to invoke RTL Compiler and '-f' option is used to passes the script to RC at the time of launching the tool. RC will execute each commands mentioned inside the script file one by one.

  Note: - If the script file is in the current working directory (synthesis directory), we need not have to provide the path for the script.

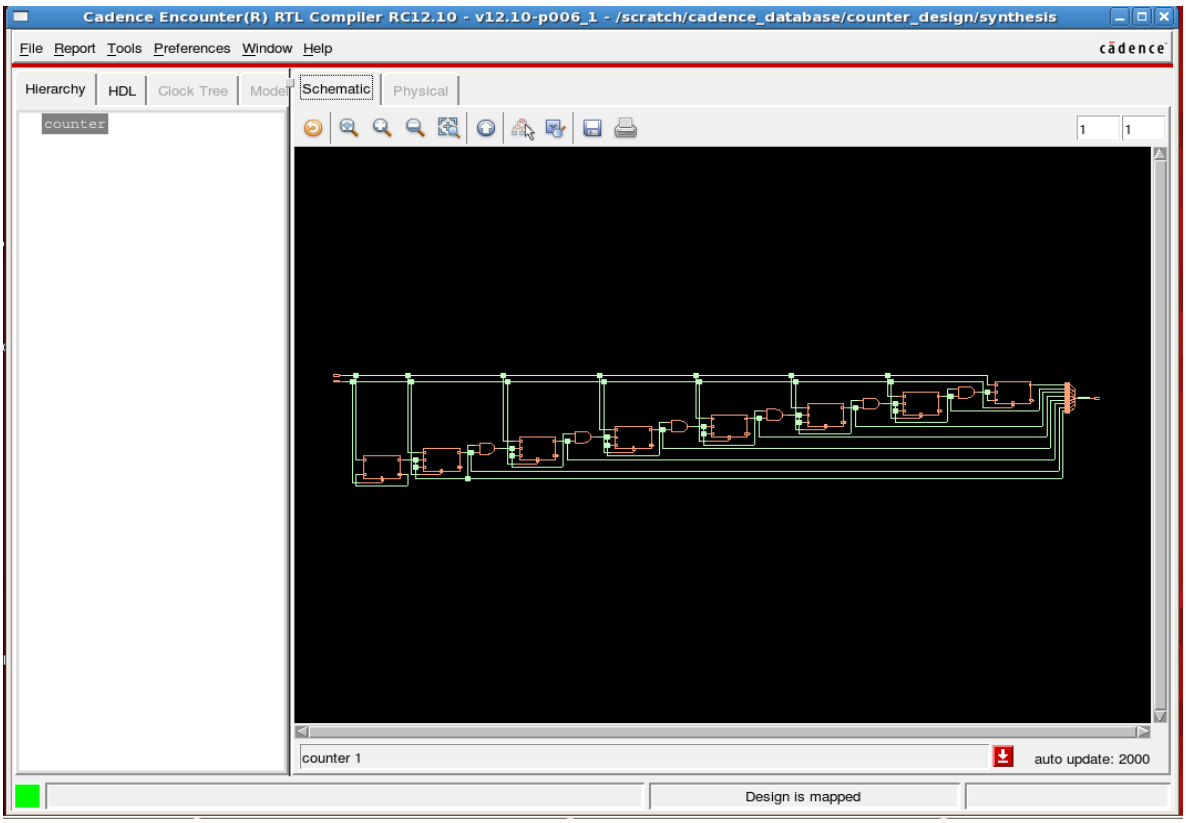
  E.g. - In case of the counter design, the command will be 'rc –f rc_script.tcl'

- While performing synthesis, always check the RC terminal whether the tool is reporting any error. Figure below shows the RC terminal after synthesis.



- After synthesis, to see the schematic, launch the gui using the below command.

  'gui_show' in the *'rc'* terminal. Use ***'gui_hide'*** command to close the gui.



- Use 'report' command to write out the results.
  - ***'report timing'*** to report the timing details.
  - ***'report power'*** to dump out the power report.

Note: - Use just report command to know what all different reports you can dump out from RC.

- After completing synthesis, Use the below command to dump out netlist, SDF, SDC etc for next stages of the slow.
  - ***write_hdl > netlist_name.v***
    You can provide any name for the netlist file but the extension of the file should be *'.v'*
  - ***write_sdc > sdc_name.sdc***
    You can provide any name for the SDC file but the extension of the file should be *'.sdc'*
  - **write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge > delays.sdf**

    timescale: used to mention the timeunit.
    nonegchecks: used to ignore the negative timing checks.
    recrem: used to split out the recrem(recovery-removal) timing check to separate checks for recovery and removal.
    edges: Specifies the edges values.
    check_edge Keeps edge specifiers on timing check arcs but does not add edge specifiers on combinational arcs.

The above commands will generate netlist, SDF and SDC in the synthesis directory.

- Use ***'exit'*** command to close RC.

  Note: - To know more about synthesis, please refer the rc_user.pdf available inside the doc/rc_ta directory of the tool.
  I.e. - <RC_tool directory>/doc/rc_user/rc_user.pdf

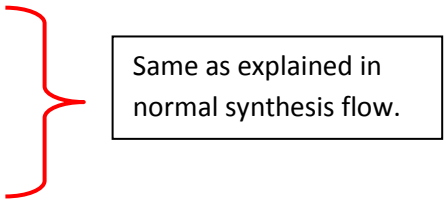
  After dumping out the netlist and SDC, we can proceed for Physical Design. Please close the RC tool.

Synthesis with DFT

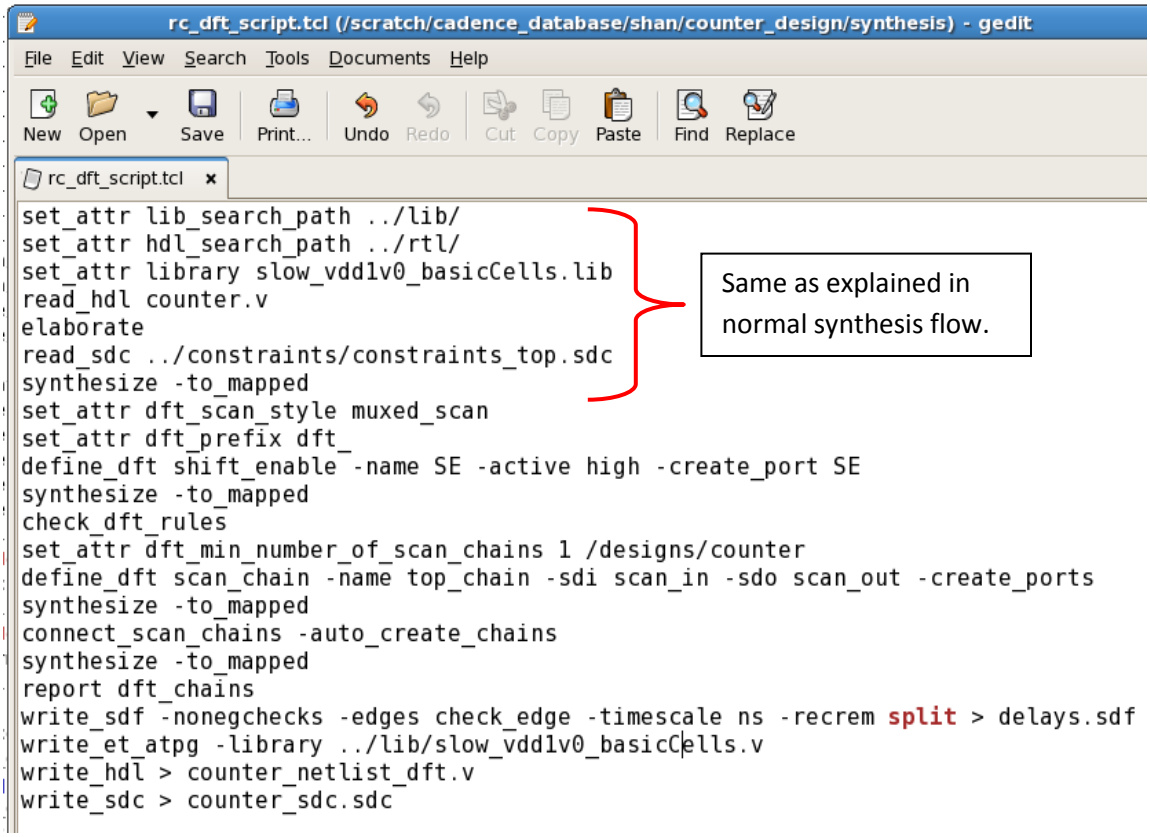
Understanding the Flow

- Let us look at the files we are working with in this lab. Change directory to synthesis and locate the **rc_dft_script.tcl**. The main purpose of this script is to set up the variables and libraries that will be used in the lab.
- Now let us look at the content of the run script(rc_dft_script.tcl). Here is a breakdown of the script flow for clarity:

  - Load all the design files and elaborate
  - Read sdc (in constraints dir)
  - Read in DFT setup
  - Synthesize to GENERIC
  - Synthesize to MAPPED
  - Run DFT flow
  - Synthesis to MAPPED
  - Write results and database
  - Write ET files and ATPG flow



Below snapshot shows rc_dft_script.tcl



Let us understand the DFT specific commands in rc_dft_script.tcl file.

- The DFT scan FF style for scan replacement using the below command.
  set_attr dft_scan_style muxed_scan
- Prefix is added to name of DFT logic that is inserted using the below command.
  set_attribute dft_prefix DFT_ /
- Define the test signals (define_dft shift_enable) using the below command.
  define_dft shift_enable -name {scan_en} -active {high} -create_port {scan_en}
- It is recommended you check DFT rules multiple times during a DFT flow using the below command**.**
  check_dft_rules

```
        Summary of check_dft_rules
        **************************
        Number of usable scan cells: 48
Clock Rule Violations:
----------------------
          Internally driven clock net: 0
              Tied constant clock net: 0
                   Undriven clock net: 0
        Conflicting async & clock net: 0
                        Misc. clock net: 0

Async. set/reset Rule Violations:
---------------------------------
        Internally driven async net: 0
              Tied active async net: 0
                 Undriven async net: 0
                    Misc. async net: 0

   Total number of DFT violations: 0

  Total number of Test Clock Domains: 1
  Number of user specified non-Scan registers:    0
      Number of registers that fail DFT rules:    0
      Number of registers that pass DFT rules:    8
  Percentage of total registers that are scannable: 100%
```

As you can see that there are no registers that fail DFT rules, which means that all of 8 registers are eligible for scan connection.

- Specify the number of scan chains required to connect all FF's using the below command. Here we have used 1 scan chain.
  set_attr dft_min_number_of_scan_chains 1 /designs/counter
- Specify the scan in and scan out ports of the scan chain using the command
  define_dft scan_chain -name top_chain -sdi scan_in -sdo scan_out -create_ports
- Once scan ports has been created, perform the technology depended synthesis using the below command.
  synthesis –to_mapped.
- Now connect the Scan chains using connect_scan_chains RC command. This will include all original FF's that were mapped to scan flops.
  connect_scan_chains -auto_create_chains
- You can view the dft chains using the below command as shown.
  report dft_chains

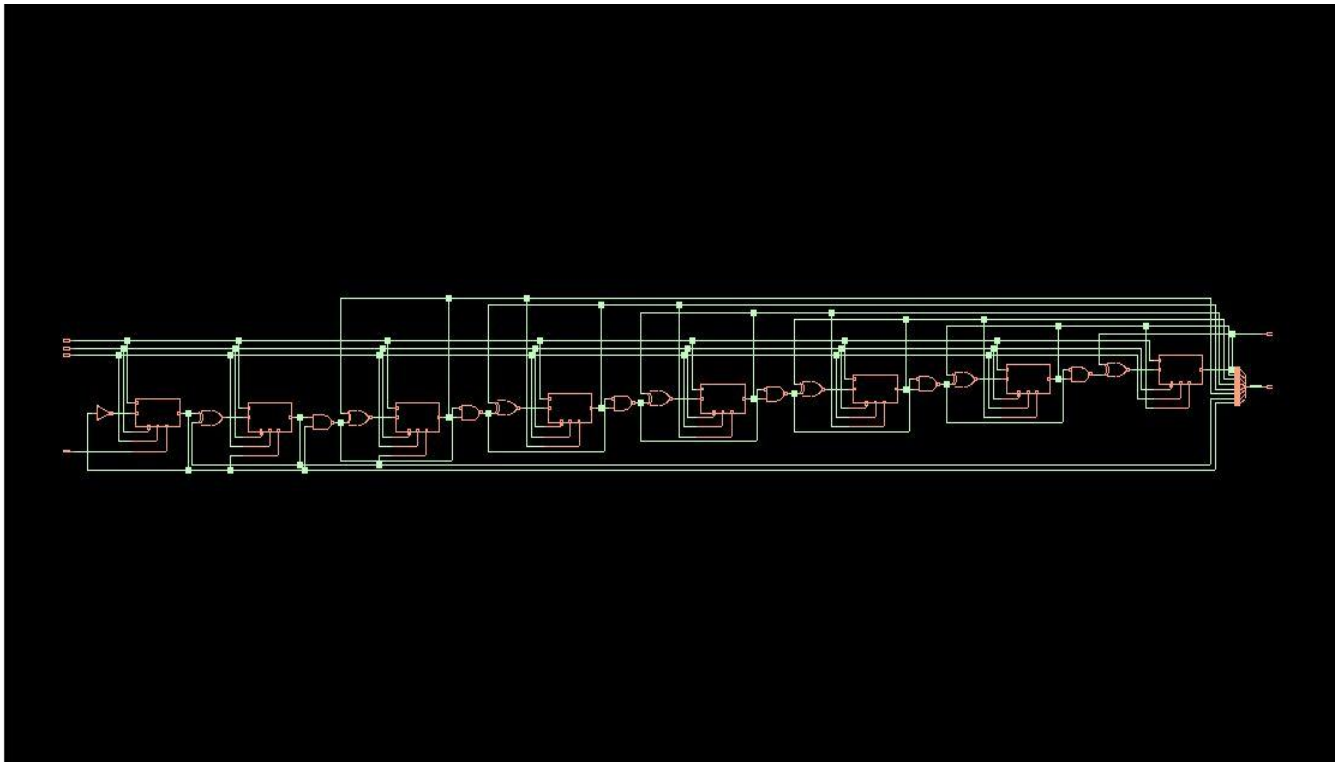
```
  Reporting 1 scan chain (muxed_scan)

  Chain 1: top_chain
     scan_in:       scan_in
     scan_out:      scan_out
     shift_enable: SE (active high)
     clock_domain: clk (edge: rise)
     length: 8
        bit 1  count_reg[0]   <clk (rise)>
        bit 2  count_reg[1]   <clk (rise)>
        bit 3  count_reg[2]   <clk (rise)>
        bit 4  count_reg[3]   <clk (rise)>
        bit 5  count_reg[4]   <clk (rise)>
        bit 6  count_reg[5]   <clk (rise)>
        bit 7  count_reg[6]   <clk (rise)>
        bit 8  count_reg[7]   <clk (rise)>
  -------------------------
```

- We will now run the final ATPG analysis and vector generation. This step will take the final scan chains and run through the ET flow for basic ATPG. This flow is implemented by the command
  write_et_atpg -library ../Lib/slow_vdd1v0_basiccells.v

  It will generate a directory '**et_scripts'** in current working location.

- Write out the final netlist, SDF & constraints using the below commands.

  Write_hdl > counter_netlist.v
  write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge > delays.sdf
  Write_sdc > count_sdc.sdc



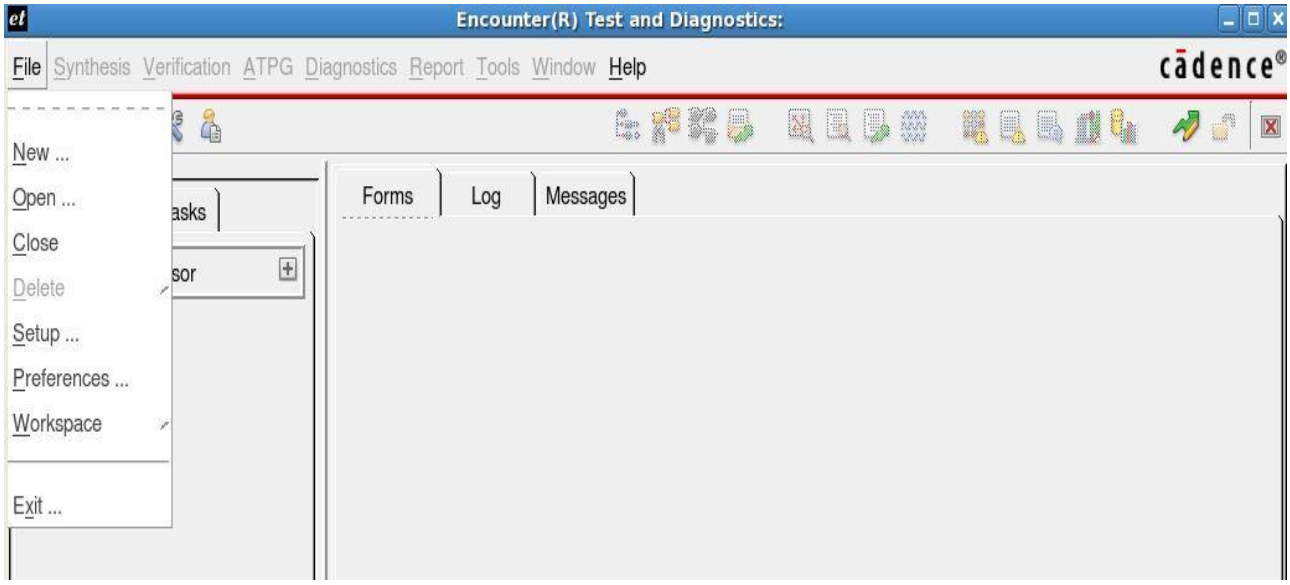
Change directory to et_scripts to see the files that are generated by RC.

- counter.et_netlist.v (completed verilog netlist used for ET)
- runet.atpg (ET ATPG run script)
- counter.FULLSCAN.pinassign (ET file specifying IO test behavior)
- et_check.sh (self error checking file used by runet.atpg)
- run_fullscan_sim (NC-sim script to verify ATPG patterns)

Encounter Test

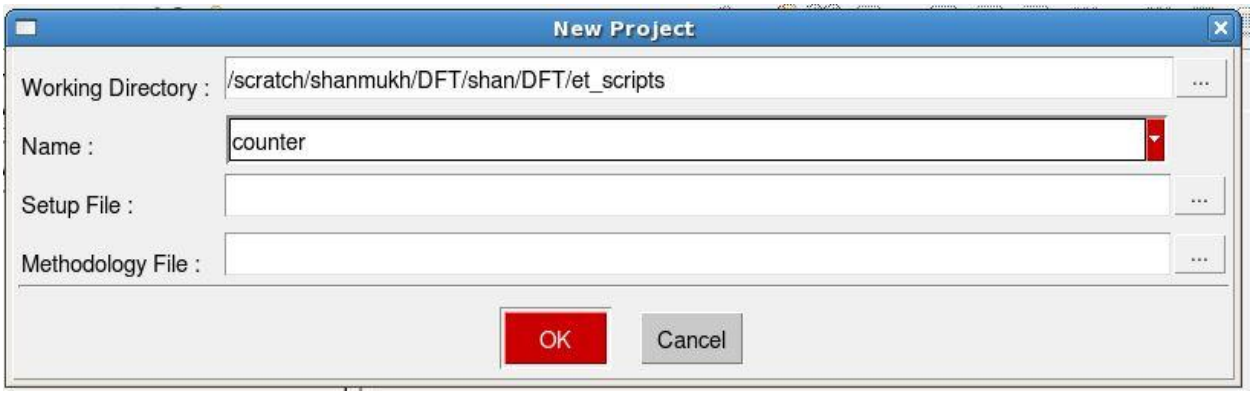
Encounter Test is the tool used to verify test logic inserted in the netlist during the Synthesis stage. Invoke the Encounter Test tool inside et_scripts directory using the command **et –gui &** or **et &**

- **Create New Project** (This step registers this project and allows you to bring up this design from any directory with the GUI).
    - Select **File -> New** as shown below

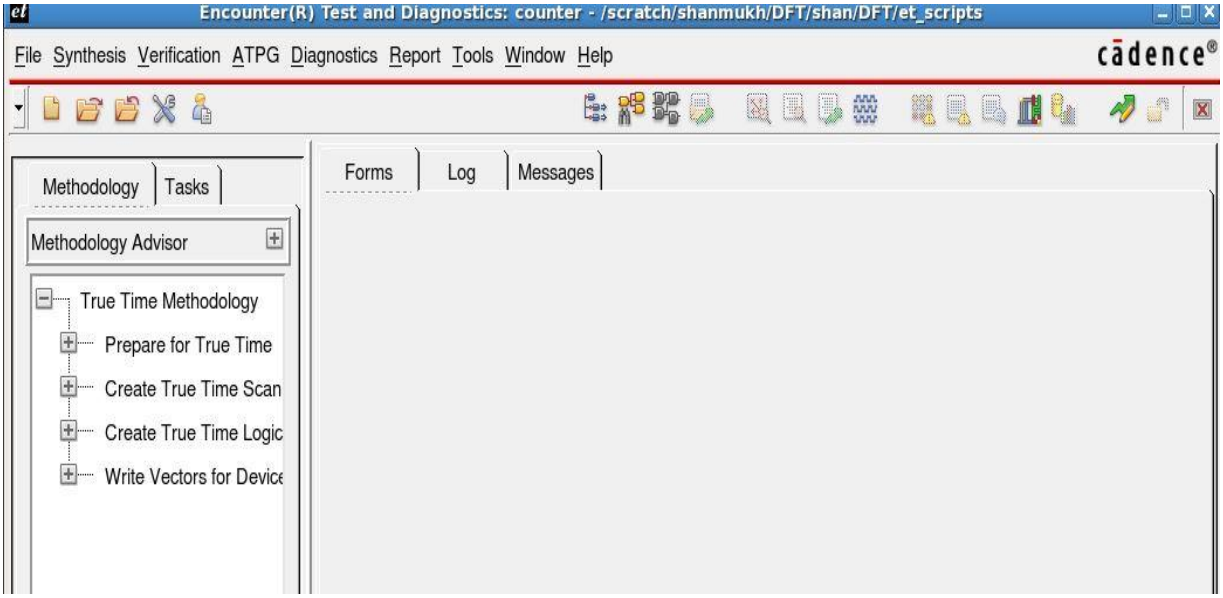


You will get a new project pop-up window.

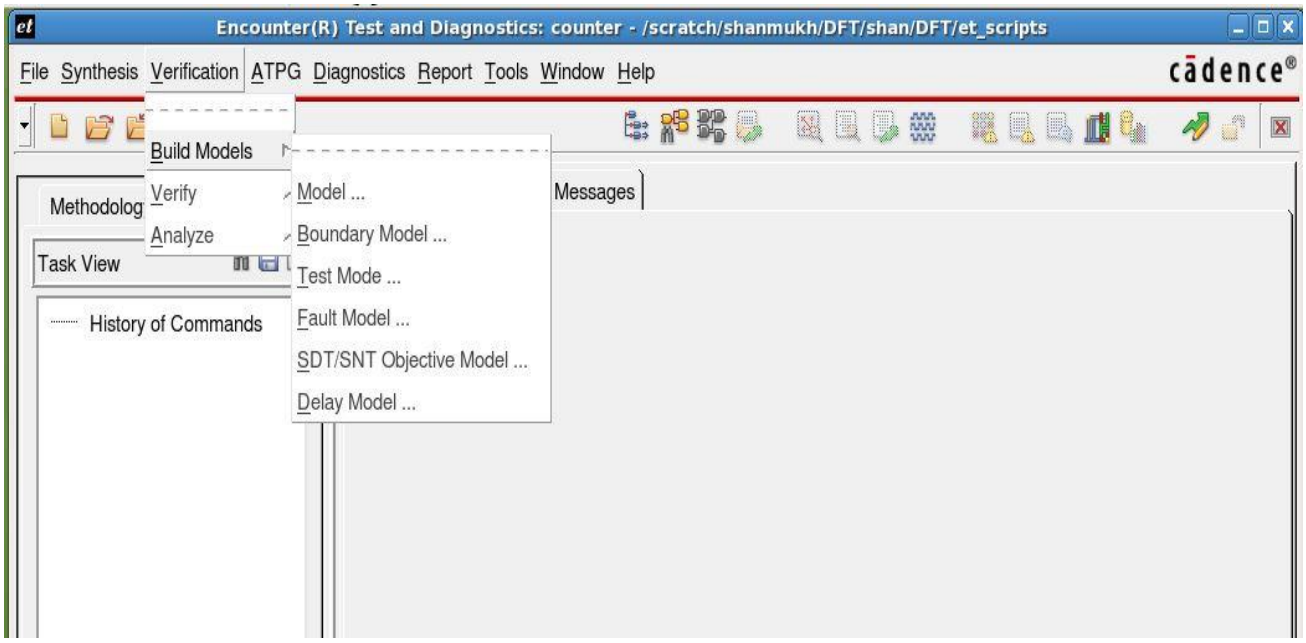
- **Working Directory** = <path>/counter/et_scripts (should be filled in for you)
- Name **counter** (This can be anything and will be a unique name for this particular project)
- Setup and Methodology are blank. If you click on the methodology tab you will see a number of default methodologies for your use. These are not needed for this exercise since we will be running our commands and flow from the pull down menus.
- Click **OK** to create a new project



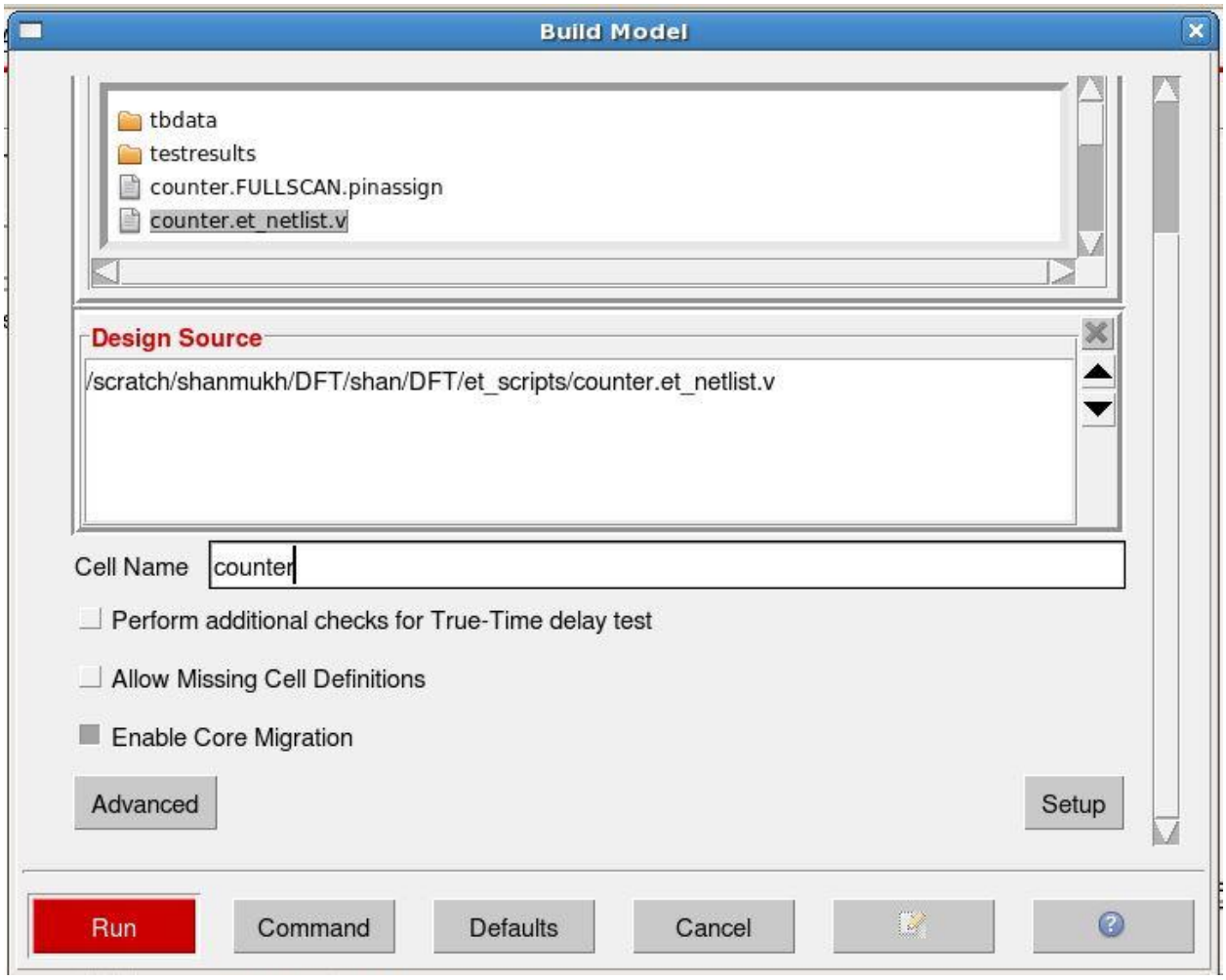
- Once the Project has been created, the gui will get modified as shown below.



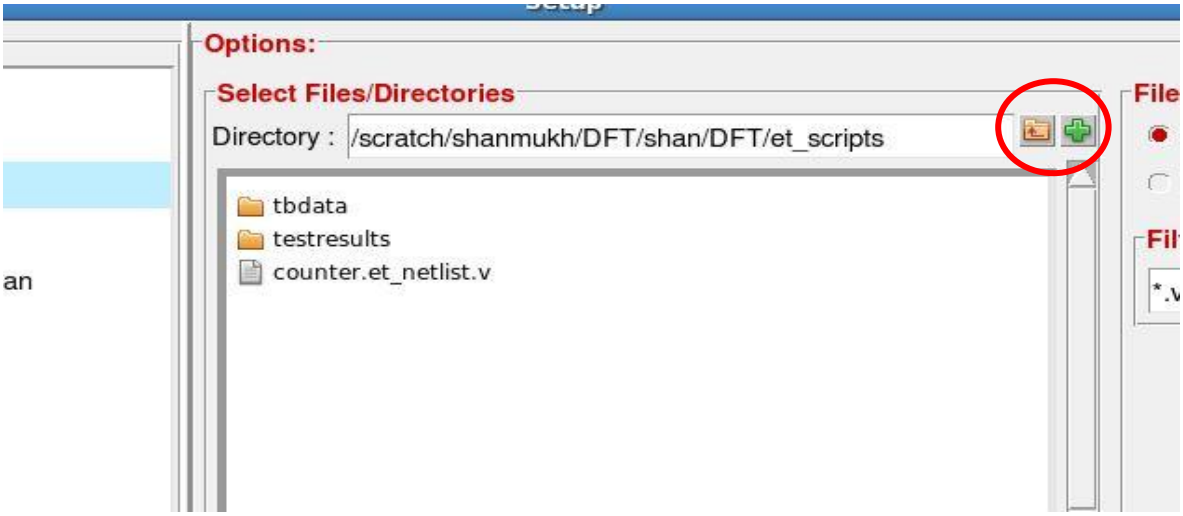
- **Build Model** – This step will read in your netlist and libraries and compile them into an Encounter Test binary model to be used for all future steps.
  - **Verification -> Build Models -> Model**



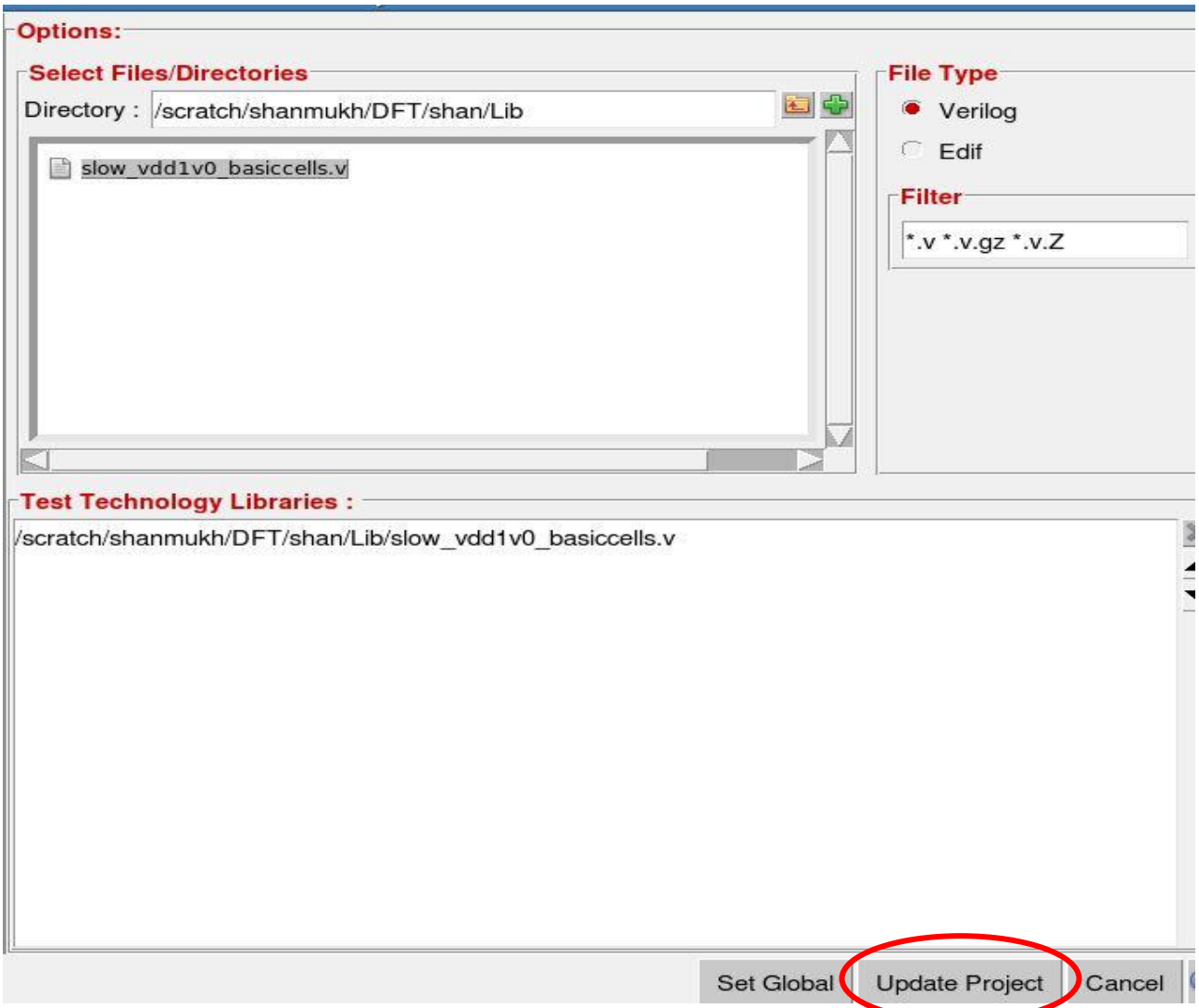
- Build model pop-up window will be appear, now select the Design Source: - counter.et_netlist.v (Click on counter.et_netlist.v to add to the Design Source pane) and specify the cell Name as counter (The top module name of your design) as shown below.



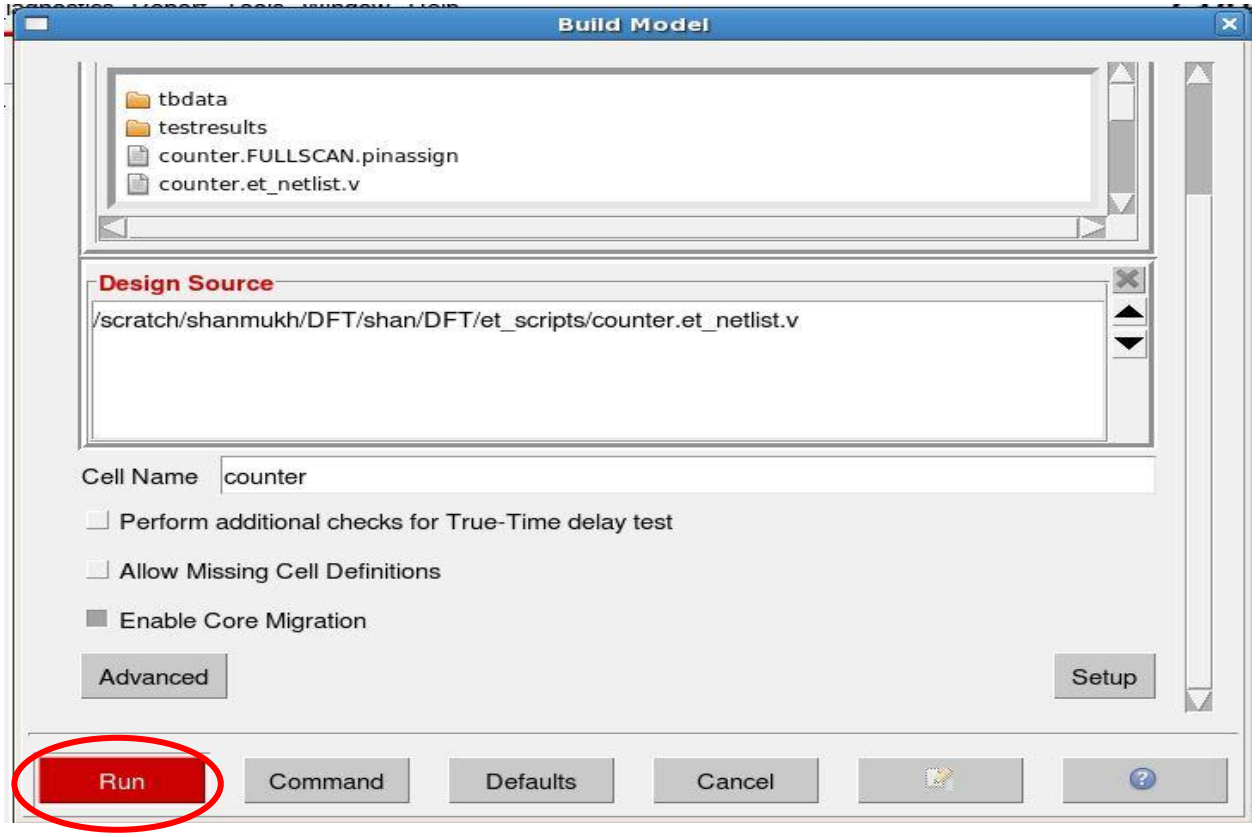
- Click on the Setup button (right side bottom) to add library technology, Browse the technologies file from the browse folder as shown in the below snapshot.



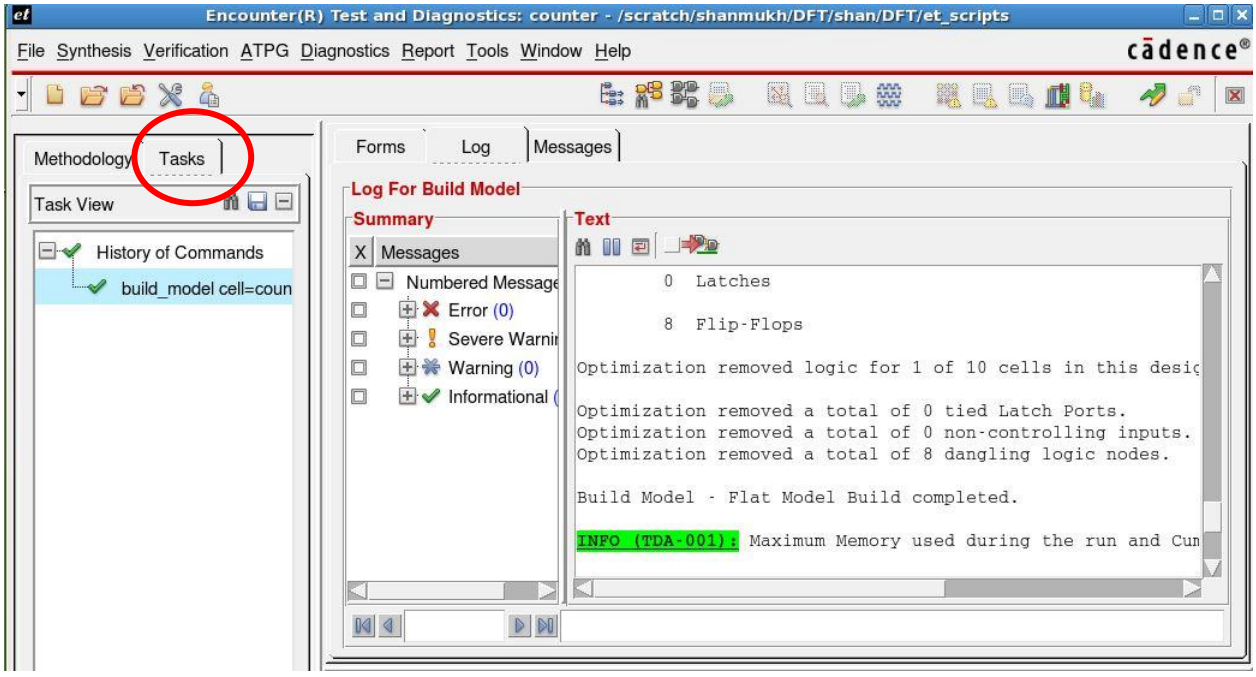
- Add the **slow_vdd1vo_basiccells.v** Library and Click on **Update Project** to save your edits as shown below.



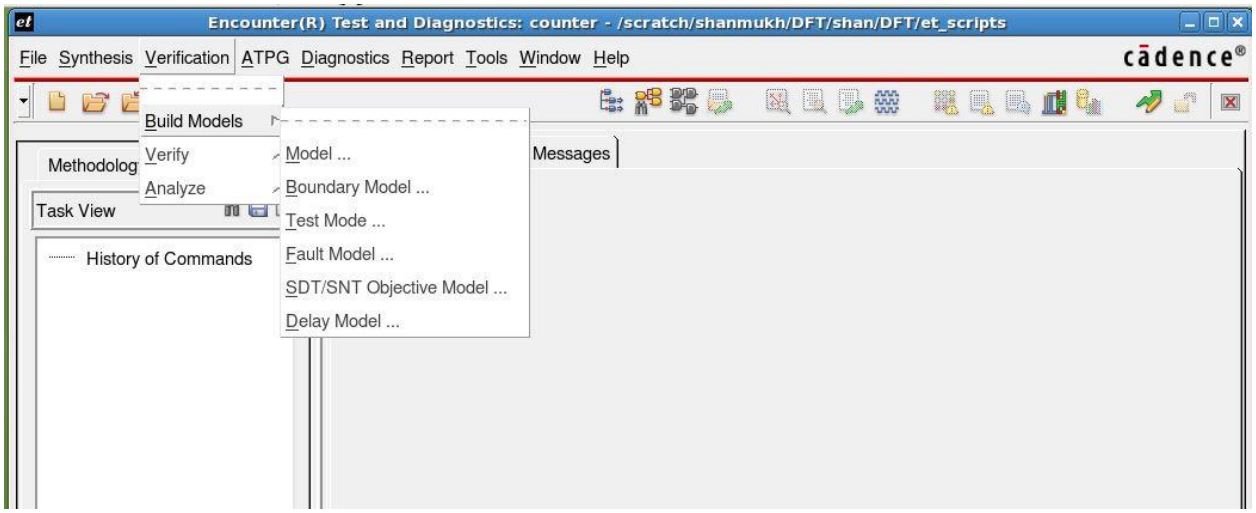
- This will bring you back to the Build Model form and click → **RUN** in the Build Model.



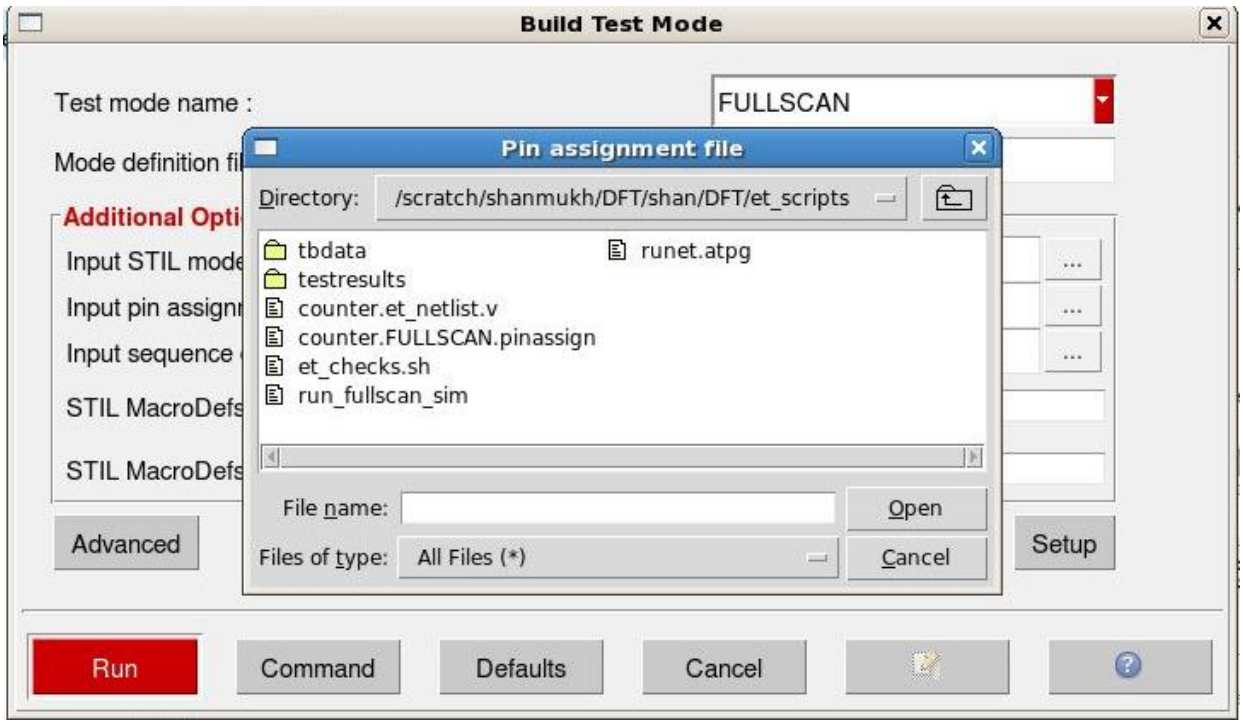
- Click on the **Tasks** Tab to see the running progress of the command. If you click on any command in the **Task** Pane, you can view the log file in the **Log Tab**.



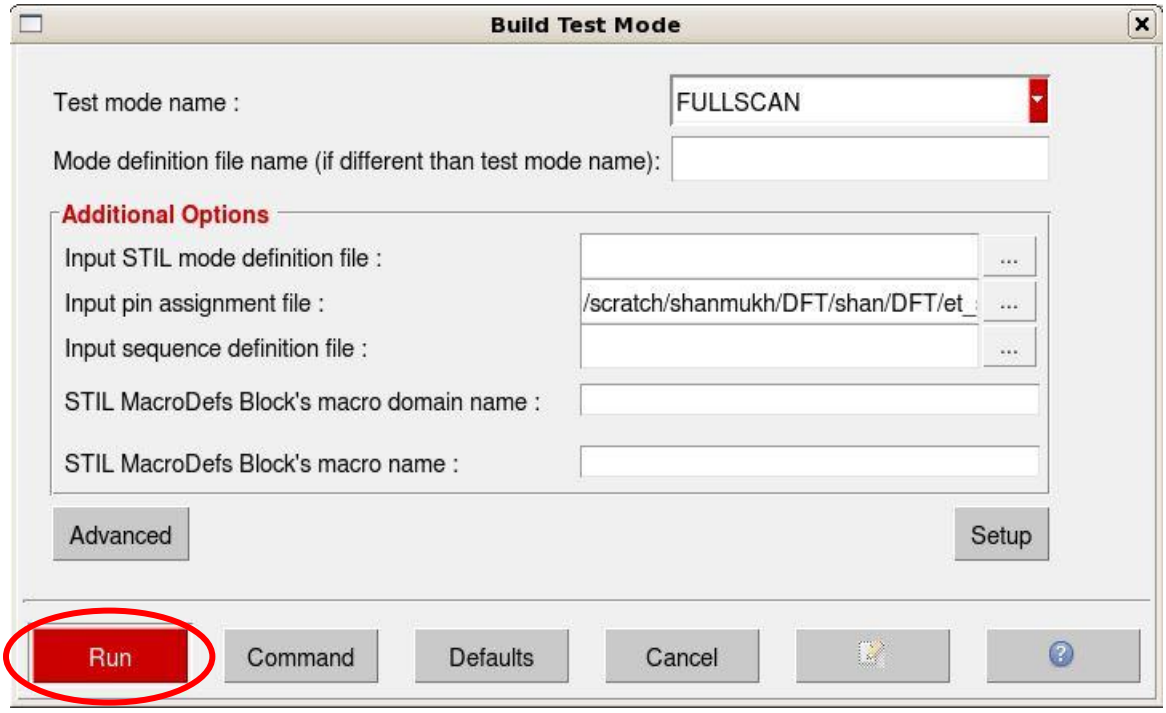
- **Build TestMode** (Creates a "test view" of your circuit based on the pin assignments set in the assignfile)
    - **Verification -> Build Models -> Test Mode**



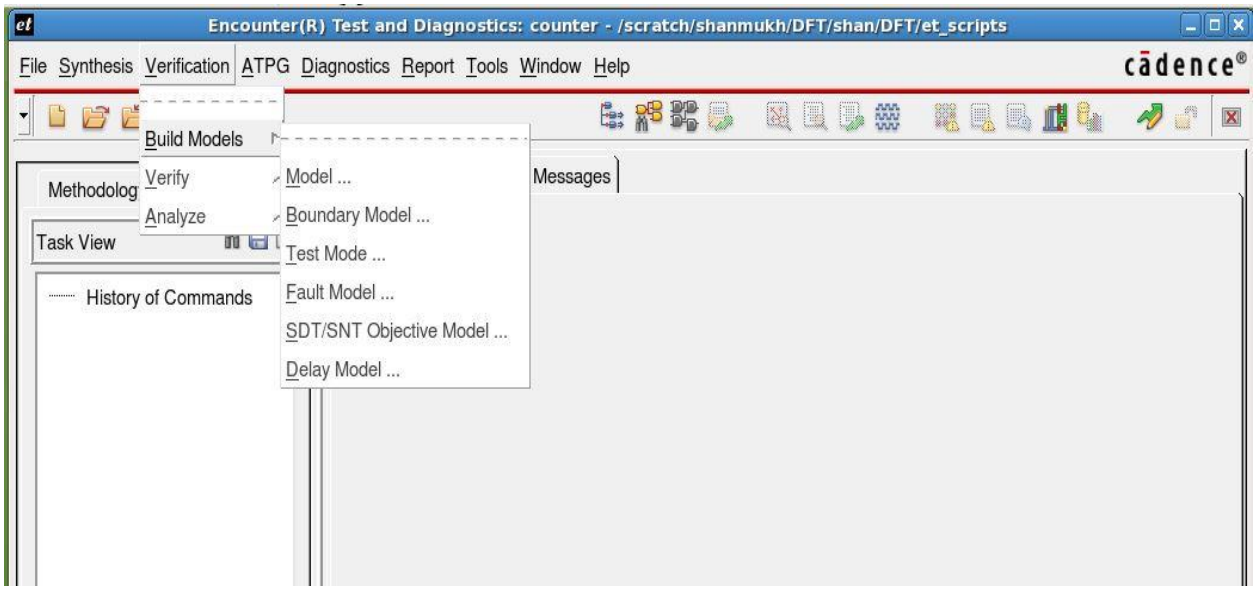
In the **Build Test mode window,** Specify the test mode name as **FULLSCAN**(This is a default mode for an ATPG SI to SO configuration) and browse the Input pin assignment file counter.FULLSCAN.pinassign from the et_secripts directory as shown below.



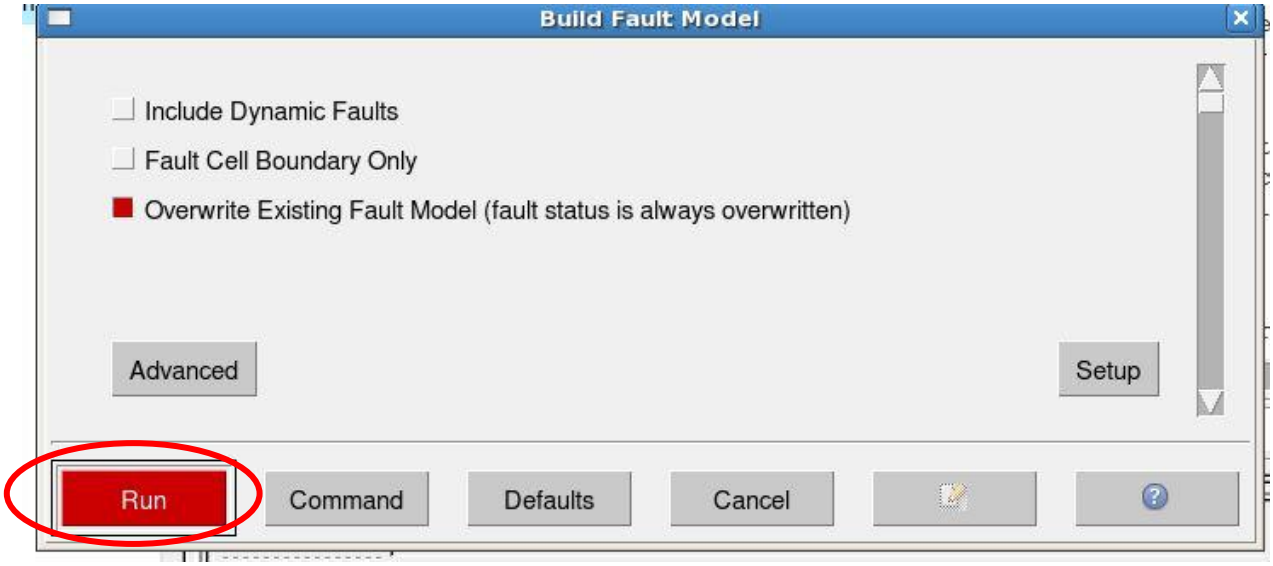
- Click **Run** in the **Build Test Mode** window and verify how many Static faults are active in this mode.



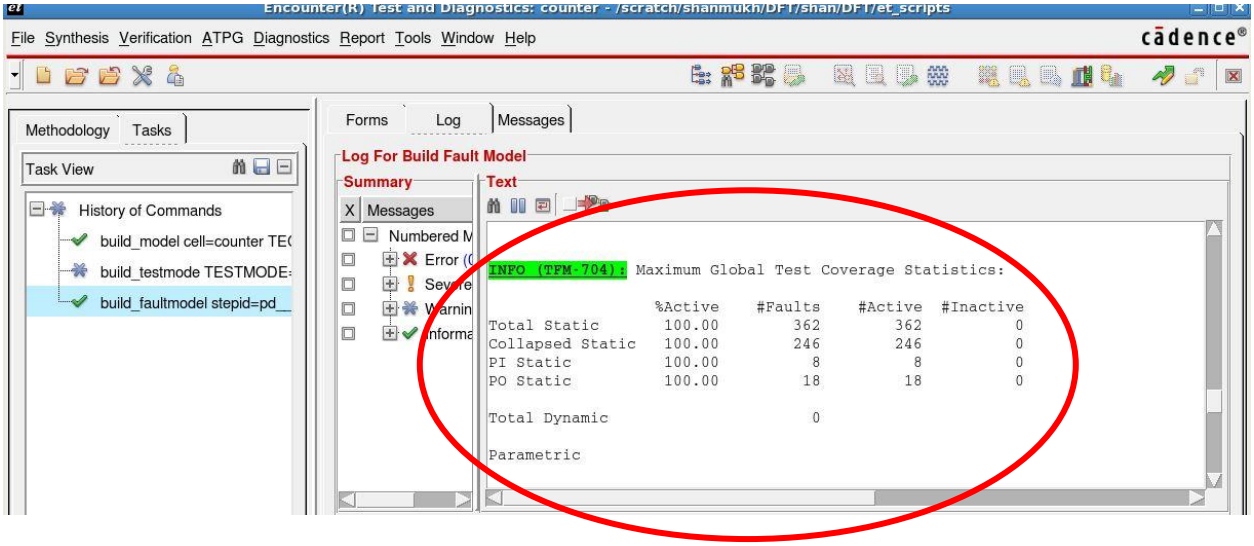
- **Build Fault Model** – This step takes the model created in the previous step and applies a fault model to the circuit by placing faults on the appropriate nodes.
  - **Verification -> Build Models -> Fault Model**



- Check boxes allow you to turn on/off **Dynamic Faults.** If selected the tool will apply the static and dynamic fault model to the circuit and click → **Run**

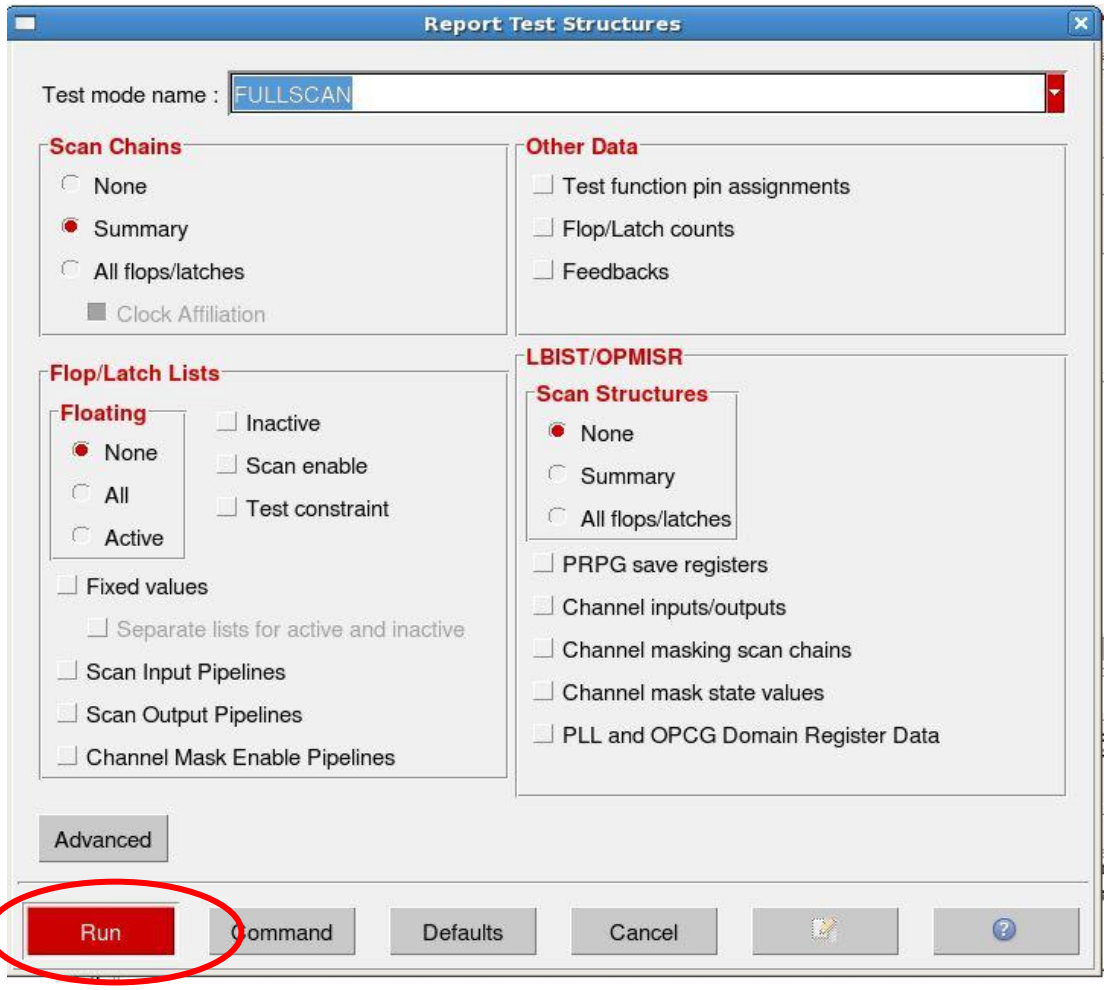


- Verify the log file containing information about the Static Faults present in the entire design and you can see the same in the below figure.

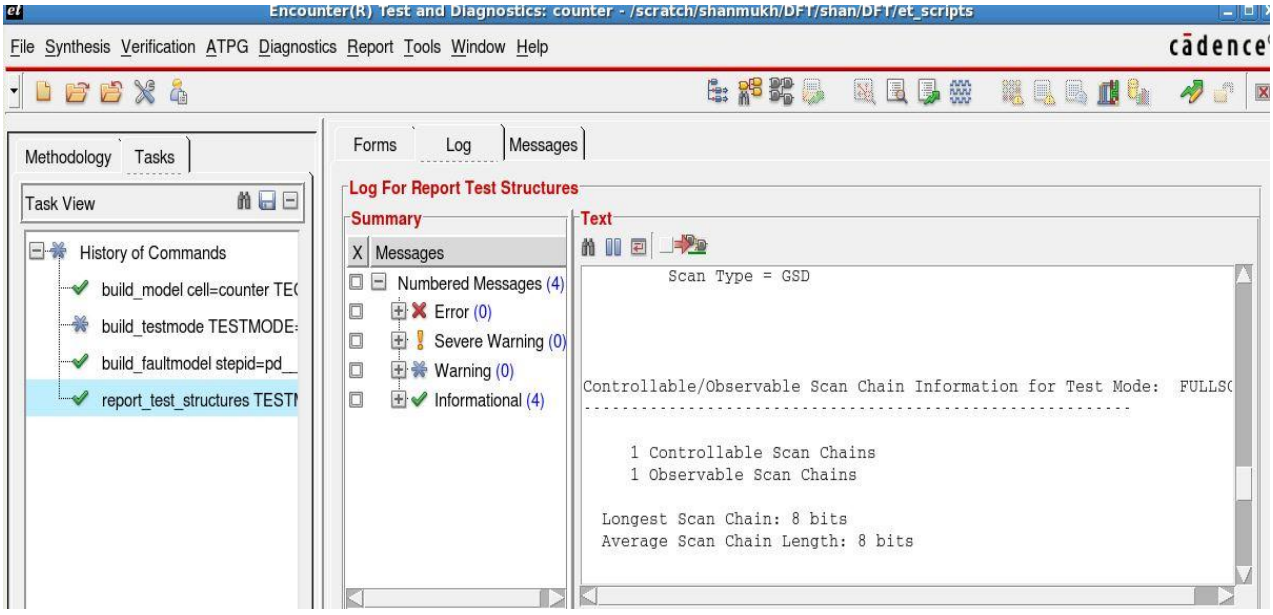


➢ Click on **Report -> Test Structures...**
This tells the tool to trace the scan chains from SO backwards and SI forwards

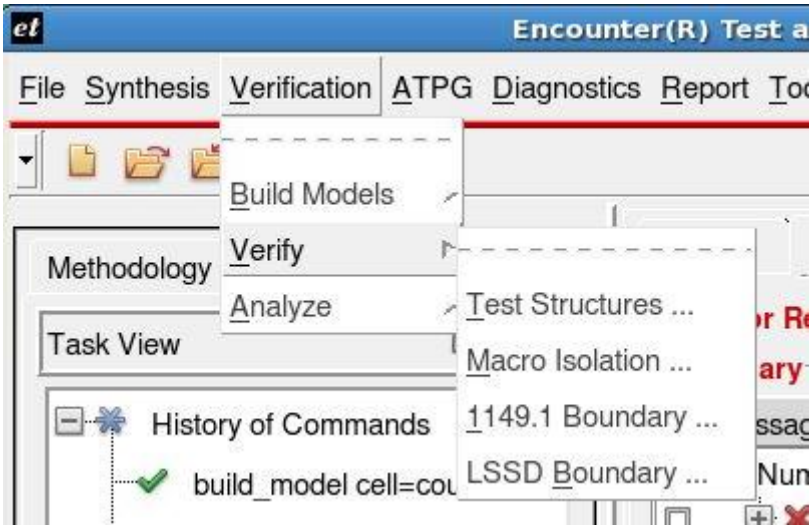
- Select testmode **FULLSCAN** in form and click **Run**



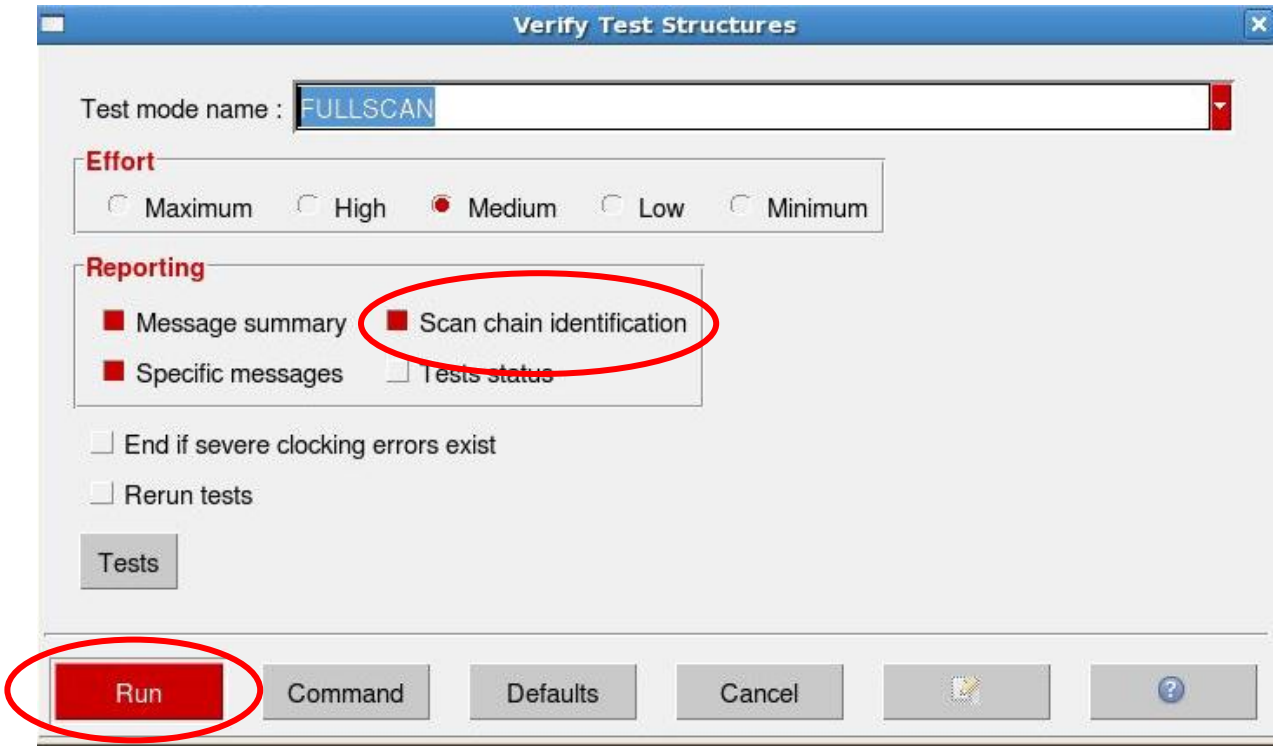
- Now verify how many Controllable chains and how many Observable chains we have in the design, you can see the same in the below snap shot.



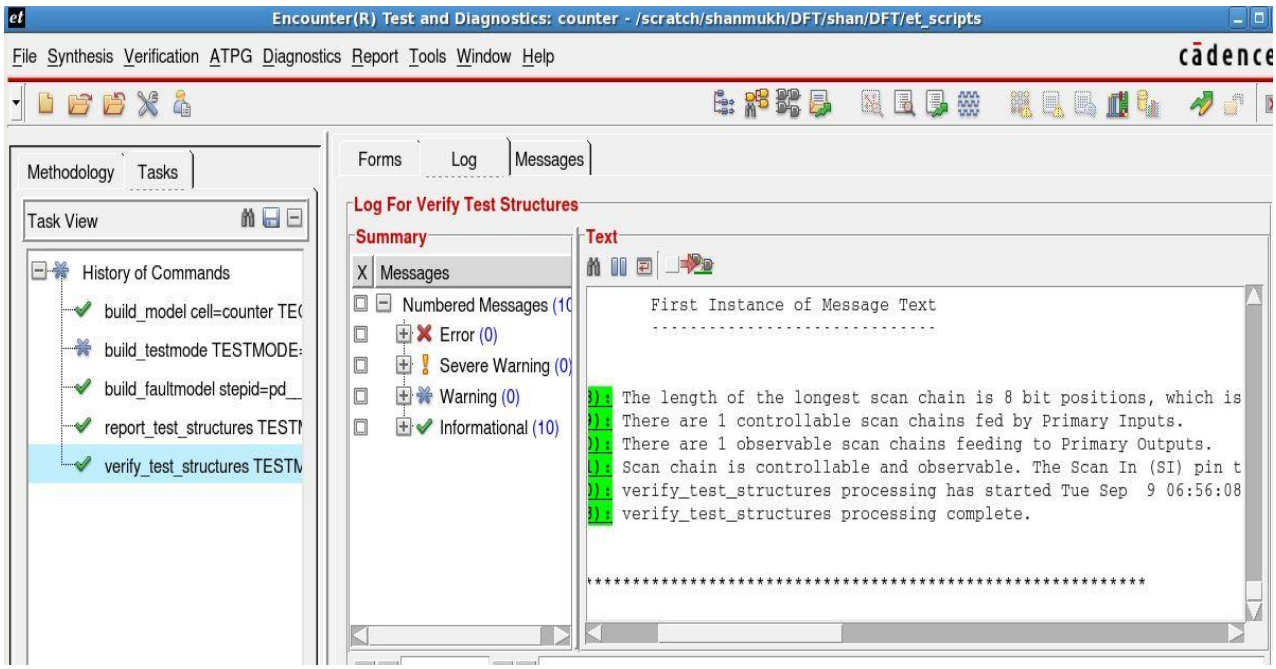
- **Verify Test Structures** (Design rule checking)
  - **Verification -> Verify -> Test Structures**



In the **Verify Test Structures** window select the **Test mode name** as **FULLSCAN and** Click on **Scan Chain Identification**. If you found a chain broken in the previous step, this will give you more debug information. Then click **Run**.



- Verify the log Informational message **TSV-381** identifies how many complete scan chains were found. Informational messages **TSV-384** and **TSV-385** will identify chains that are broken.



- Once you have taken your circuit through the build and verify process. It is now ready for vector generation.

ATPG Vector Generation

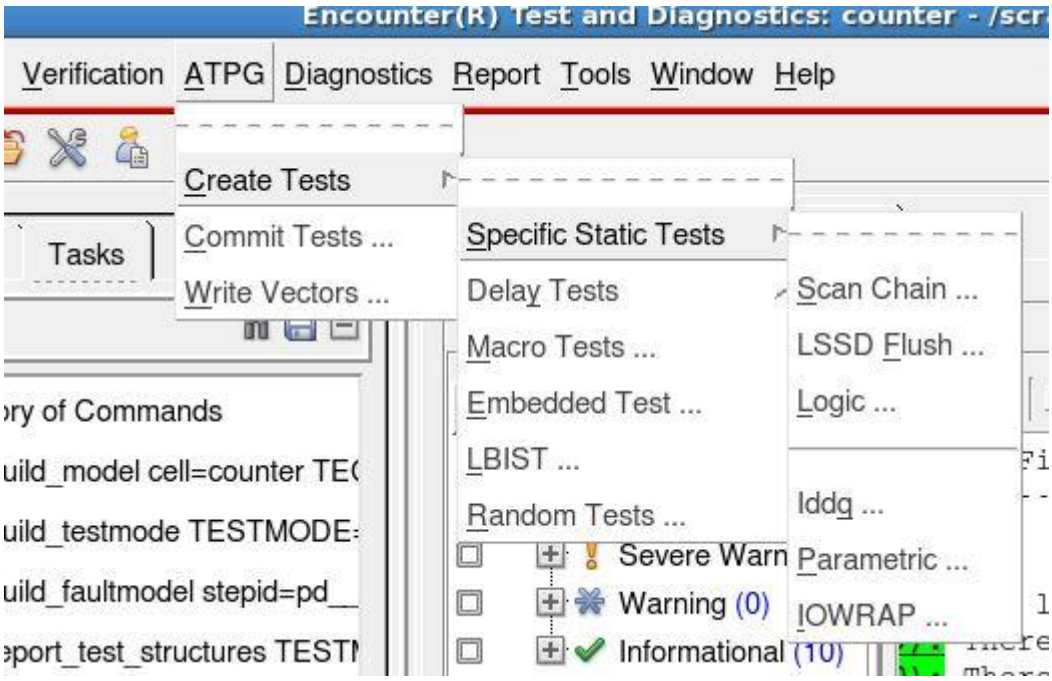
Before writing the ATPG vectors, we will perform the scan and logic test to verify how much test coverage can be achieved using the generated test patterns.

1) **Scan test**
2) **Logic test**

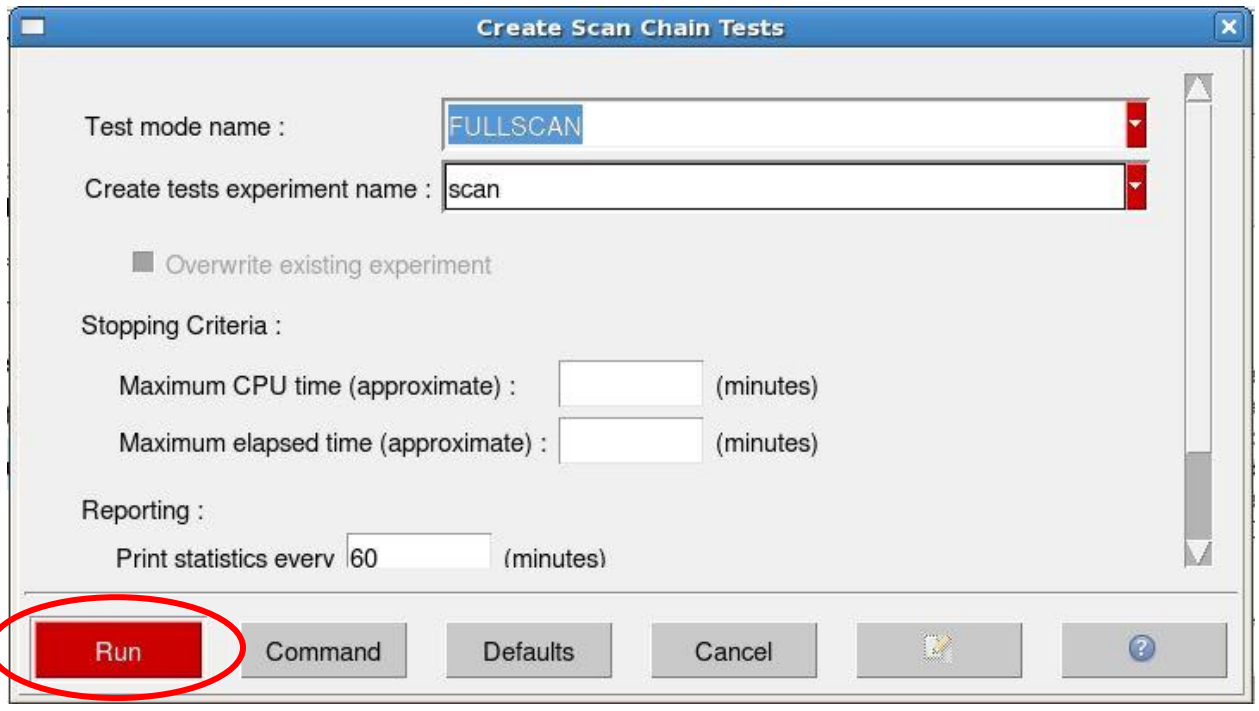
➢ Scan test

In Scan test, we generate patterns to verify simple shifting through the scan chains. This is mainly to identify manufacturing bugs and to insure you can shift safely from Scan in(SI) to Scan out(SO).

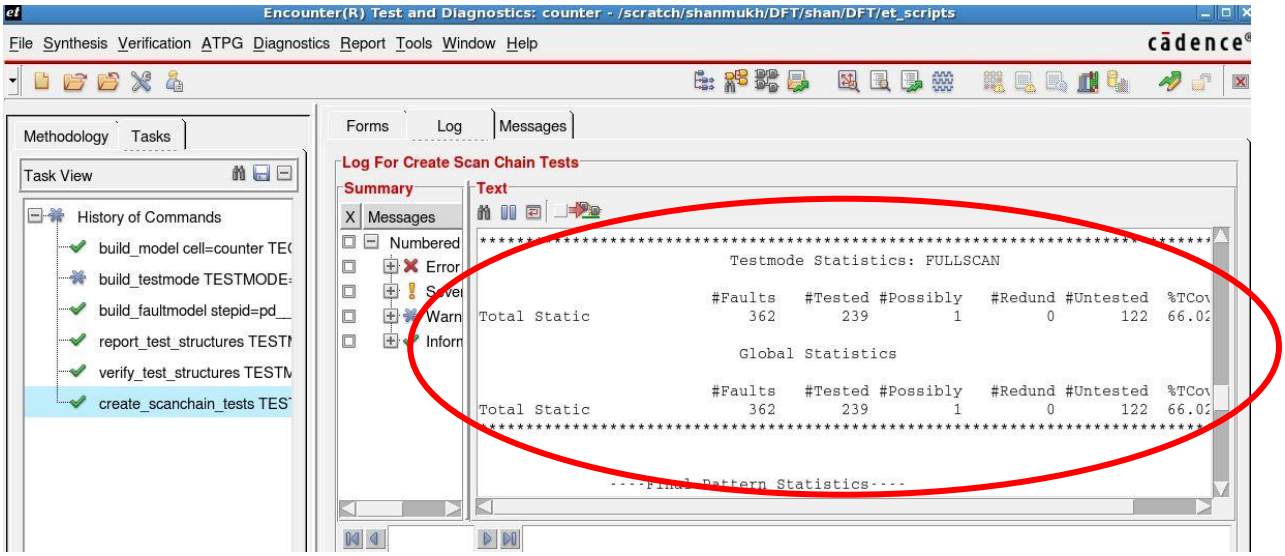
- Go to **ATPG -> Create Tests -> Specific Static Tests -> Scan Chain**



- In the **create scan chain tests** window, select the **Test mode name** as **FULLSCAN** and give **Create tests experiment name** as **scan** (or name of your choice)and click **Run**.



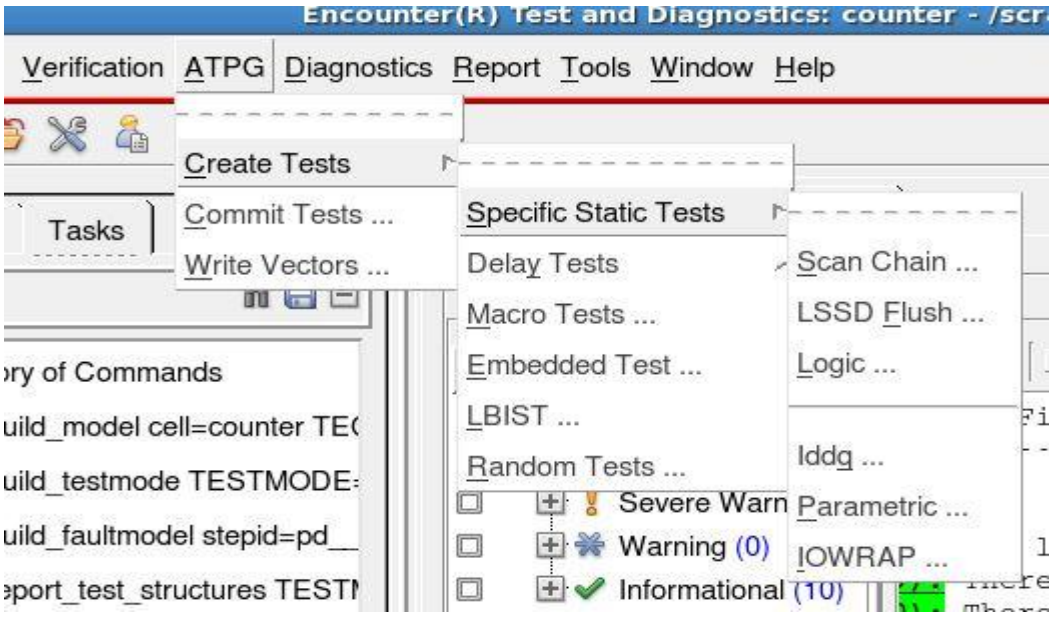
- The ATPG engine is now generating test patterns that test the scan chain. Take a look at the log file and observe the *resultant Fault Coverage in the Testmode, Global Coverage and number of test sequences generated as shown below.*



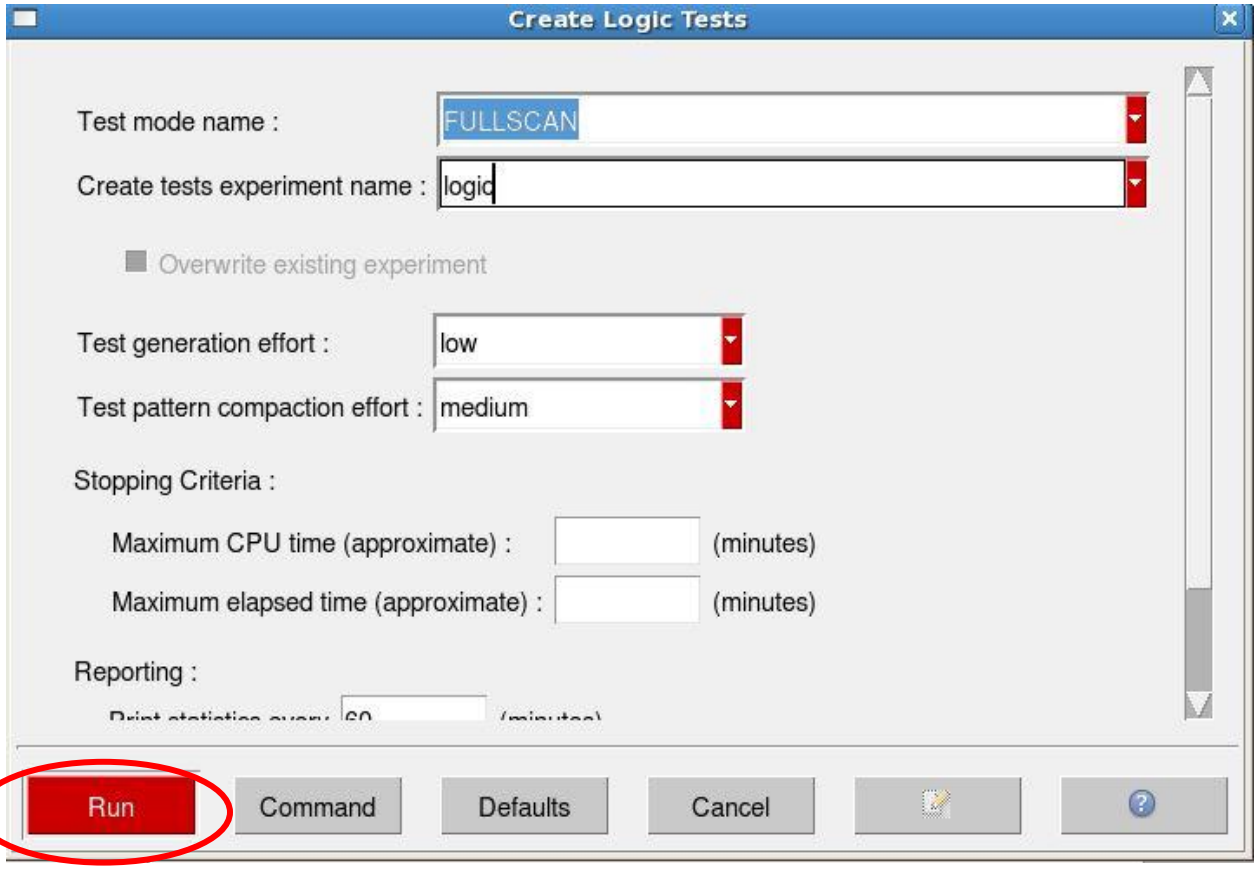
➢ Logic Tests

Now we are going to create the standard Stuck-At model ATPG patterns.

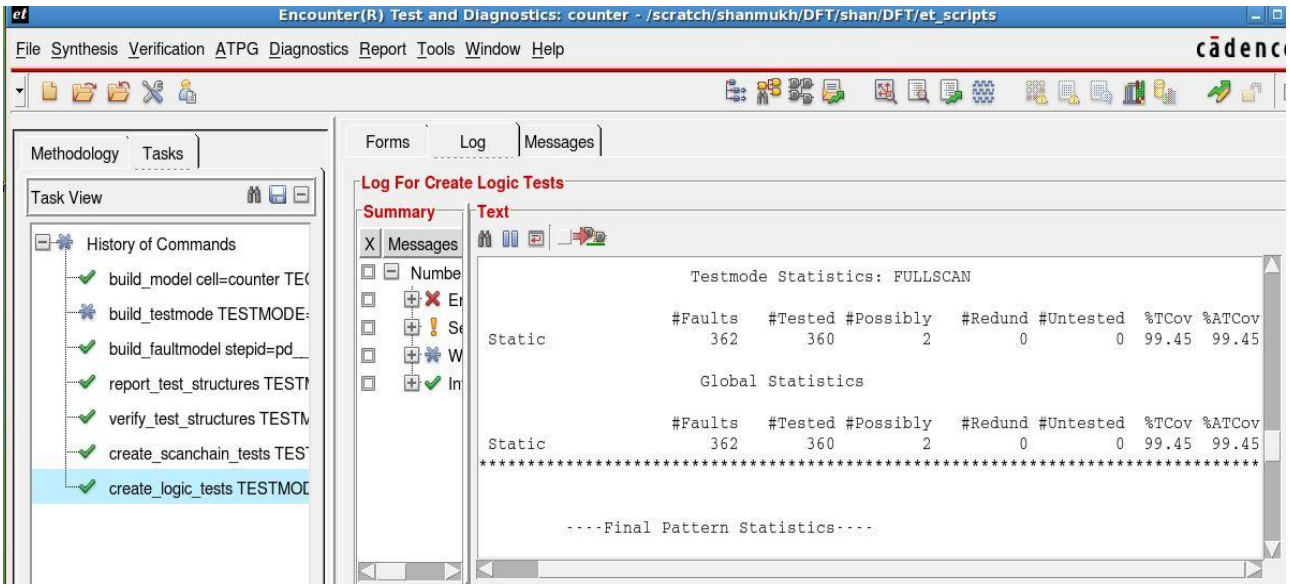
- Go to Pulldown menu **ATPG -> Create Tests -> Specific Static Tests -> Logic** as shown below



- In the **create logic test** window select the **Test mode name** as **FULLSCAN** and specify **Create tests experiment name** as **logic** (or name of your choice) Notice in this step that the previous Experiment **scan** is no longer available and click → **Run**

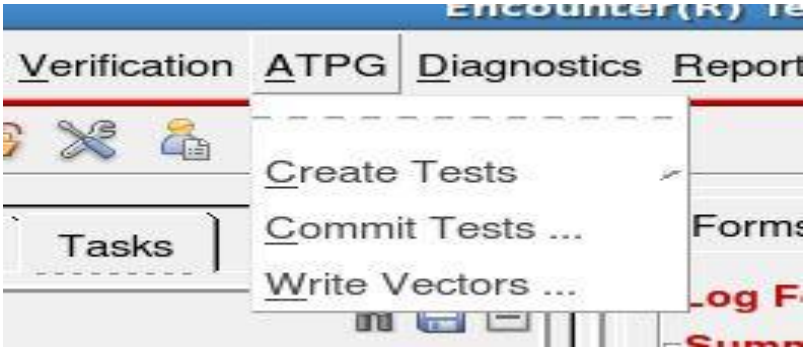


The ATPG engine is now generating test patterns that will test the structural integrity of the design. Take a look at the log file, observe *resultant Fault Coverage in the Testmode (FULLSCAN), Global Coverage and total number of Test Sequences.*

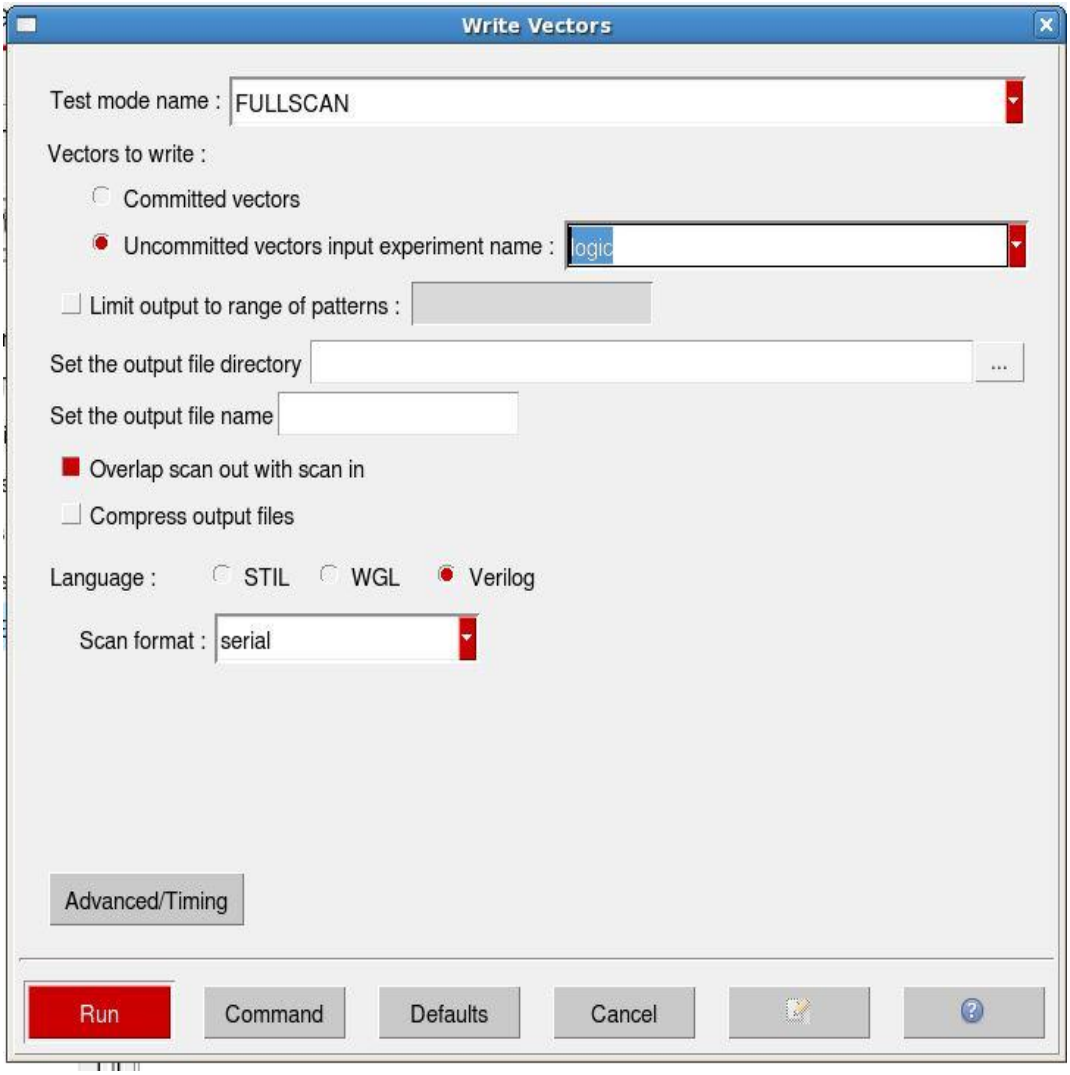


➢ **Write out Verilog patterns**

- Go to Pulldown menu **APTG -> Write Vectors**

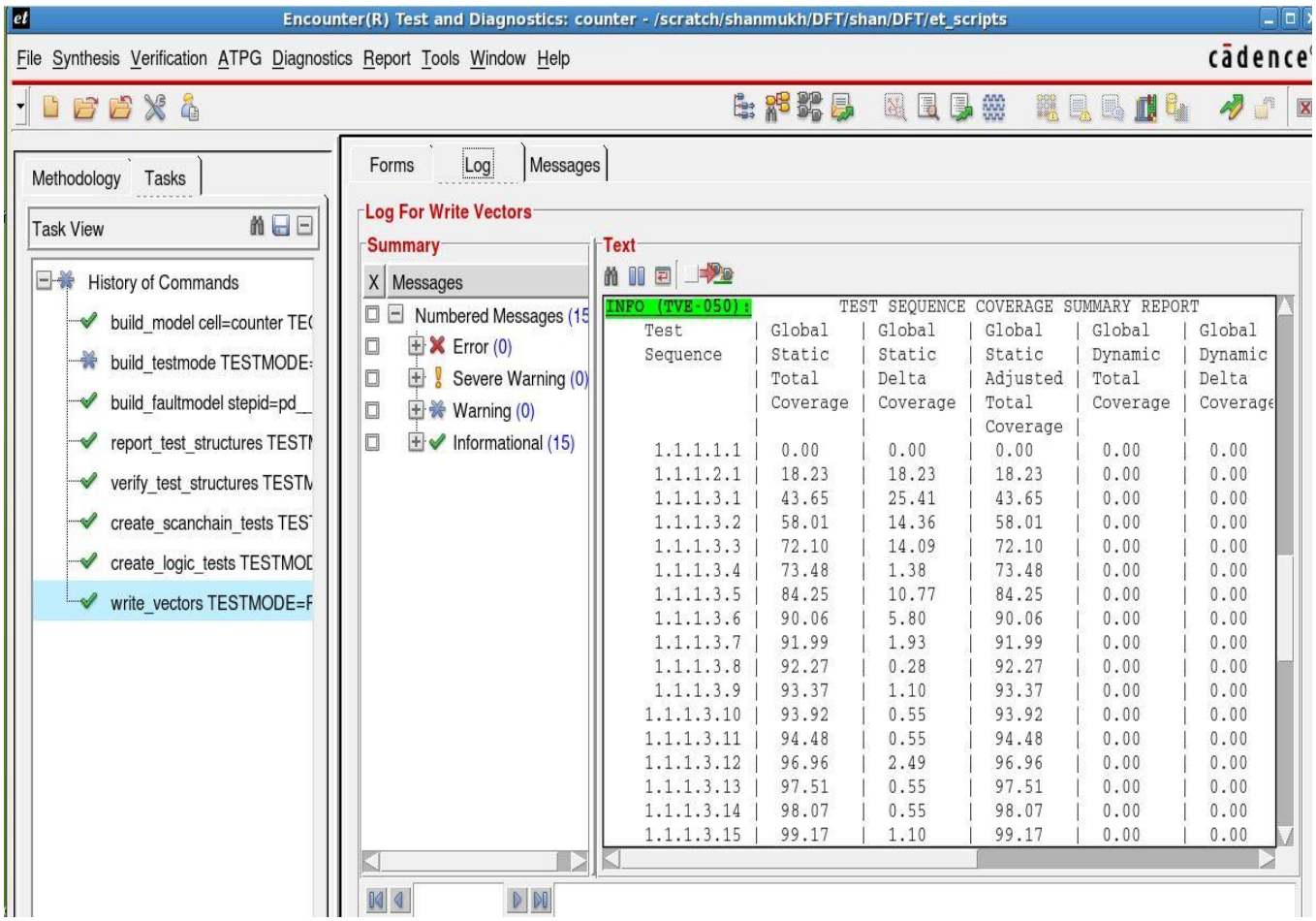


- In the write vectors window specify the T**est mode name** as FULLSCAN and select Vectors to write as **Uncommitted Vectors input Experiment name** and specify name as **logic** and specify the Language as **Verilog.** Click **Run** and review the log file.



- You can view all patterns in the log window and the results will be saved in the **/testresults/Verilog**

```
Terminal                                                    Terminal
[shanmukh@lnx-shanmukh et_scripts]$ ls
counter.et_netlist.v        et_checks.sh  run_fullscan_sim  testresults
counter.FULLSCAN.pinassign  runet.atpg    tbdata
[shanmukh@lnx-shanmukh et_scripts]$ cd testresults/
[shanmukh@lnx-shanmukh testresults]$ ls
logs  verilog
[shanmukh@lnx-shanmukh testresults]$
```



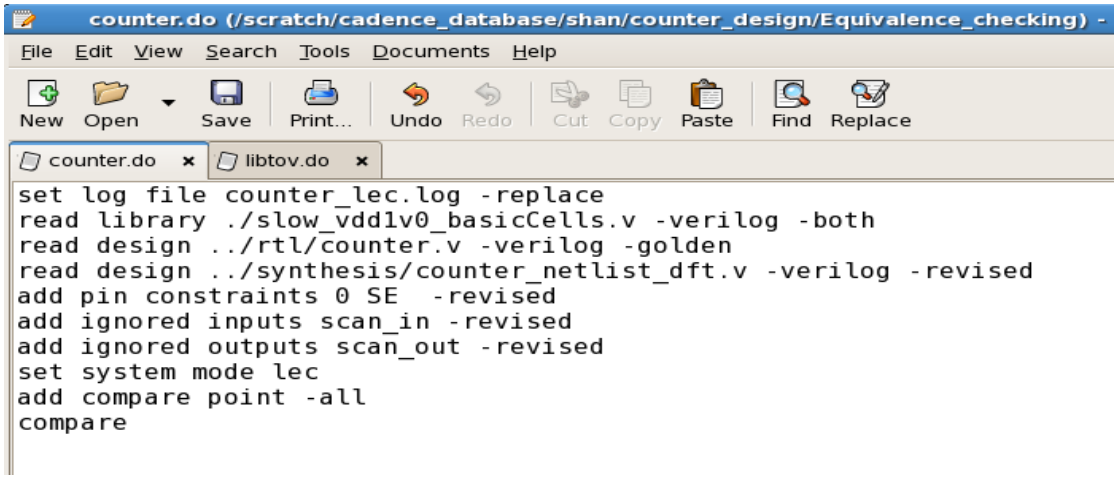
Logic Equivalence Checking

Conformal LEC is a tool used for formal verification of designs at various stages in the flow. Formal verification is the process of verifying designs using mathematical methods. Equivalence Checking is the process of verifying the correctness of a modified or transformed design (revised design) by comparing it with it with a reference design (golden design).

- Invoke Conformal LEC inside Equivalence checking directory in non-GUI by using the **command "lec –xl –nogui -color -64 -dofile counter.do"**
  - -xl :- Launches Encounter® Conformal® L with Datapath and advanced equivalence checking capabilities
  - -nogui :- Starts the session in non-GUI mode
  - -color :-Turn on color-coded messaging when in non-GUI mode
  - -64 :- Runs the Encounter® Conformal® software in 64-bit mode
  - -dofile <filename> :- Runs the script <filename> after starting LEC

DOFILE

Let us understand the content of the dofile. Dofile is a script file used to run LEC and below is an example for the dofile.



Below are the basic set of commands used.

- Save log file.
  set log file <filename.log> - replace
  Save log file and replaces if any log file exist with same name if any.
- Read the Verilog library by entering:
  read library <filename> -verilog –both

[Both verilog and liberty format can be used but verilog format is preferred. Steps to generate .v from .lib using Conformal is mentioned at the end of this session]

- -verilog :- to indicate that library is in Verilog format
- -both :- use same library to model or structure both golden and revised design.

- Read the Golden Design (RTL)

  read design <filename> -verilog –golden

  - -verilog :- to indicate that RTL is coded in Verilog
  - -golden :- to input the golden design

- Read the Revised Design:

  read design <filename> -verilog –revised

  - -verilog :- to indicate that netlist is in Verilog
  - -revised :- to input the revised design

- Ignore the scan input(scan_in) and Scan output (scan_out) pins (as these instances are not available in golden design and primary output key point is compare point)

  add ignored inputs scan_in –revised [ignores scan_in pin]

  add ignored outputs scan_out –revised [ignores scan_out pin]

- Constraint the scan enable (SE) pin to zero to keep the revised design in functional mode.

  add pin constraints 0 SE -revised [tool keeps the design in functional mode and ignore scan_in pin while compare. Also scan_in is not a compare point]

- Change the mode of operation from ‘setup’ to ‘lec’

  Set system mode lec

Note: Conformal lec got two modes of operation i.e. SETUP mode and LEC mode. Setup mode is used to prepare the design to be compared. Any command that affects the way the design is modeled will need to be issued in this mode. LEC mode is where the designs will get modeled, key points mapped and where the compare process takes place.

- Compare golden Vs. revised netlist

  add compare points –all

  compare

  Once the compare process is completed, Conformal LEC will print a summary report that tells how many key points are equivalent, non-equivalent, aborted and not compared.

```
--------------------------------------------------------------------------------
// Command: add compare point -all
// 16 compared points added to compare list
// Command: compare
================================================================================
Compared points      PO     DFF       Total
--------------------------------------------------------------------------------
Equivalent           8      8         16
================================================================================
```

- Generate verification report.

  report verification

  Reports a table of all violated checklist items

Note: Use command 'set gui on' to turn on GUI window. In case of mapping issue or comparison issue or not equivalence, use mapping manager or debug manager or Schematic viewer options in LEC to resolve the issue.

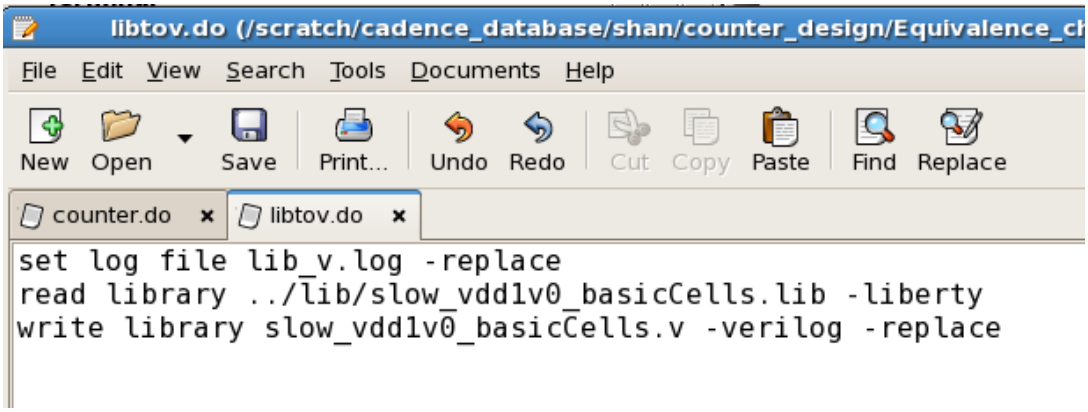
Note: Same is the flow to compare netlist generated at different stages of physical design. Use proper modelling directives and constraints.

Create .v from .lib

- Invoke LEC by using the command "**lec -xl -nogui -64"**
- Read library in liberty(.lib) format by using the command **"read library <.lib file> –liberty –both"**
- Write out the verilog file by using the command **"Write library <file name*> -verilog"**

  ***file name** can be any name with extension .v

Below is a example dofile to generate **'.v'** from **'.lib'**



Physical Design

Encounter Digital Implementation (EDI) is the tool used for physical implementation. Inputs required for physical implementation are Netlist, SDC LIB and LEF (Library Exchange Format – This file contains the details the physical details for the standard cells).

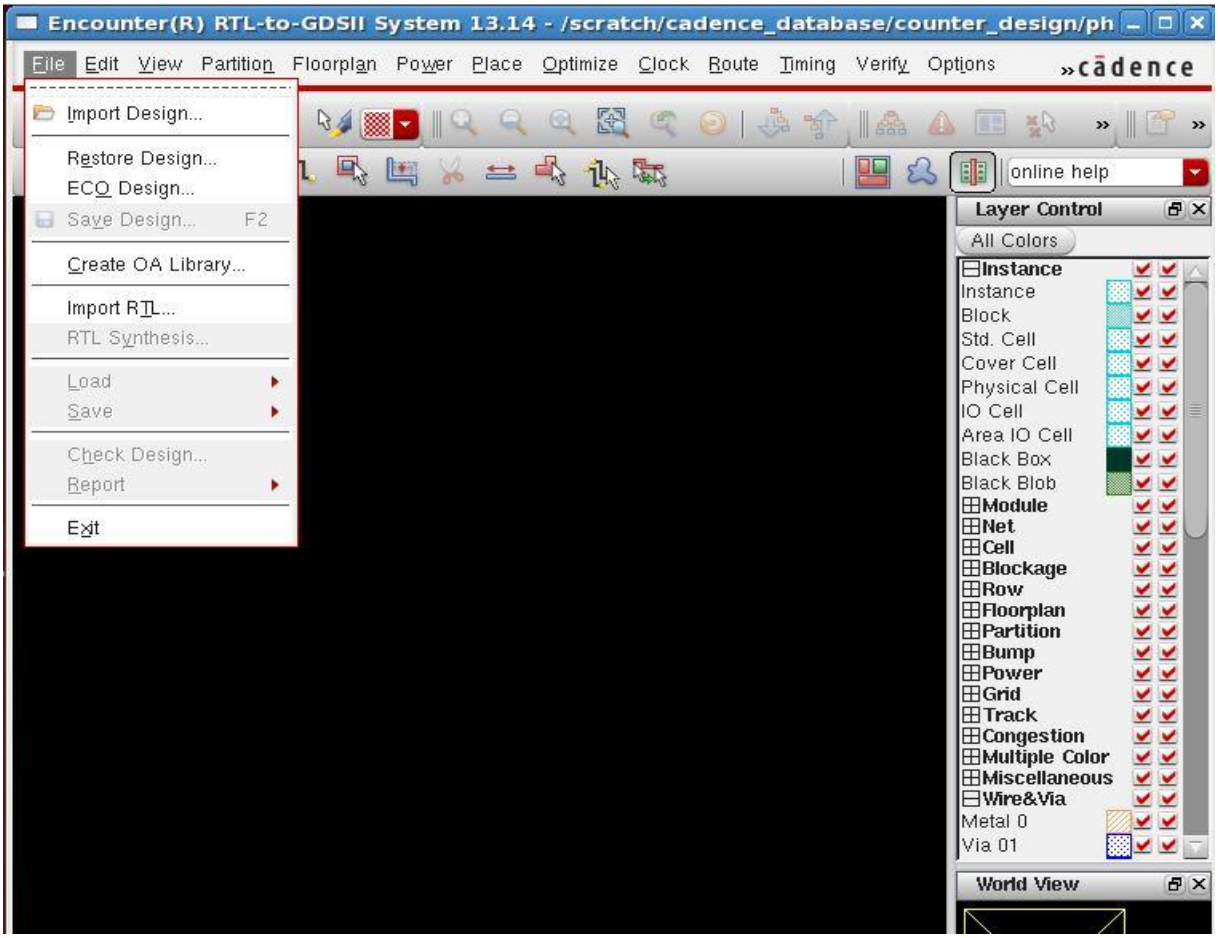
- Invoke Encounter tool inside the 'physical_design' directory typing 'encounter' then click enter

Encounter RTL-to-GDSII System <version> window will pop-up

Note: Log files and other process files will be generated at various stages of the design flow. Initial lines in the log files will give idea about the files.

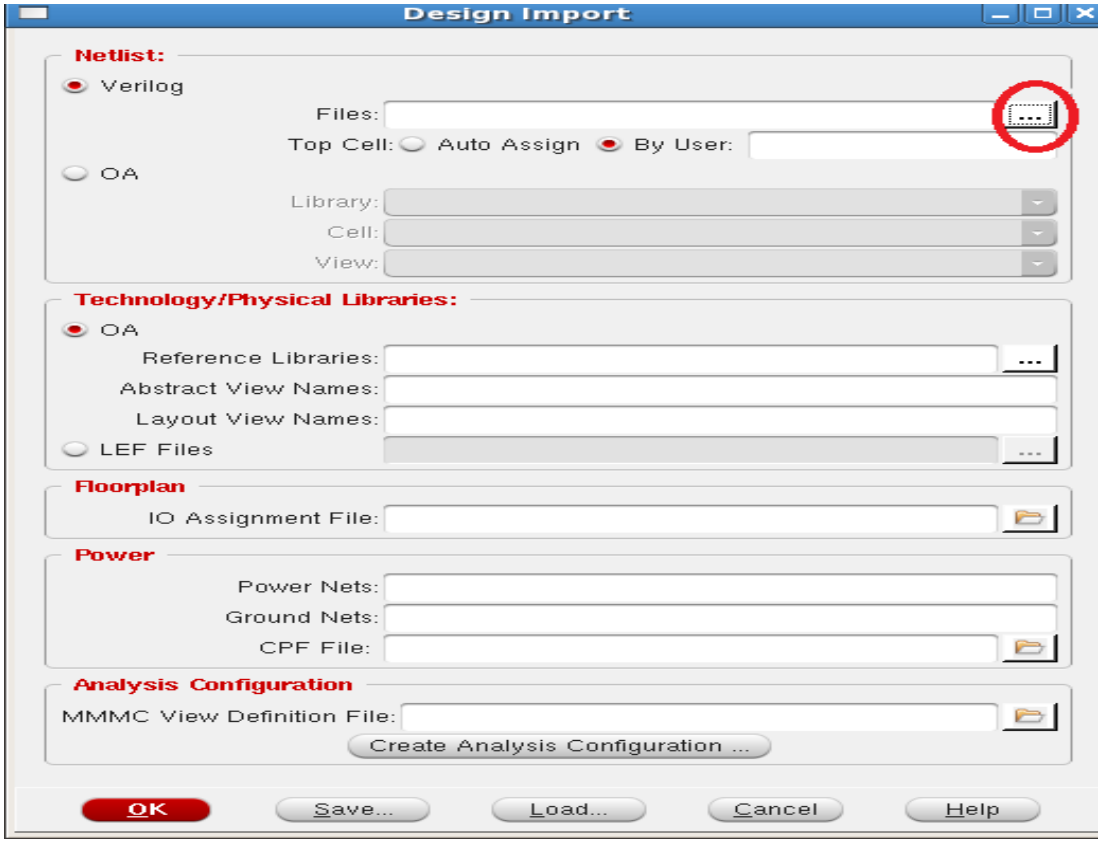
Import Design

- To import the design click on File→ Import Design

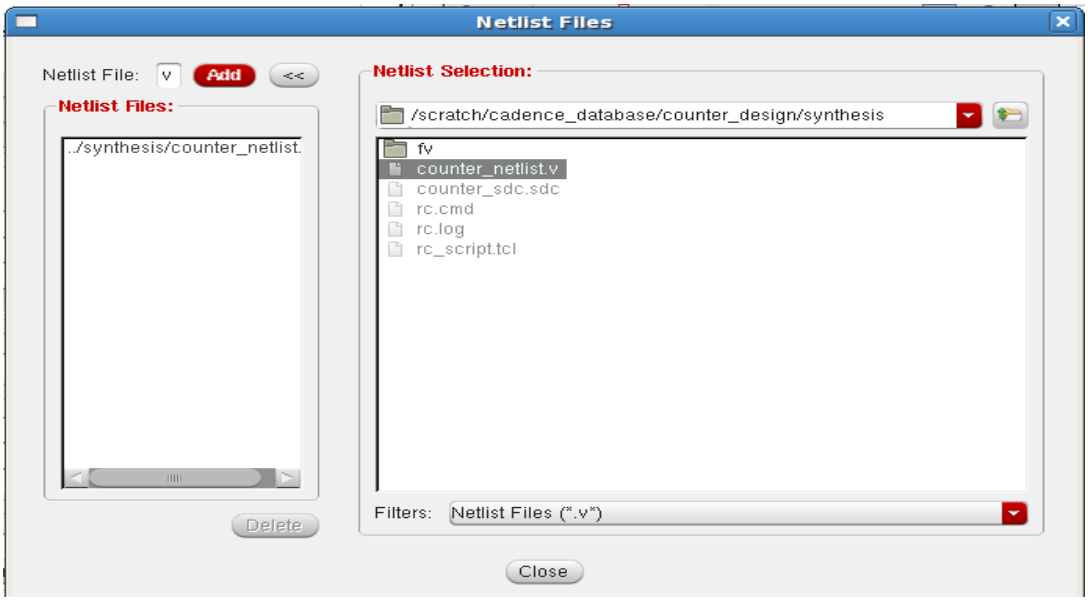


- Design Import form will popup, browse to the gate level net-list (.v file) that you have created from the synthesis stage.

- Click on highlighted portion

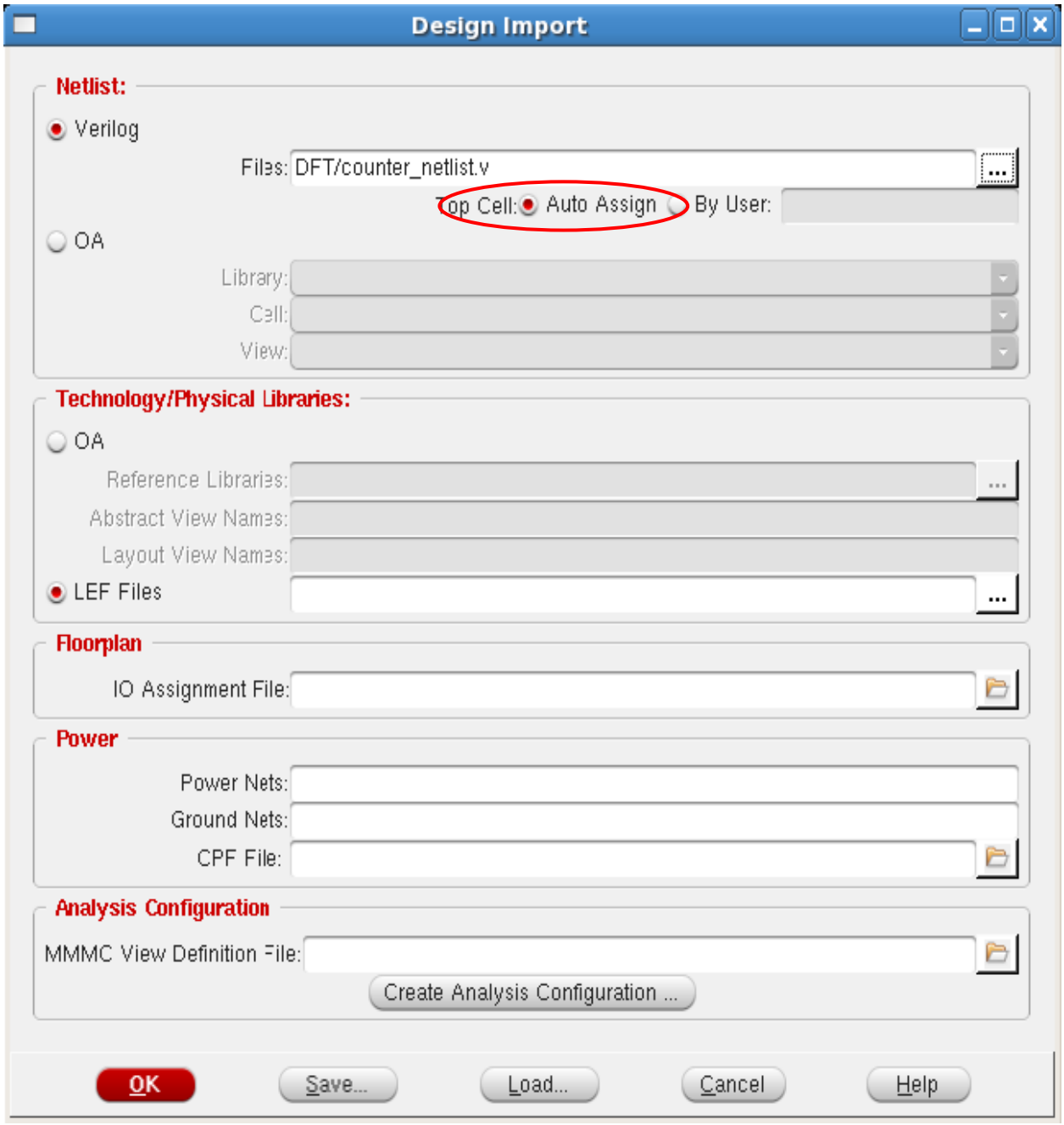


- Under Netlist Selection browse to the .v file you have created using write_hdl command at synthesis stage. Then click on 'Add', then it should appear under the Netlist Files section as shown in the below figure.

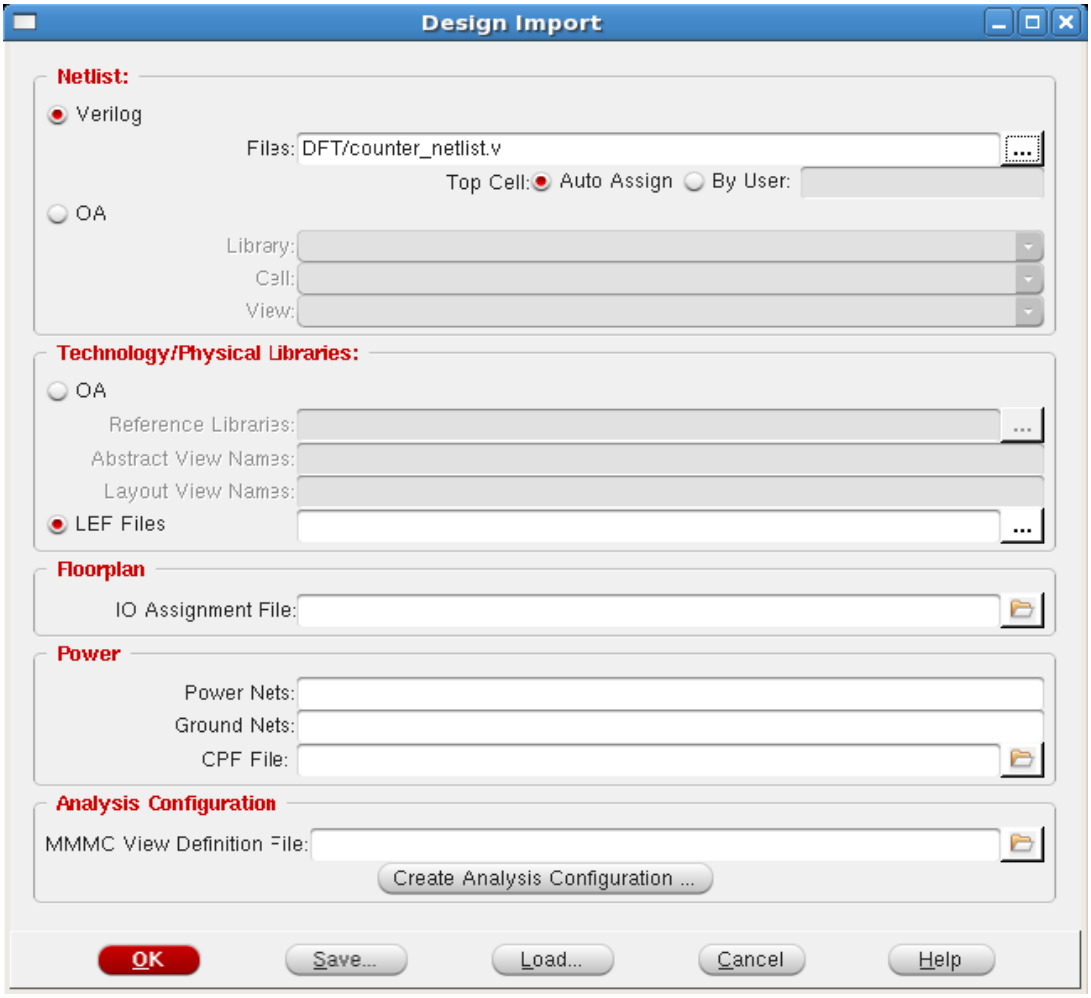


- Click on Close button

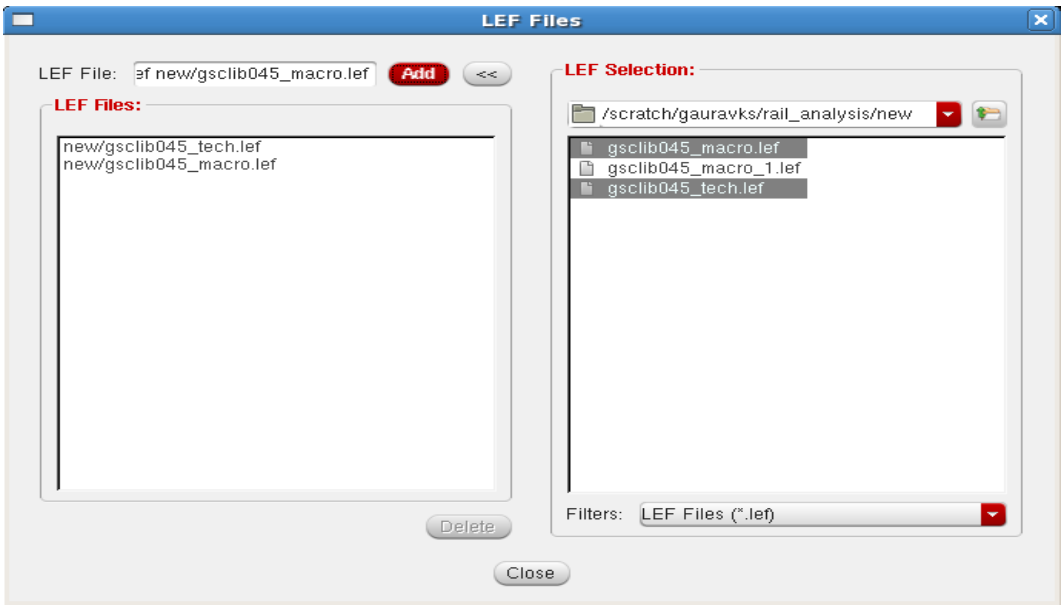
- Change Top Cell to Auto Assign



- Click on LEF Files

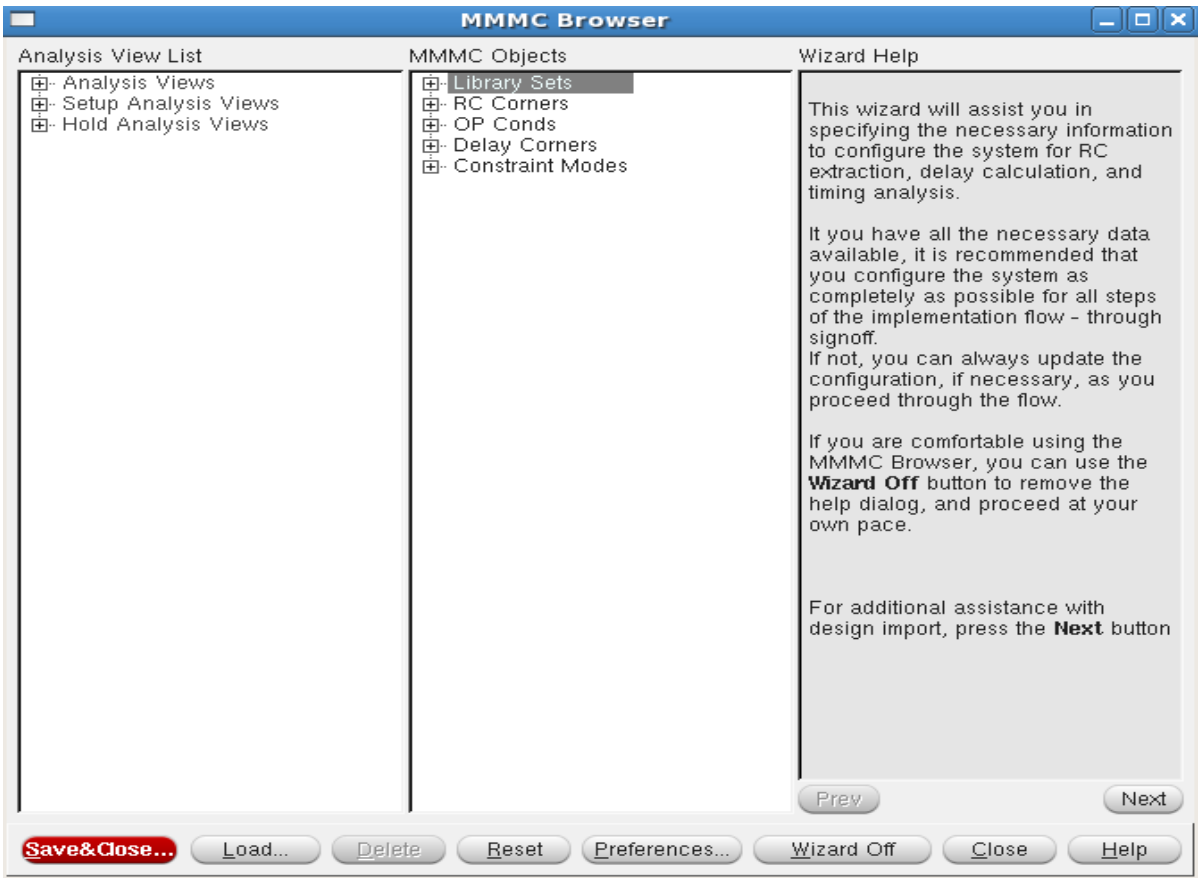


- Browse the LEF (Library Exchange Format) file and click close

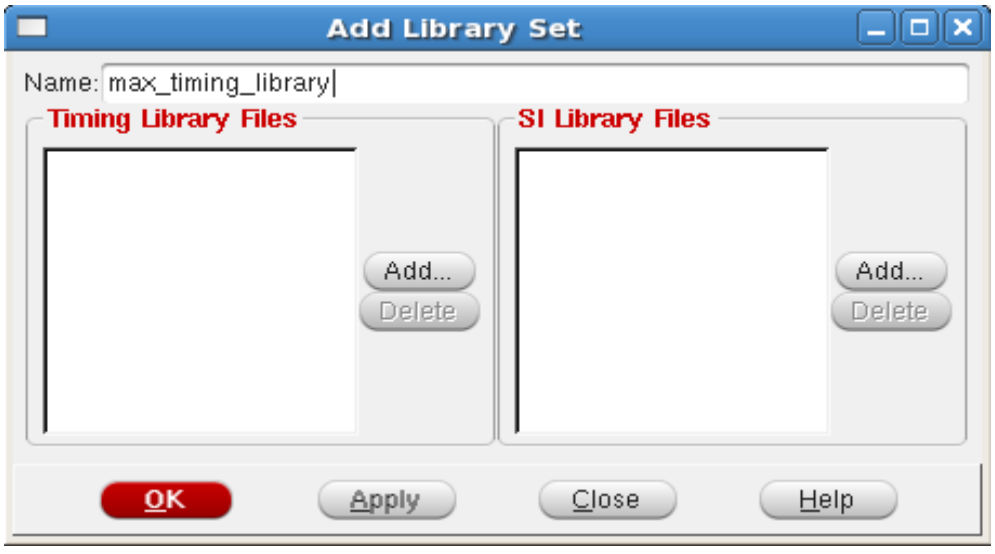


- Give names to Power and Ground nets as VDD and VSS respectively

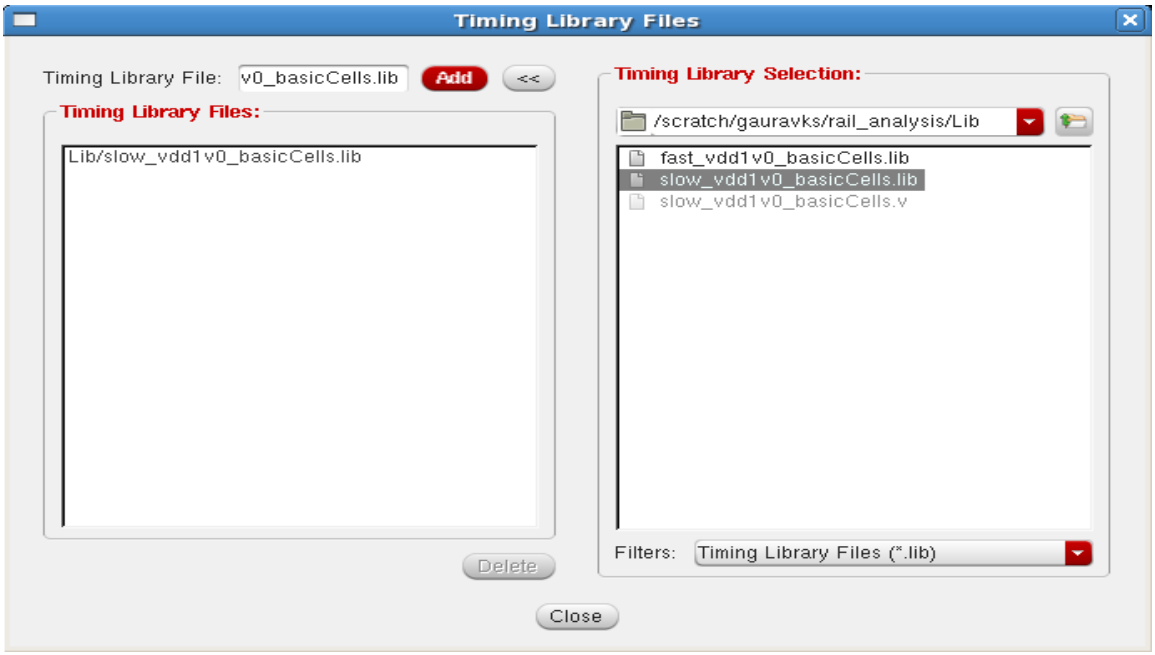
- Click on Create Analysis Configuration
  - A MMMC Browser form appears,
  - Double Click on Library Sets



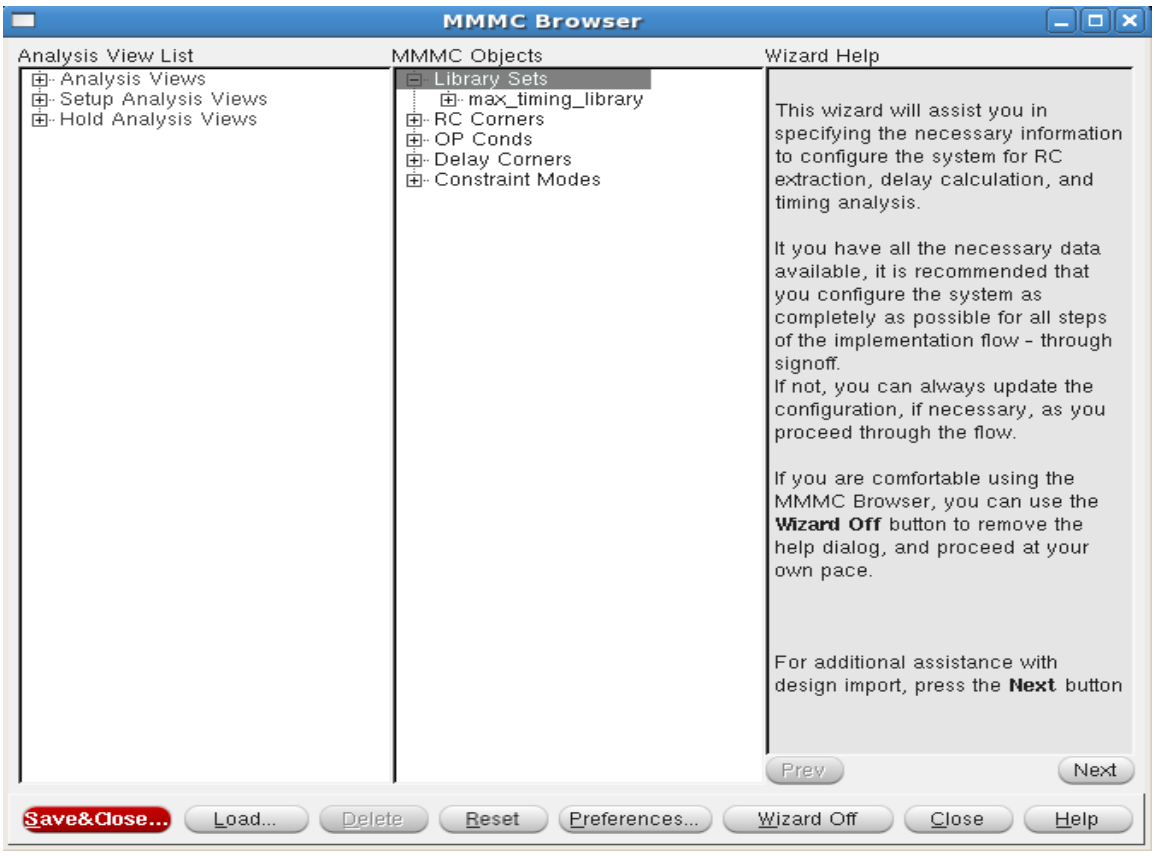
- In the Add Library Set give it name max_timing_library and click on 'Add…' under 'Timing Library Files'



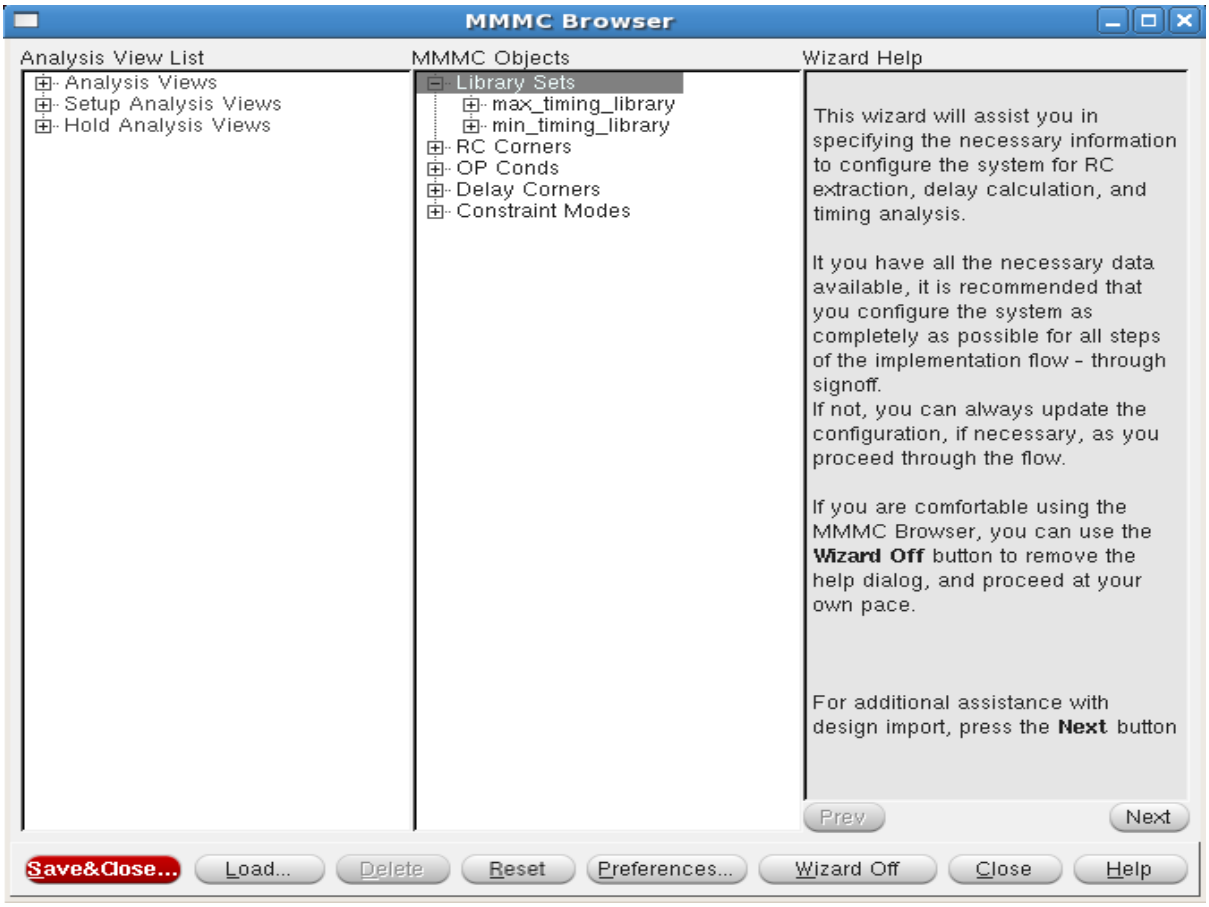
- Maximum timing would be provided by slow_vdd1v0_basicCells.lib. Browse to 'lib' directory then select slow_vdd1v0_basicCells.lib then click on 'Add'. (slow.lib should now appear under Timing library Files) click on Close



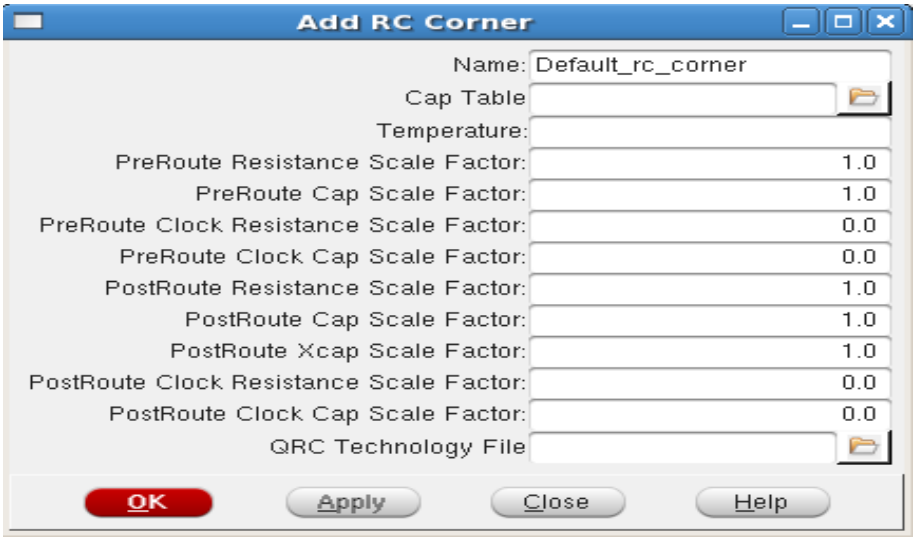
➢ Upon clicking close the MMMC Browser form should look like the below figure



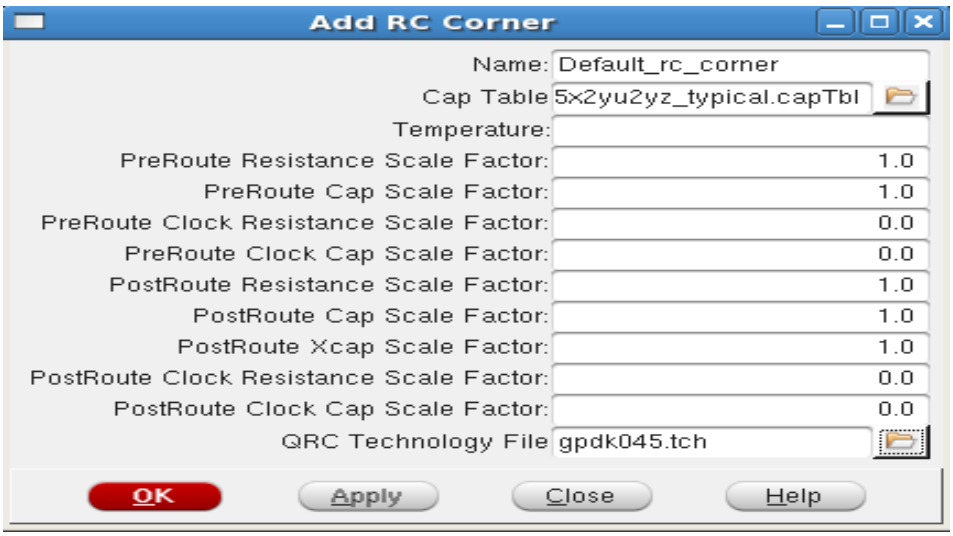
- Similarly for min timing repeat the above steps that we followed to set max timing to slow.lib (double click again on Library Sets , give the name as min_timing_library, Browse to 'lib' then select fast.lib then click on 'Add' then click on close).



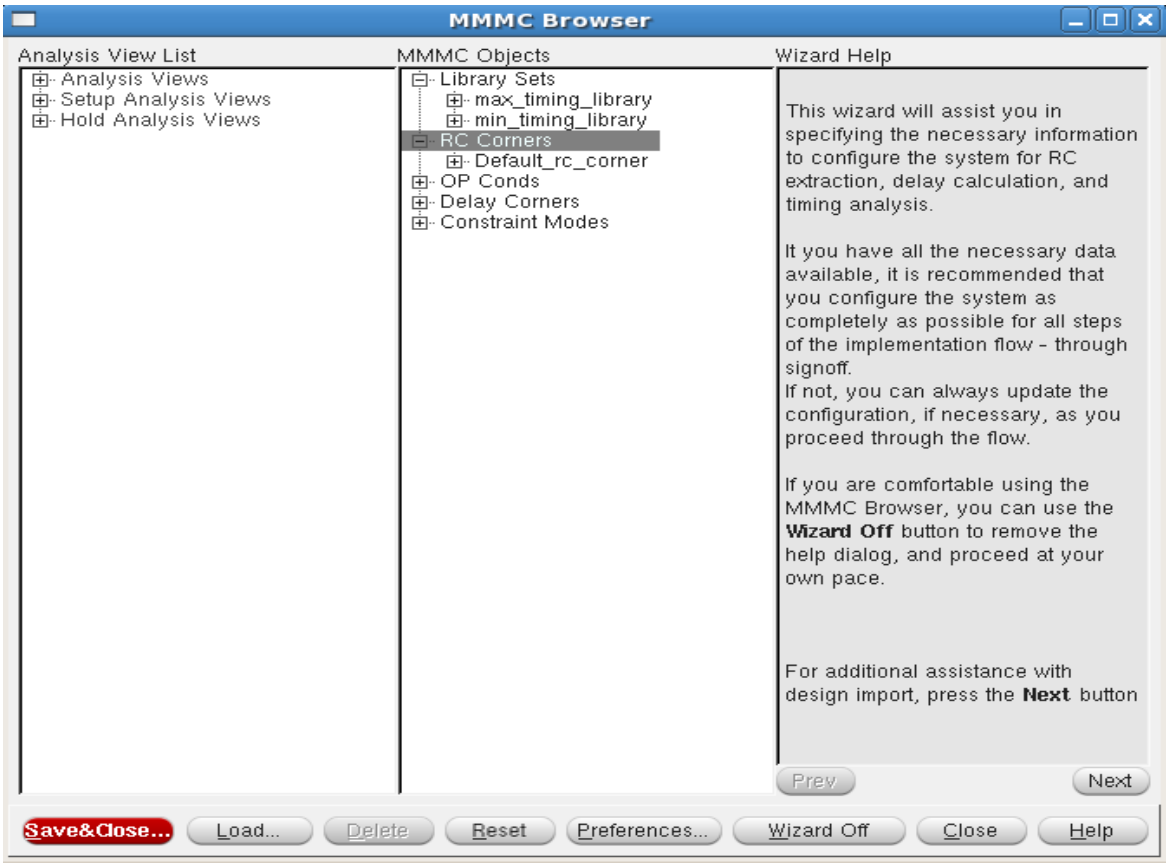
- Double click on 'RC Corners' and 'Add RC Corner' form will appear, give the name as Default_RC_Corner.



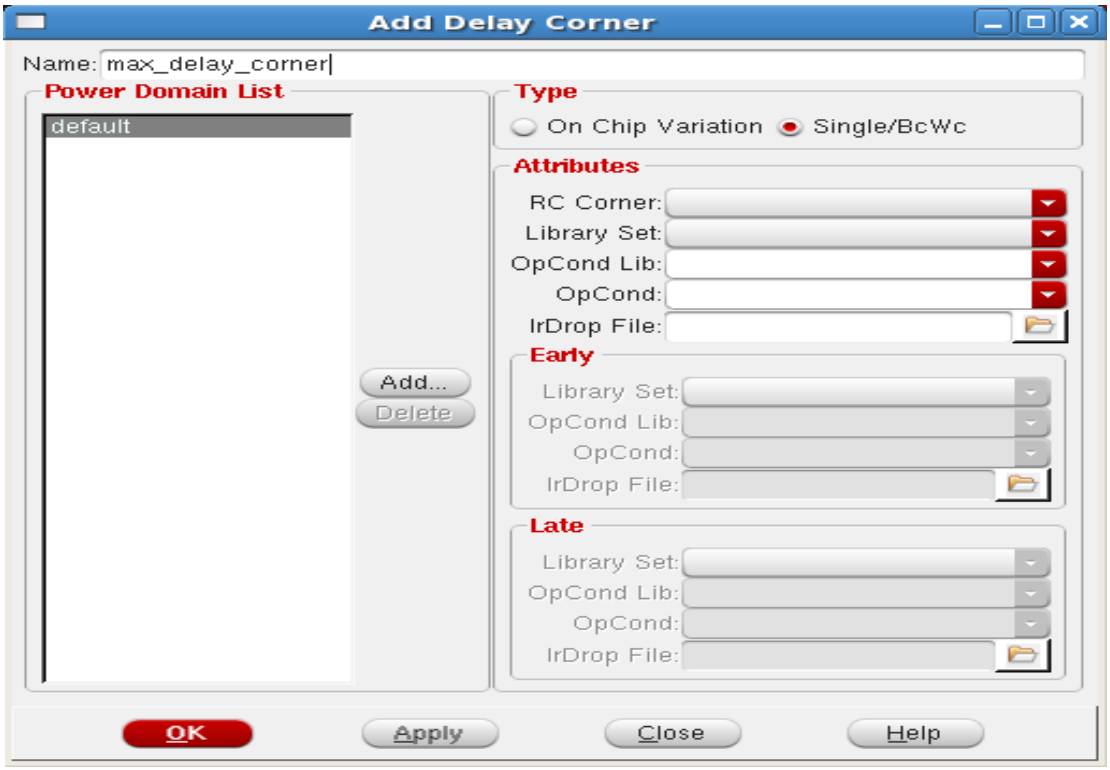
- Now browse the Cap Tables named as cln28hpl_1p10m+alrdl_5x2yu2yz_typical.capTble and technology file gpdk045.tch and click Ok.



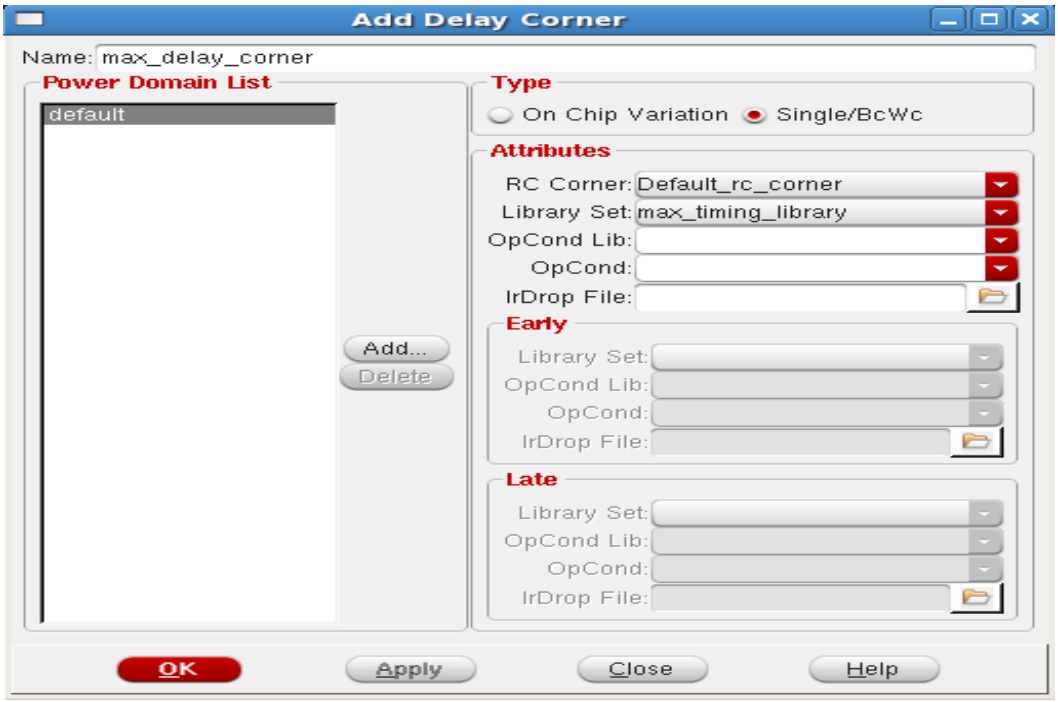
- It will come with the below status.



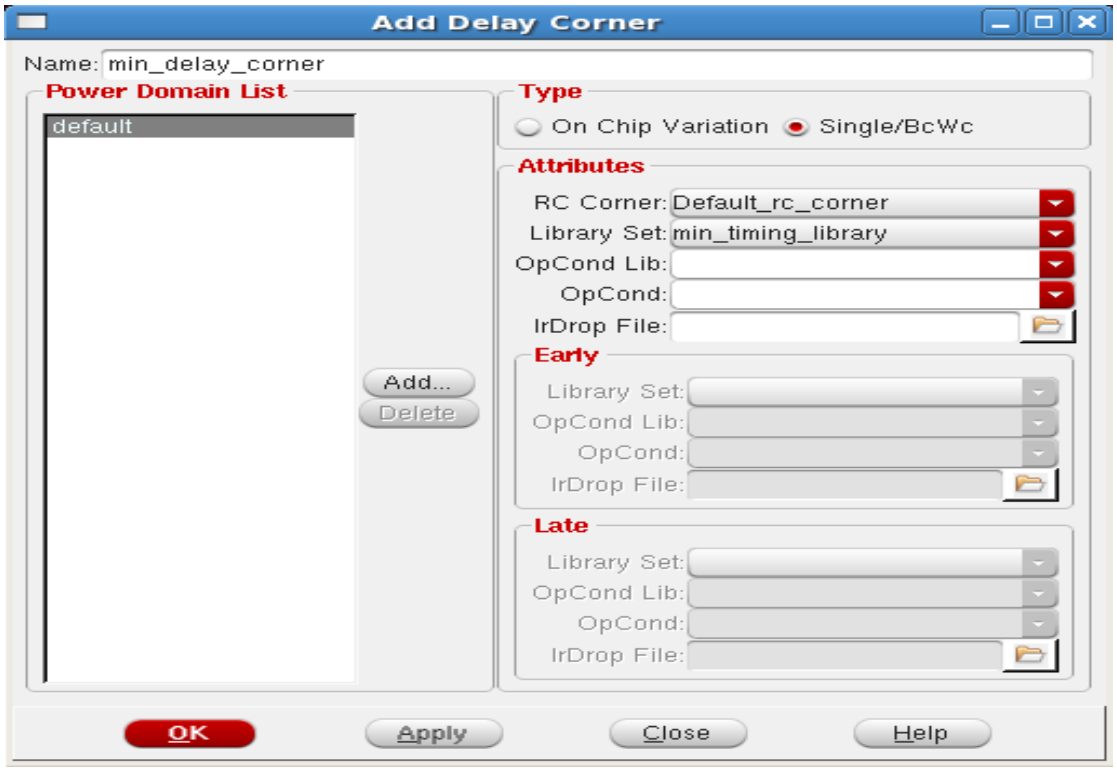
- Double click on 'Delay Corners' and 'Add Delay Corner' form will appear, give the name as max_delay_corner.



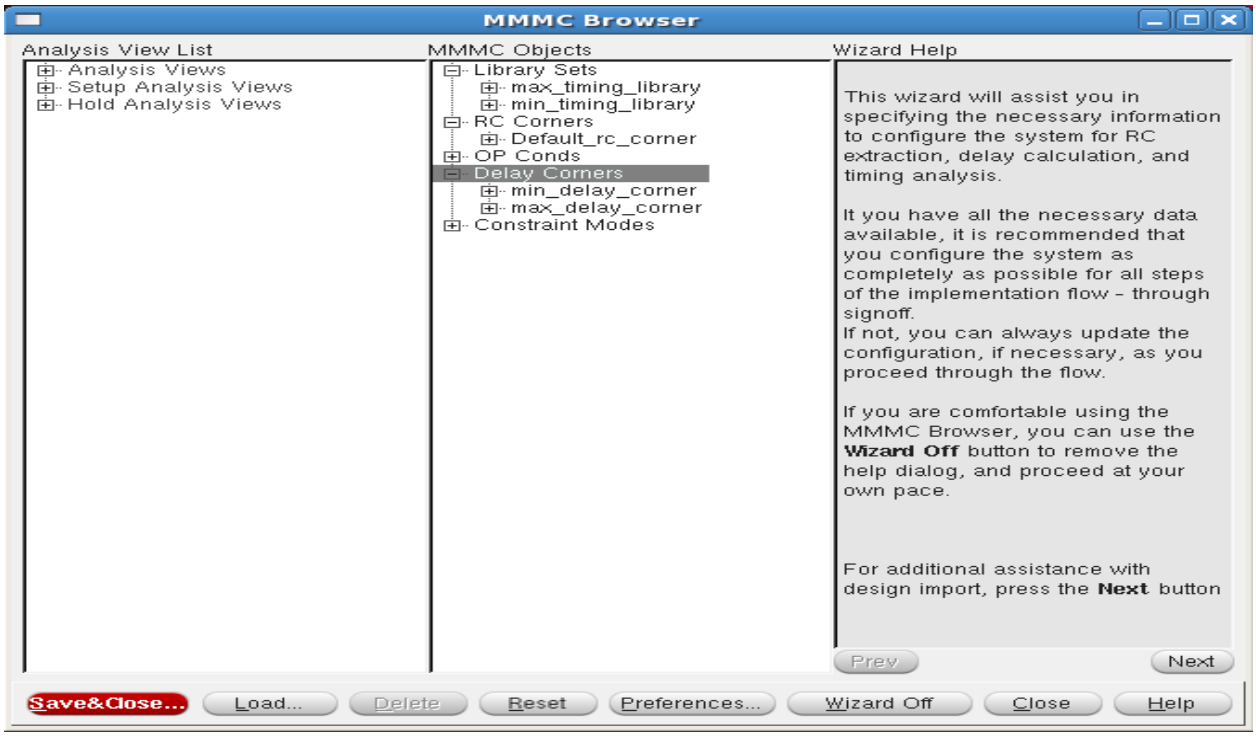
- In the Library Set option under Attributes scroll to max_timing_library then click on 'OK' the below should be the status.



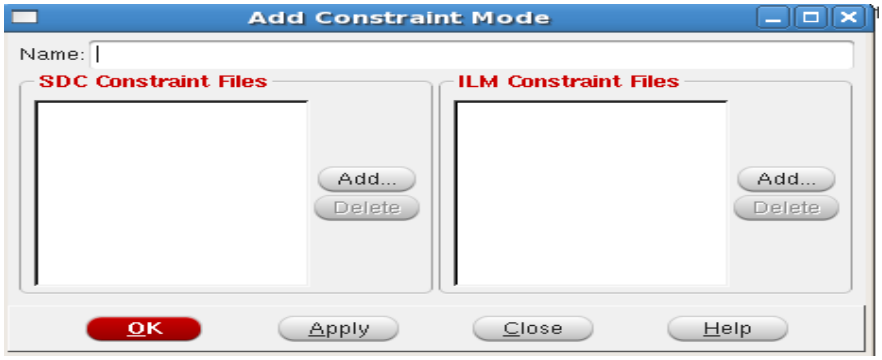
- Similarly again double click on 'Delay Corners' and this time give name as 'min_delay_corner' and add 'min_timing_library' in Library set then click on 'OK'.



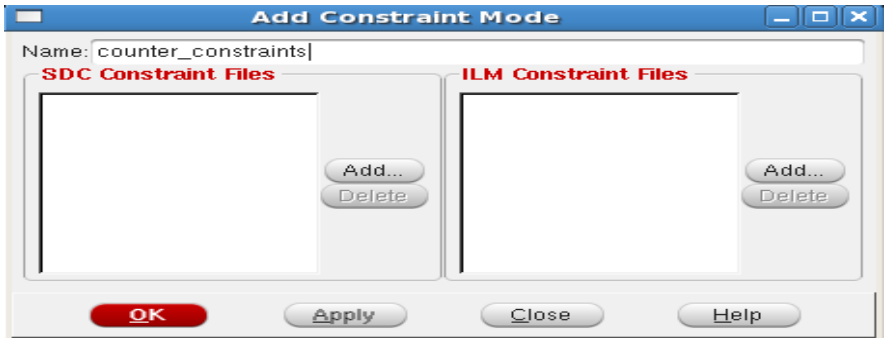
- The below status will appear.



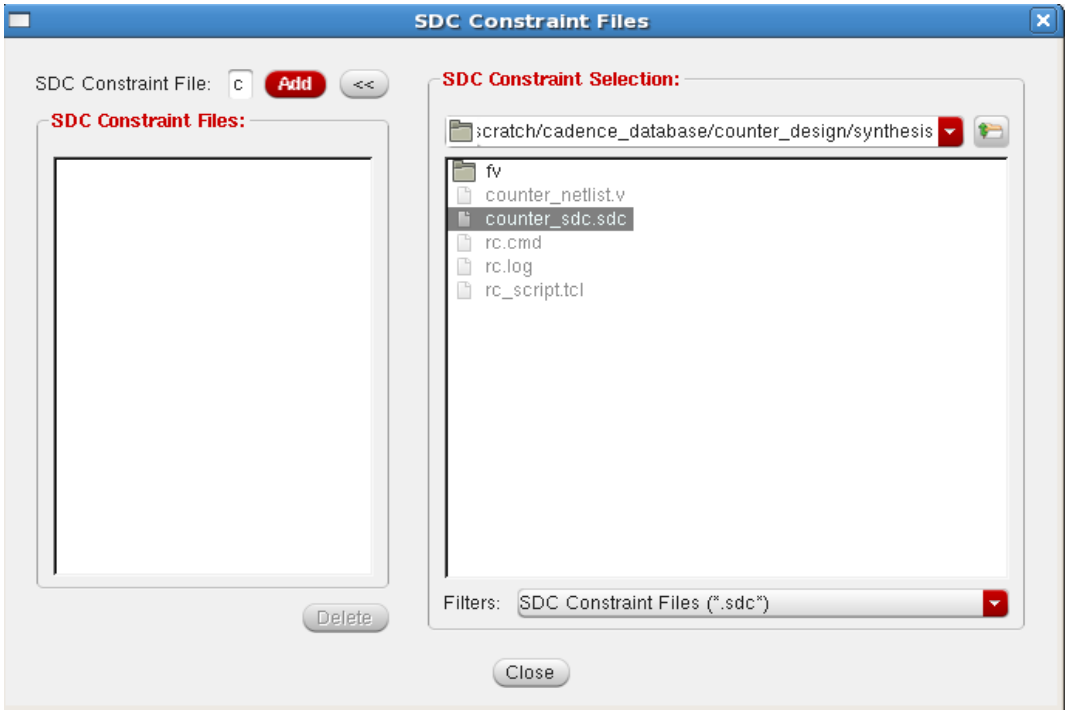
- Double click on Constraints Modes and an ‘Add Constraint Mode’ form appears.



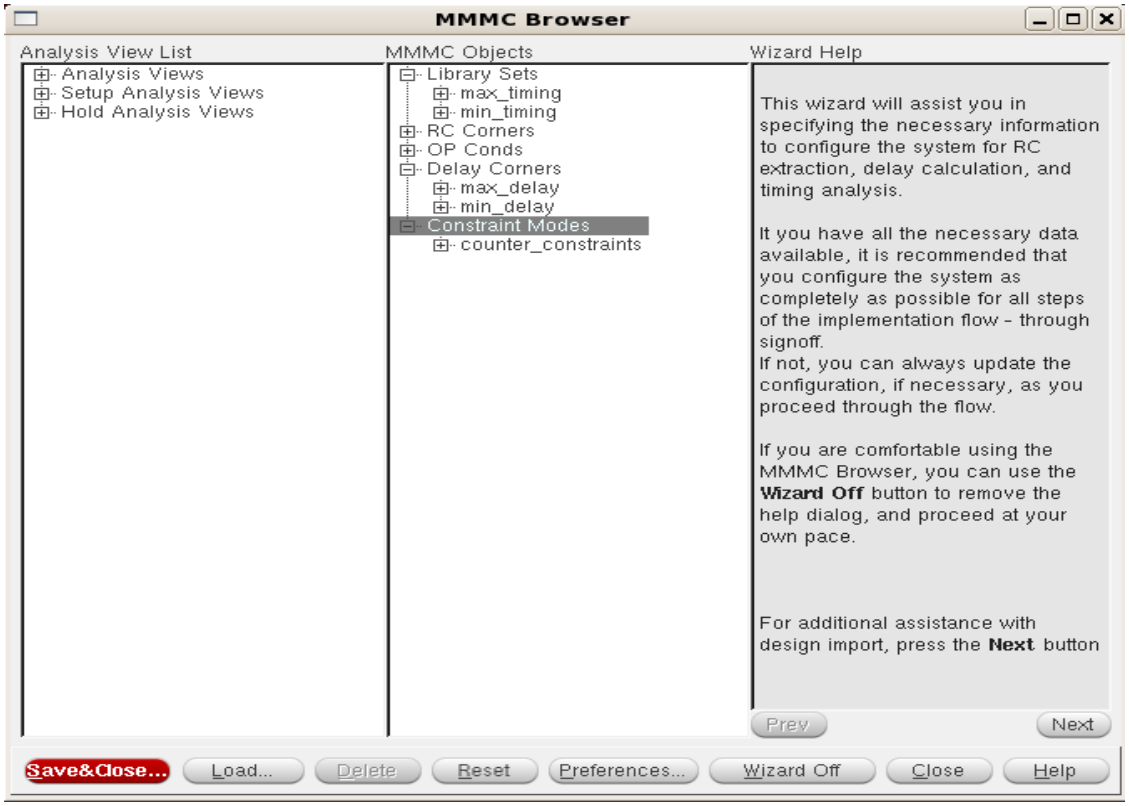
- Give any relevant name for e.g. counter_constraints



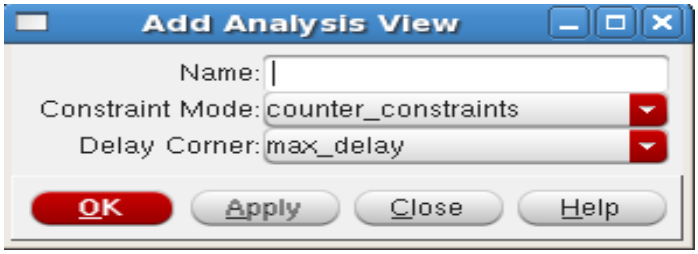
- Click on ‘Add’ under ‘SDC Constraint Files’ and browse to your constraints file(.sdc) that you have created from synthesis stage using ‘write_sdc’ command then click on ‘Add’ and click on ‘close’



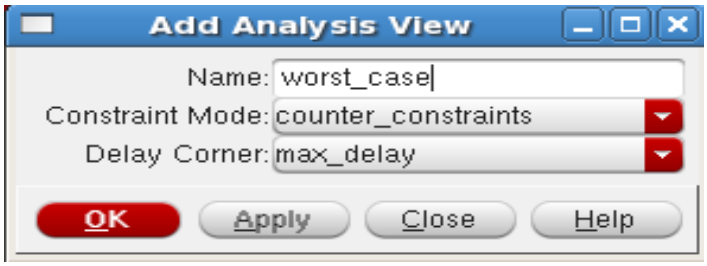
- After clicking on close the MMMC Browser should look like below figure.



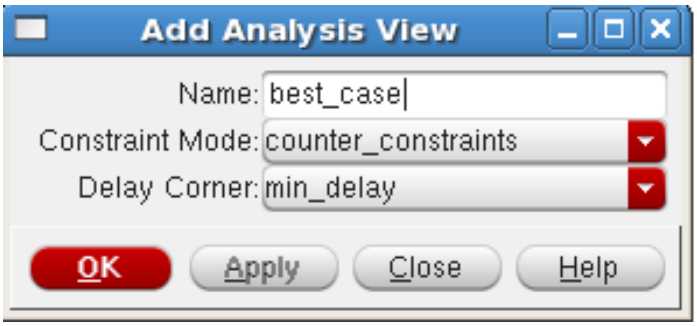
- Now we need to setup analysis for best case and worst case scenario, we will do worst-case analysis for setup and best case analysis for hold.
- Double click on 'Analysis Views', a new Add Analysis View form appears



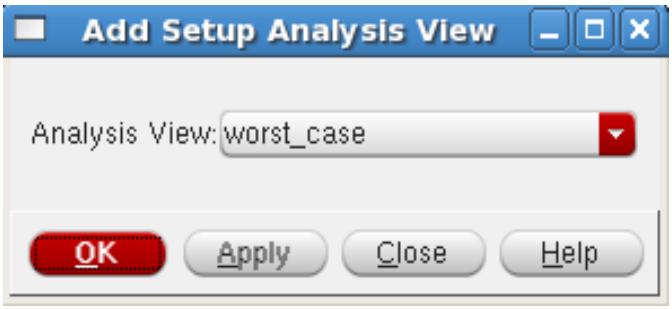
- Give name as worst_case and select max_delay for Delay Corner and click 'OK'.



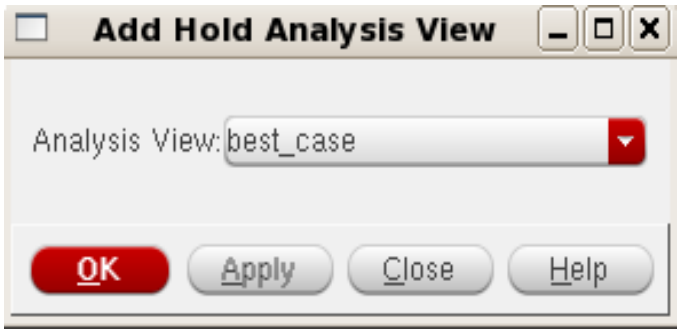
- Again double click on ‘Analysis Views’ a new ‘Add Analysis View’ form appears, give name as best_case ,scroll to min_delay and then click ‘OK’.



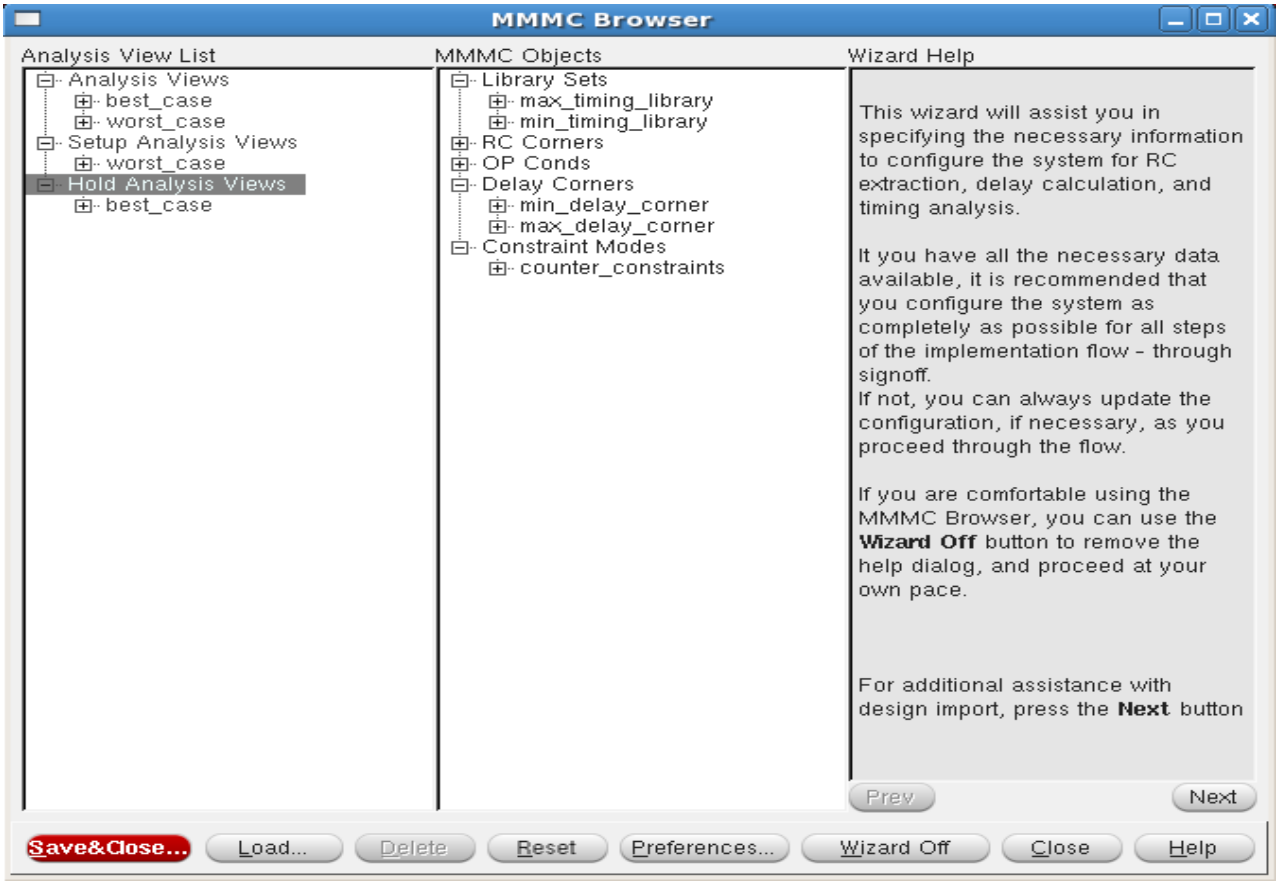
- Double click on Setup Analysis Views, in the ‘Add Setup Analysis View’, the Analysis view should be set to worst_case, if it is set to best_case then scroll to worst_case and click ‘OK’.

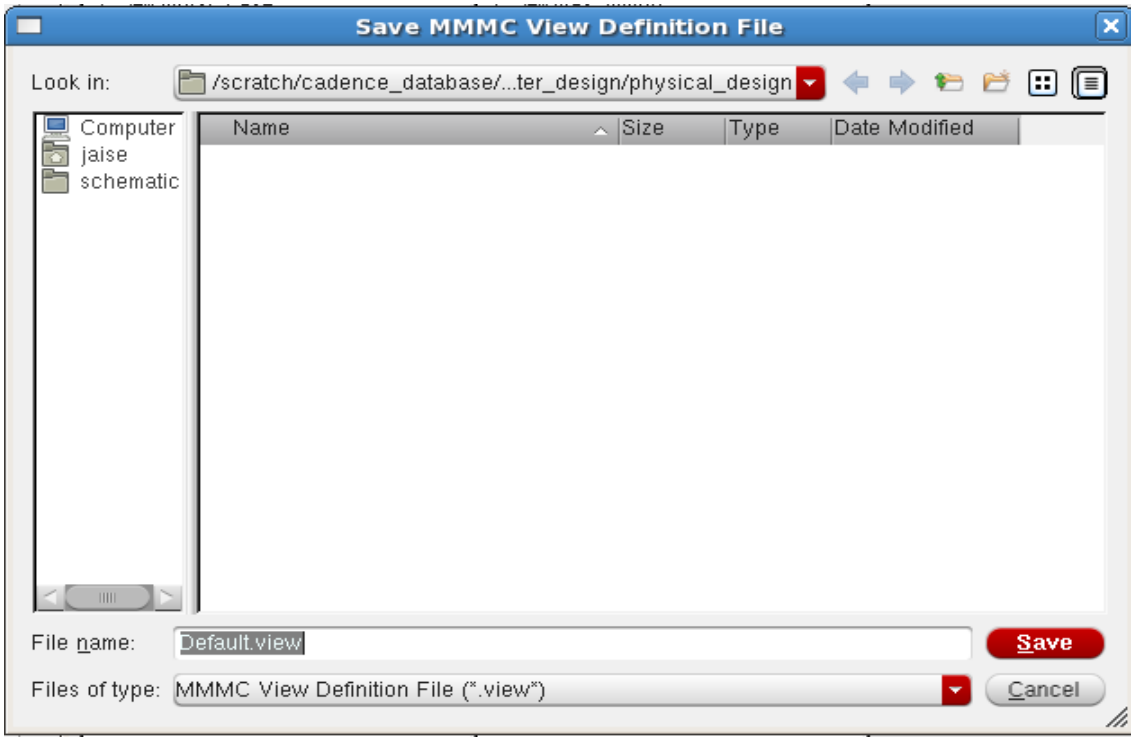


- Double click on ‘Hold Analysis Views’, in the ‘Add Hold Analysis View’ the Analysis view should be set to best_case, if it is set to worst_case then scroll to best_case and click ‘OK’.

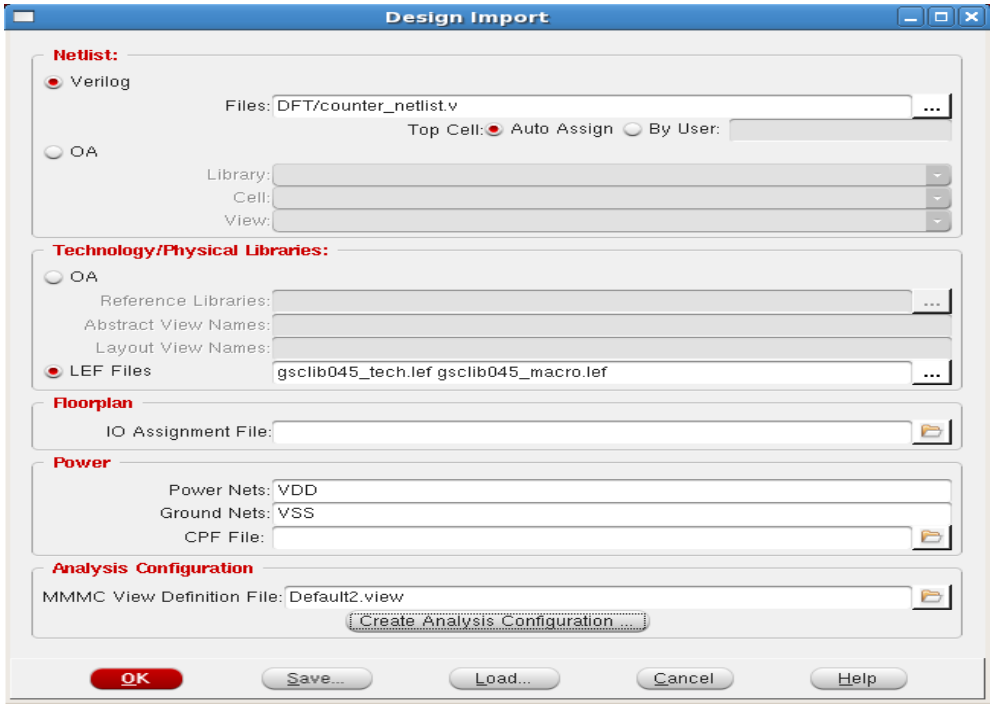


- After clicking ok the MMMC Browser should look like below. Click on Save & Close.

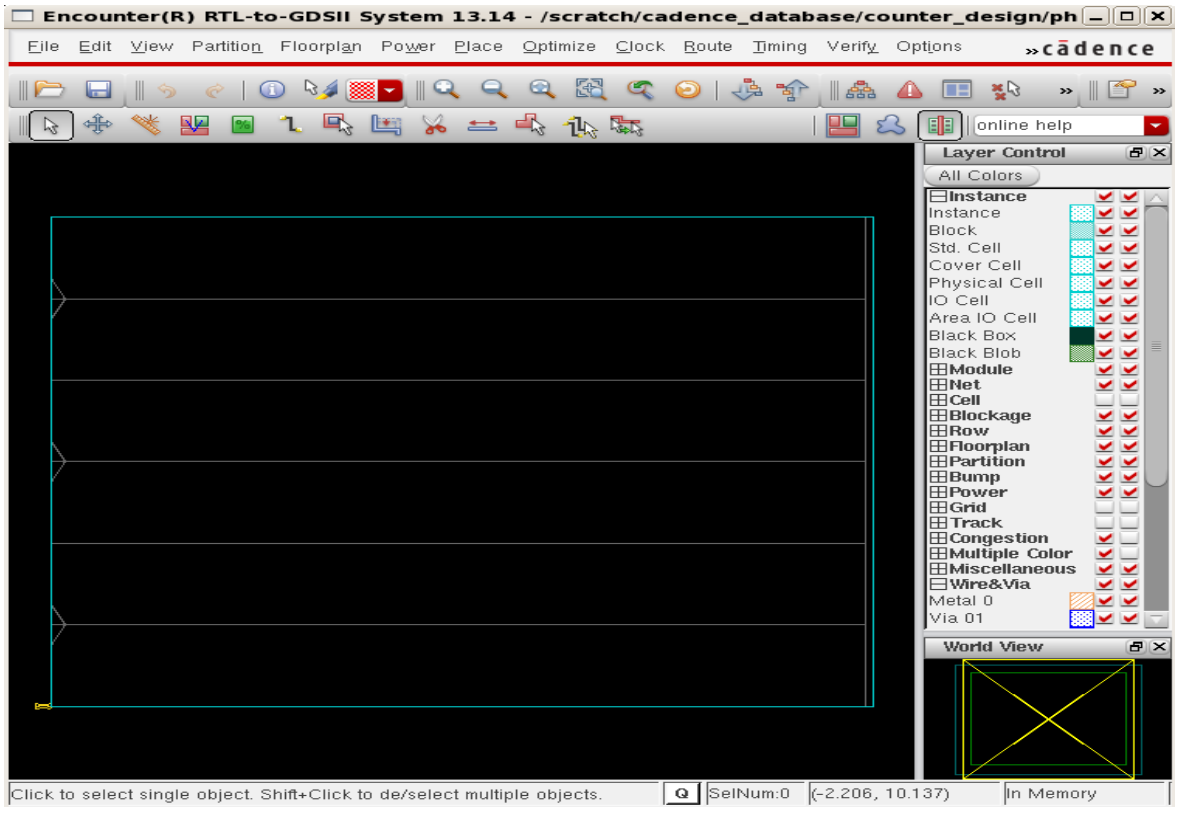




- After saving the design import form appears and it should look like the below figure and Click on 'OK'.



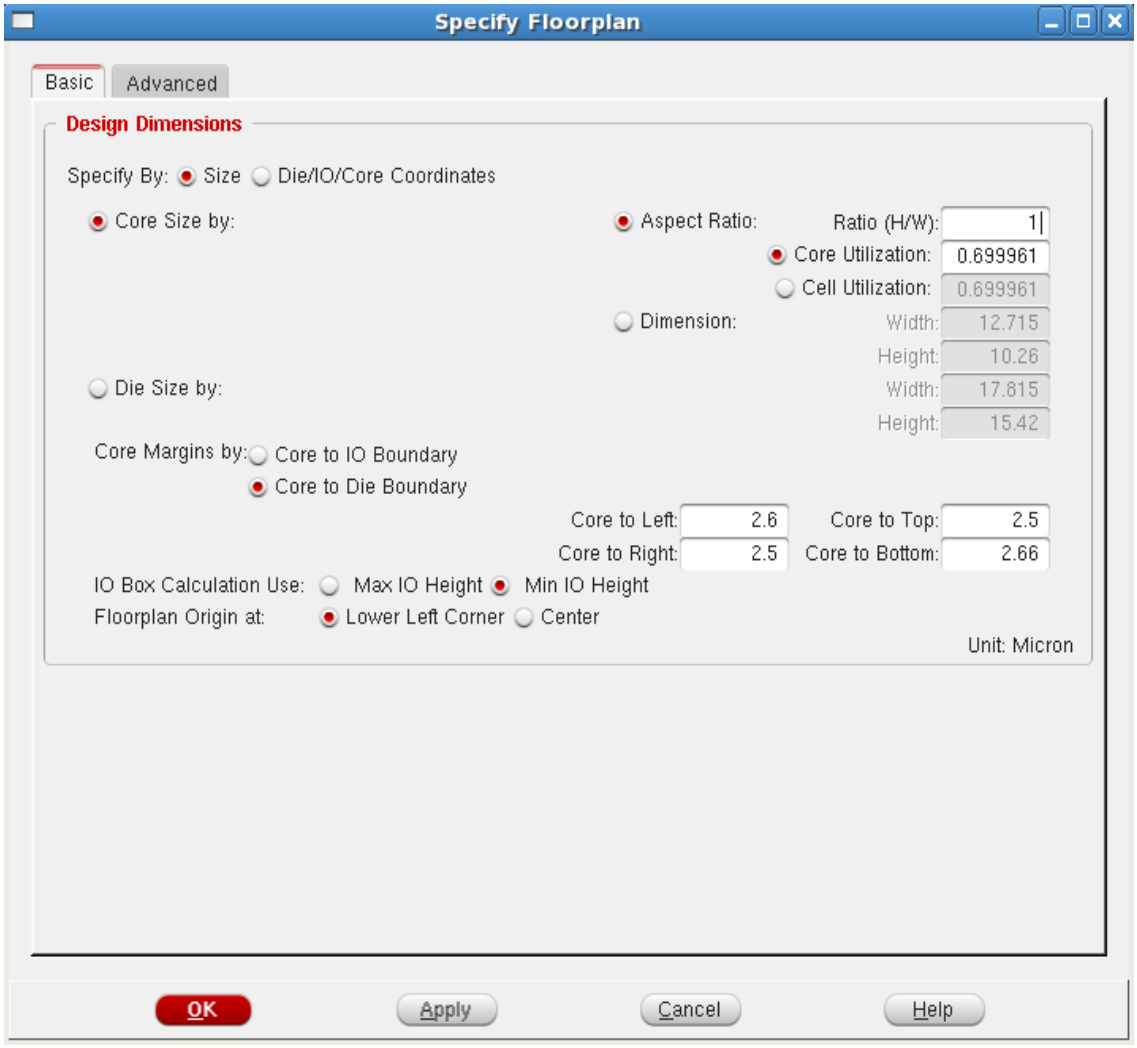
- The below window appears and you can notice at the bottom right corner that the design is in memory now i.e. the design has successfully been imported. Press f (to fit it screen).



Floor Planning

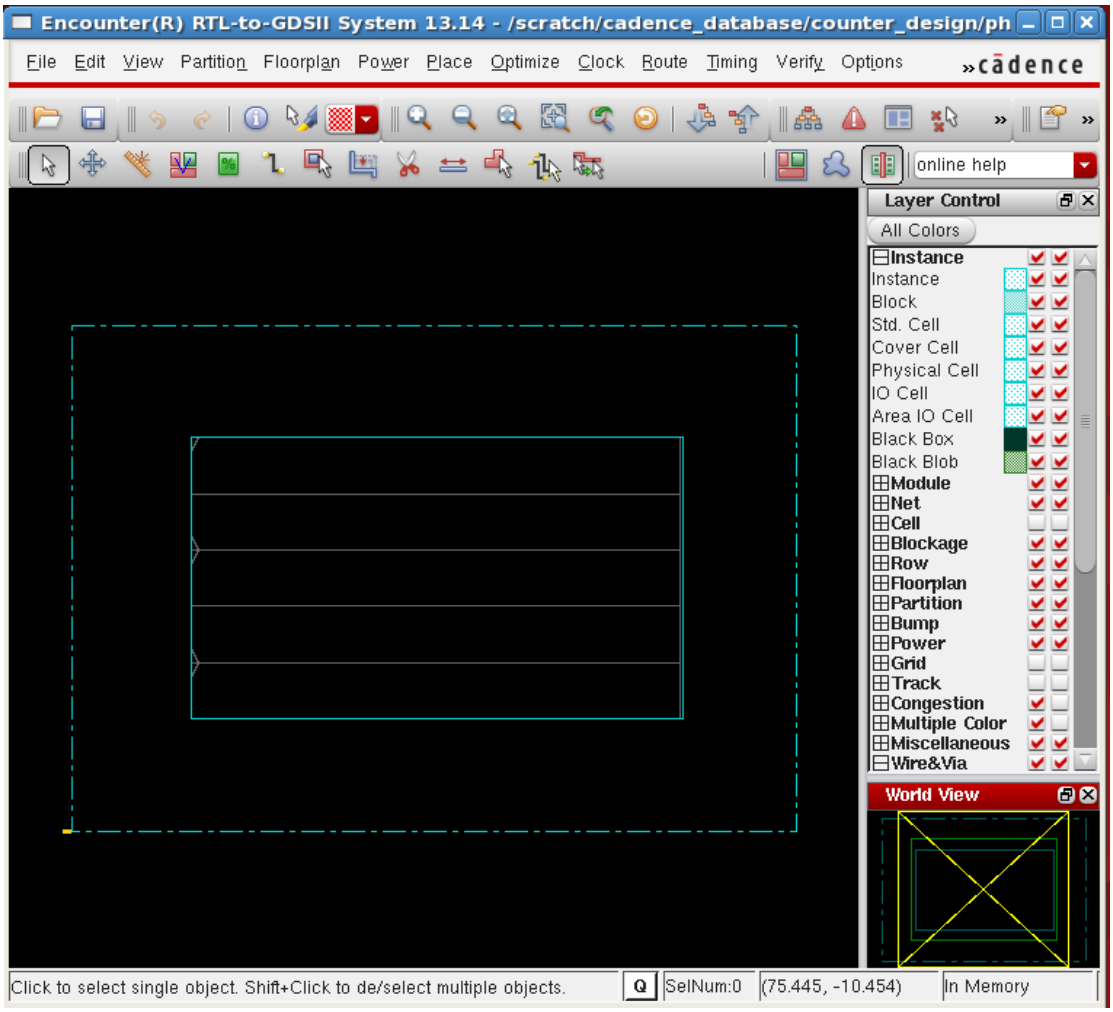
Floor Plan defines the actual form or aspect ratio, the layout will take, the global and detailed routing grids, the rows to host the core cells and the I/O pad cells (if required), the area for power rings, the (pre)placement of blocks/macros, and the location of the corner cells (if required). After the design import, an initial floorplan is displayed in the display area.

- Click 'Floorplan' -> 'specify floorplan' and a 'Specify Floorplan' window will open.



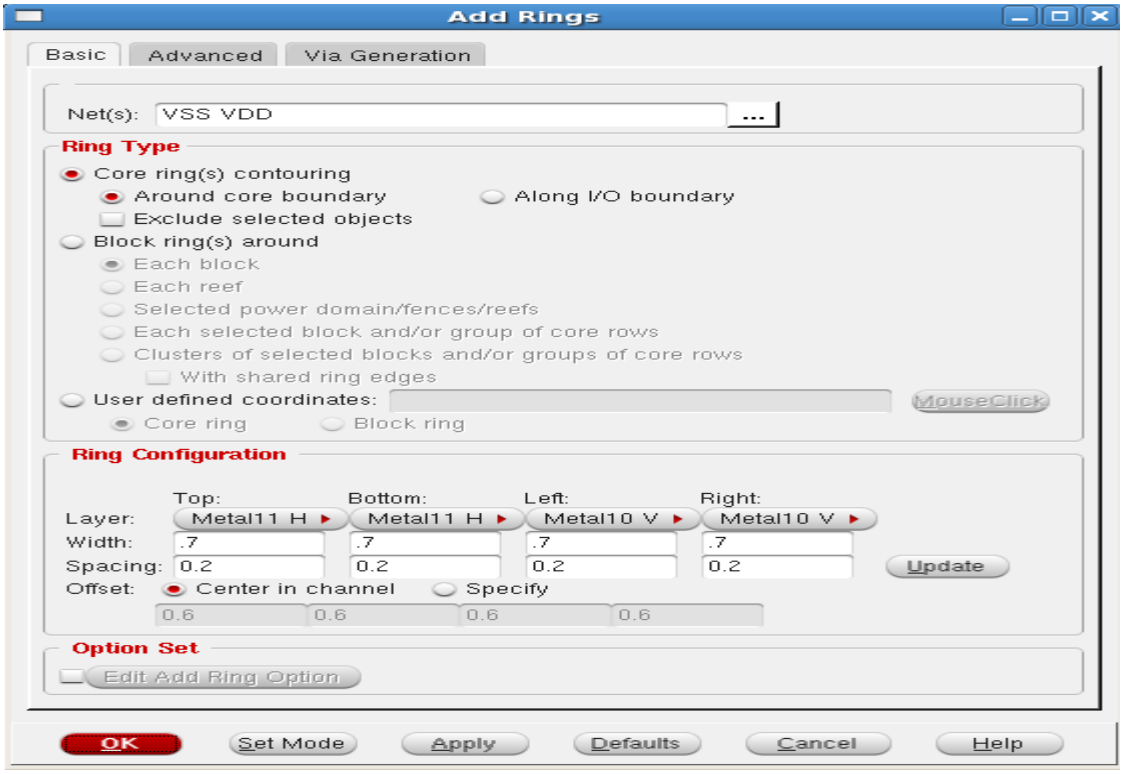
- Select the Aspect Ratio as per the requirement. Give some dimension in "Core to left", "Core to right", "Core to top","Core to bottom". E.g. give 2.5 to each. This is to create the space for Power rings which will be created in power planning. After defining core area, click 'OK'

- After Floor Planning, the Encounter window will look like the below image.

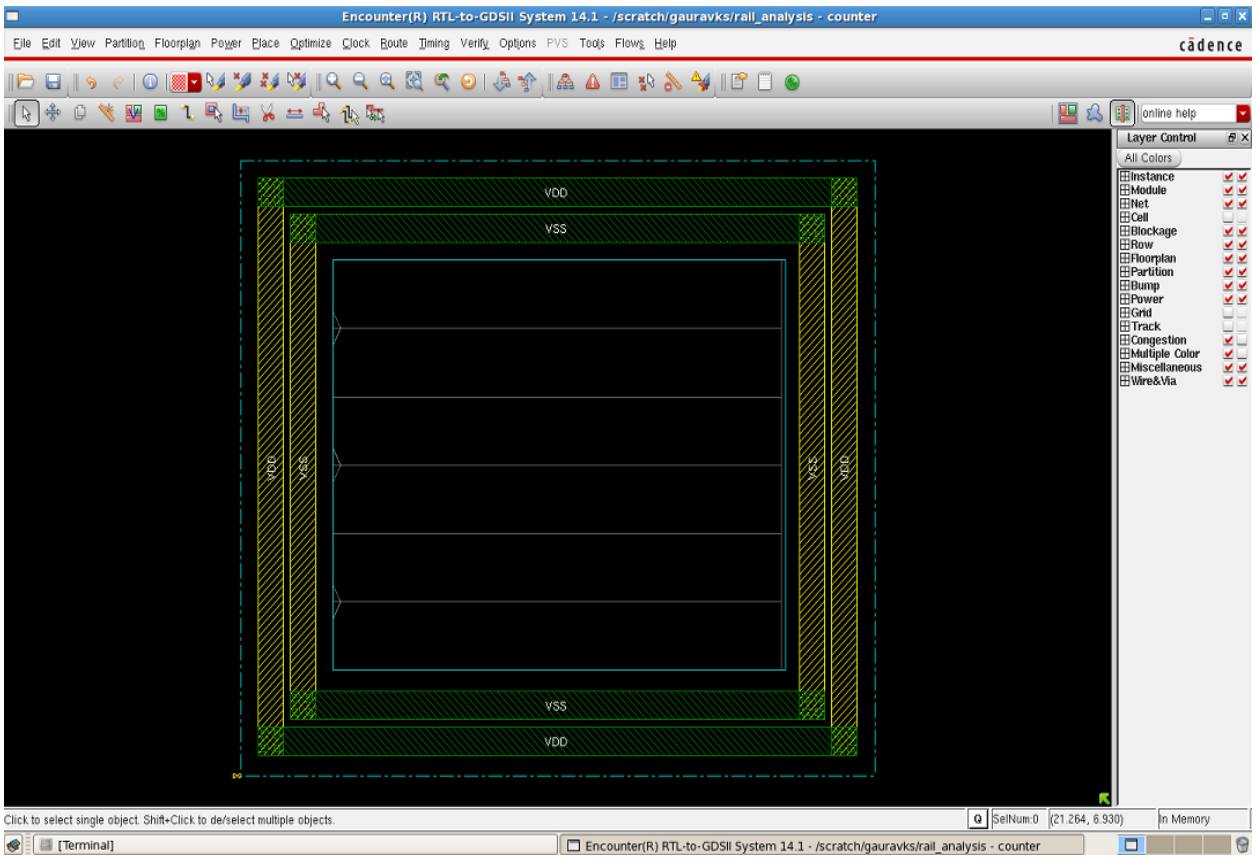


Power Planning

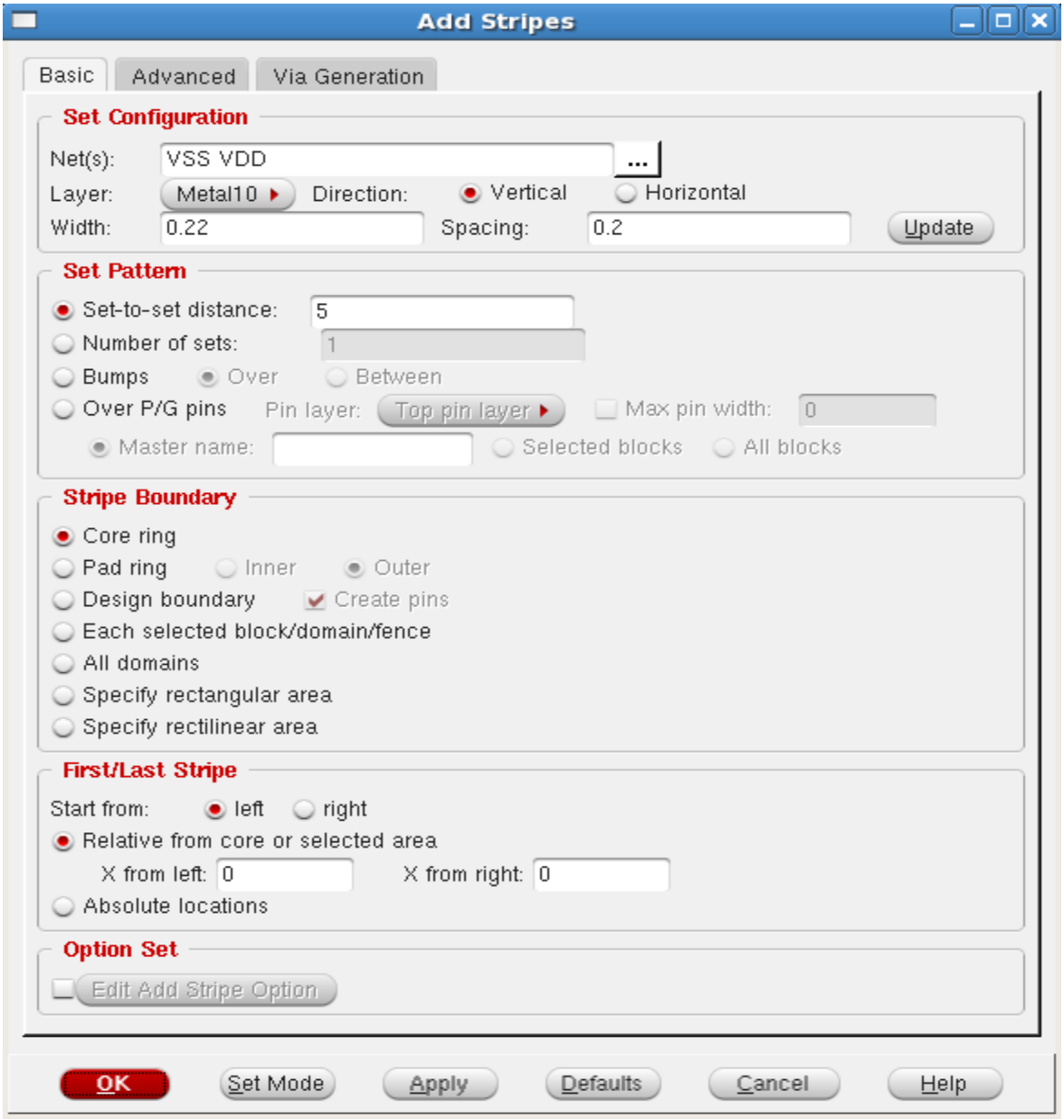
- Click on power -> power planning -> Add Rings and the ring window pops up.
- Select the VDD and VSS power nets under the net option using the browse keys.
- Select the Top and Bottom layer as Metal11, Left and Right as Metal10 (For 45nm technology total 11 metal layers are available. Top and Bottom layers are Horizontal layers and Highest Horizontal metal layer available in the library is Metal 11. Similarly Left and Right layers are Vertical layers and Highest Vertical metal layer available in the library is Metal 10). Set the width as per the requirement and taking the space between core boundary and I/O pad considerations. Select the option for offset as “center in channel” and click OK.



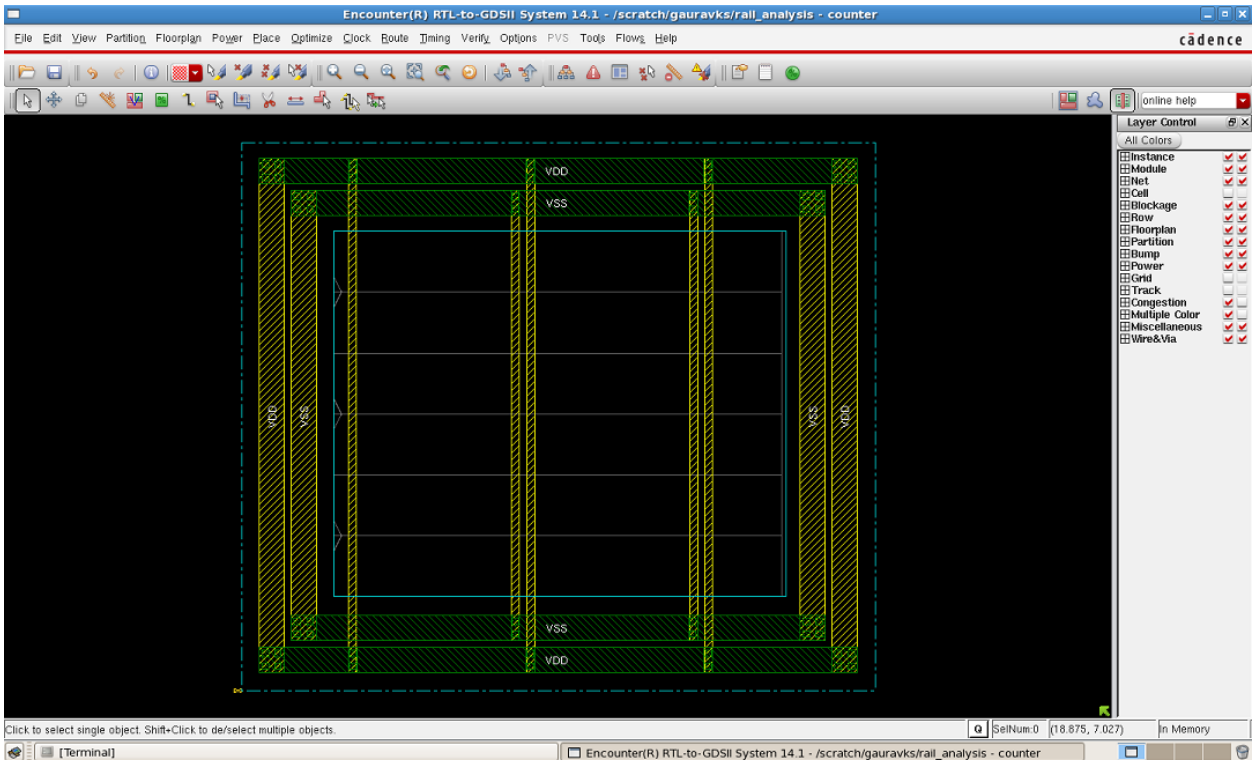
- The power ring will get created as shown in the below image.



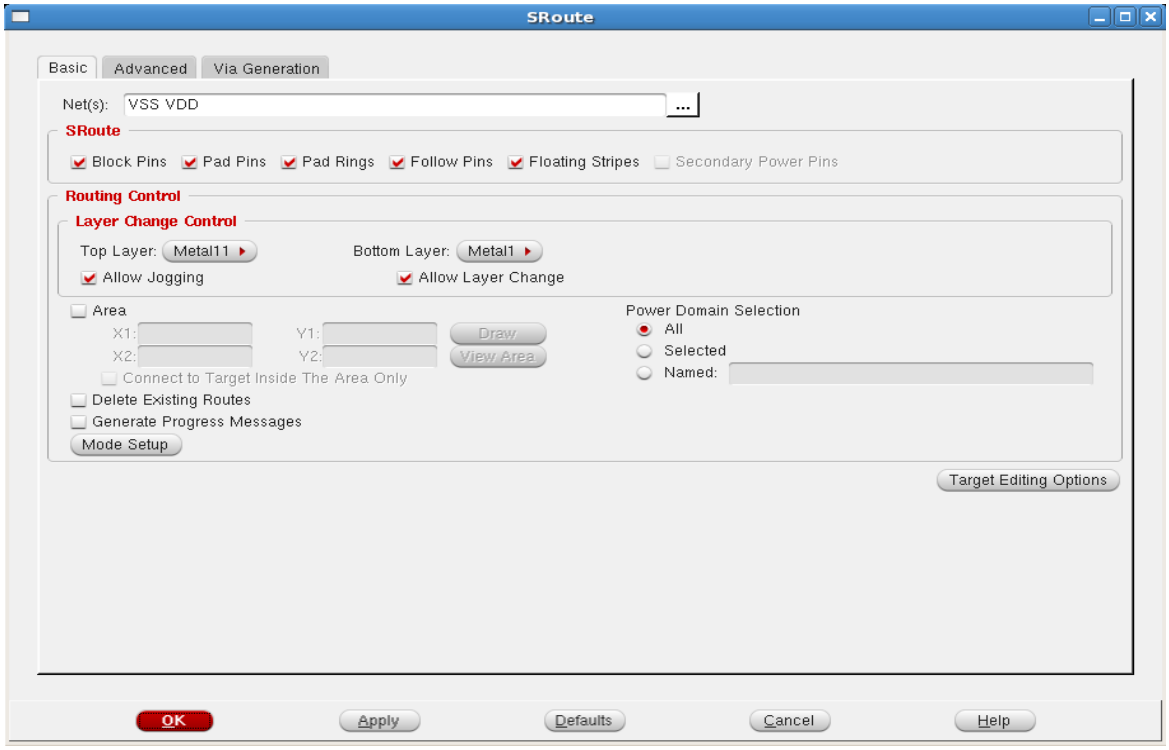
- The next step in power planning is to create power strips. Select Power, click Power Planning and click Add Stripe.



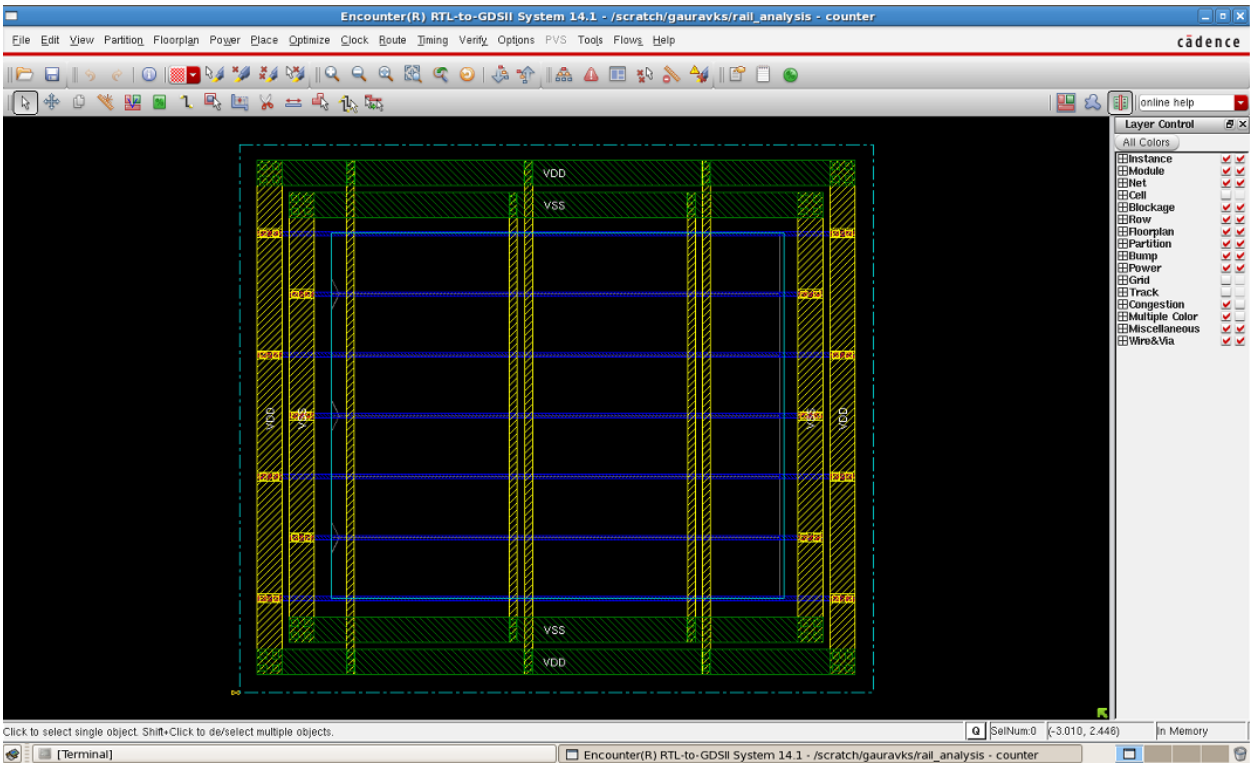
- Select the VDD and VSS power nets under the net option using the browse keys.
- For adding the stripes, select metal layer as Metal 10 and chose direction as vertical (if direction chosen is horizontal, chose metal layer as Metal 11). Click OK and the design will get the vertical thin strips of type Metal 11. Top metal layers are selected for power routing because they compensate very less resistance, so that drop can be reduced.
- Specify the width and spacing between the stripes and also specify the number of sets (of stripes) under the set pattern option. And click 'OK'.
- After adding the power stripes, the Encounter window will look like the image shown below.



- Now save the floorplan file (counter.fp) as, file→save→floorplan. Click ok.
- After the power planning, go to Route -> Special Route. A new Window Sroute will appear.
- Select the VDD and VSS power nets.

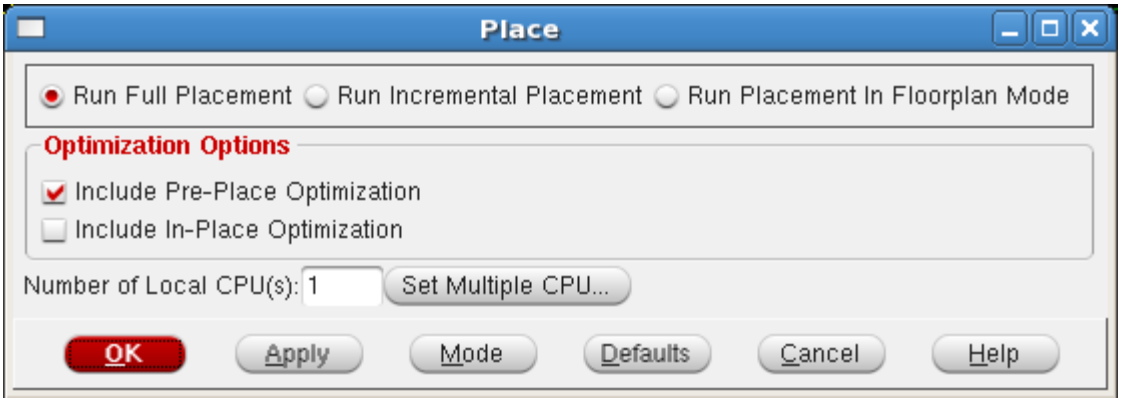


- Click OK with all default settings. This is done to provide power to standard cells. The horizontal blue colored metal1 stripes as shown in the below image are created as a result of Special Route.

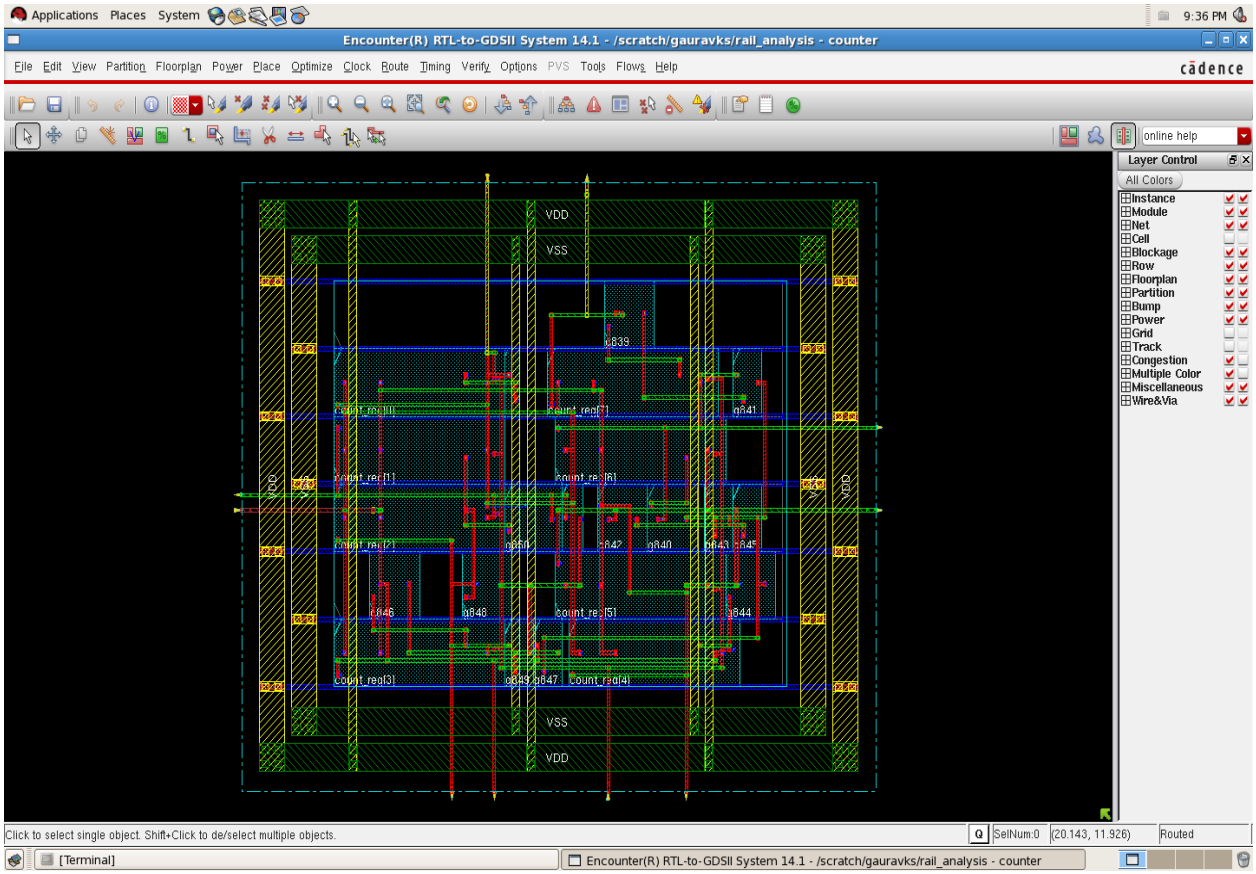


Placement

- This step places the standard cells in the rows, according to the imported Verilog netlist.
- Select 'Place' -> 'Place Standard Cells' in the main menu.



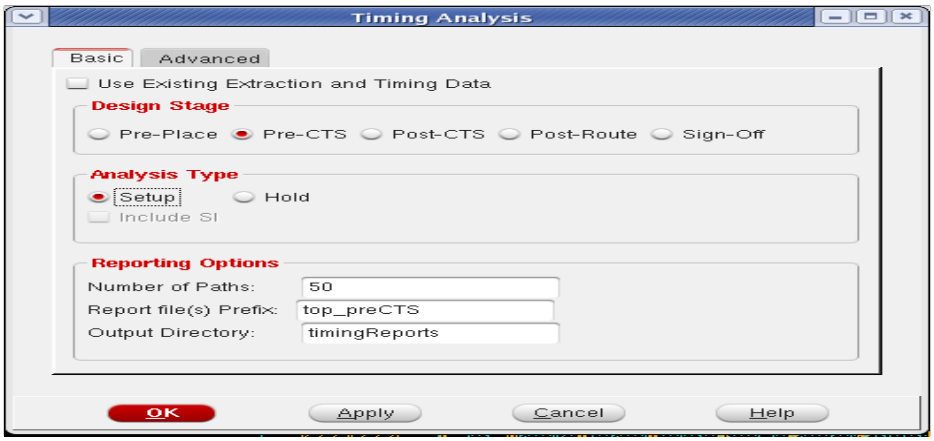
- Select Place > Check Placement... in the main menu to get information regarding Placement density, unplaced and placed cell status.
- Click OK on Place window and in physical view the blue colored standard cells can be seen as a result of placement of standard cells.



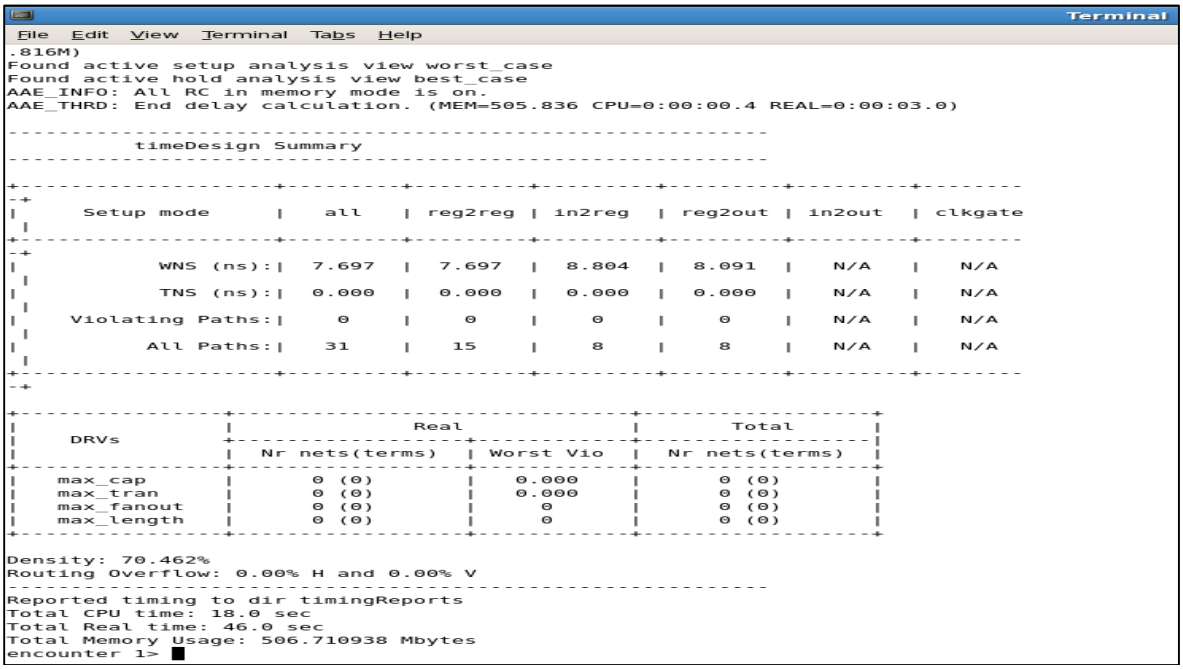
- The multi-coloured lines visible in the tool window are the connections between standard cells using metal layers. If any part of this design is Zoom-in, metal layers can be viewed easily.

Pre-CTS Timing

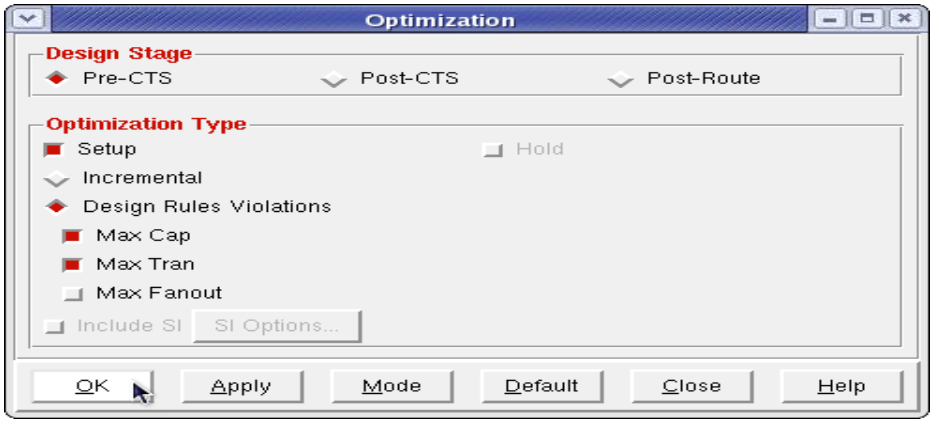
- Before CTS, timing analysis has to be done for any setup violations.
- Click on Timing, and select Report Timing. A 'Timing Analysis' window will get open. In the window select the 'Pre-CTS' as Design Stage and select the 'Setup' as Analysis Type.



- Click OK to complete the Timing analysis. The timing information will get display on terminal in tabular form. In the table displayed on the terminal under “timeDesign Summary”, check for any negative value under WNS (Worst Negative Slack) and TNS (Total Negative Slack). The terminal will look as the image below.



- If there is any of the negative slack value under WNS or TNS, we need to optimize our design.
- Select ‘Optimize’ -> ‘Optimize Design’ in the main menu. Check the Pre-CTS box. The other default selection of boxes asks to correct setup, max capacitance and max transitions violations. The tool will optimize the design and the optimized timing results will be displayed over terminal again.



- During optimization the tool will perform following things.
  - Adding buffers
  - Resizing gates
  - Reconstructing the circuit
  - Remapping the logic
  - Swapping the pins

- Deleting the buffers
- Moving the instances

➢ Once you done with timing analysis tool internally dump out timing report directory.

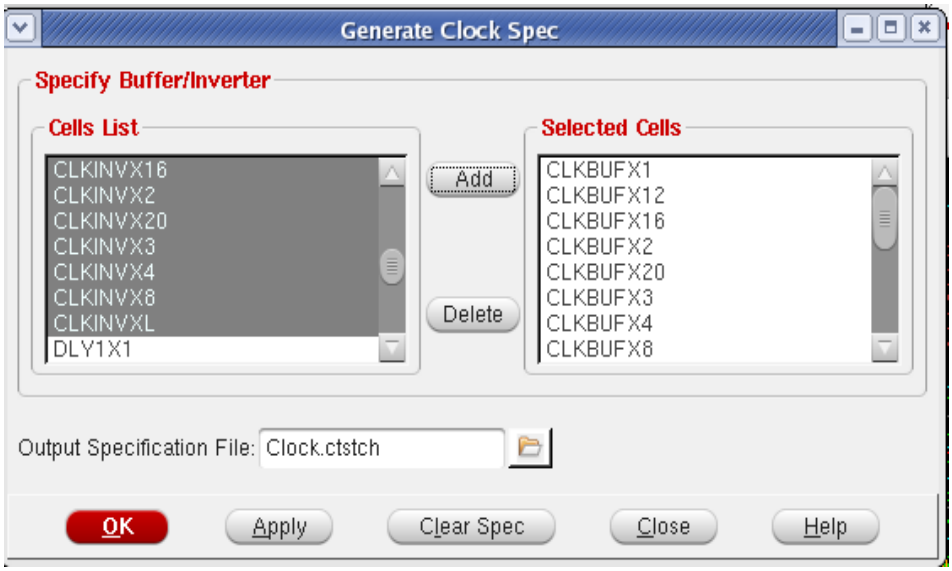
Clock Tree Synthesis

➢ As the paths that will propagate the clock signal in the design are not necessarily balanced, some registers may receive the active clock edge later than others (clock skew) and may therefore violate the assumed synchronous design operation.

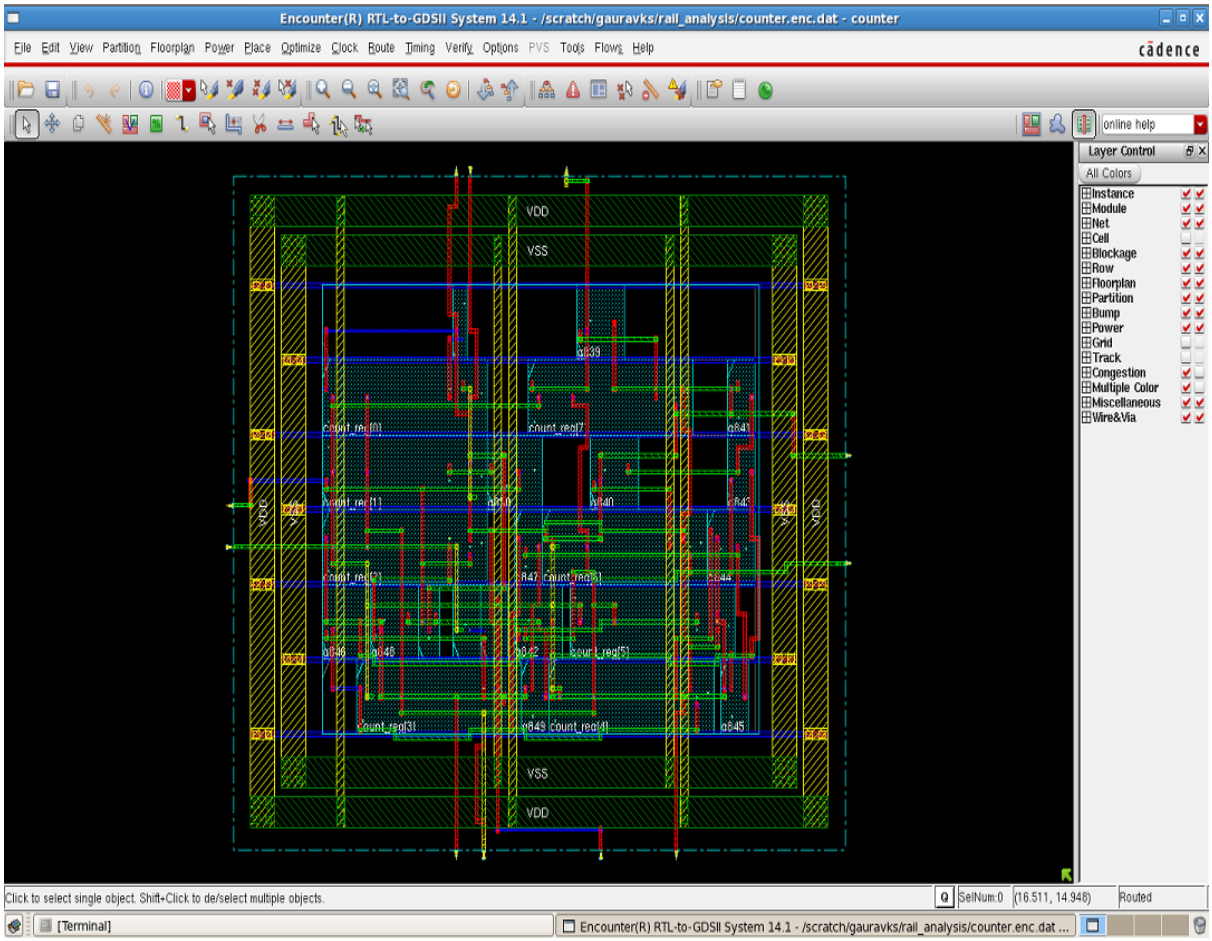
➢ To create a balanced clock tree, you have first to create a clock tree specification file. Encounter can create a first draft version of the file you can then edit to specify design specific data.

➢ Select 'Clock' -> 'Synthesis Clock Tree...' in the main menu. Then, in the Basic tab, click the 'Gen Spec' button. A new window "Generate Clock Spec" will open.

➢ From Cells List, Select all cells starting with "CLK" and click on Add button to add them to the Selected Cells. Select a name for Output specification and click 'OK'.



➢ Specify a name for 'Results Directory' and click OK. The tool window looks like the image below.



Post-CTS Timing

- Again Perform the Timing by clicking on Timing and selecting Report Timing. Select 'Post-CTS' under Design Stage and do the select 'Set-up' as Analysis Type. Check the terminal for timing violations and optimize the design if any violation exists.
- Similarly check for Hold violations and optimize if violation exists.

Note: Steps to overcome timing violations in counter design are mentioned in STA session using ETS

Routing the Design

- This step generates all the wires that are required to connect the cells as defined in the imported Verilog netlist.
- Select 'Route' -> 'NanoRoute' -> 'Route' in the main menu. Check the Timing Driven box. A higher value increases the effort toward meeting the timing constraints and decreases the effort toward relieving congestion. Click OK to start the routing.

- The tool will perform the Routing and the Routing statistics can be seen on terminal window including DRC violations.

Post-Route Timing Analysis

- **Optimizing Timing in On Chip Variation (OCV) Analysis Mode:**

Optimize timing in on-chip variation (OCV) analysis mode to account for variations in process, voltage, and temperature (PVT) across the die. When it takes OCV into account, the software calculates early and late delays, and uses them to evaluate setup and hold timing checks. You introduce the delays into the

analysis by specifying different min/max corner timing libraries and operating conditions. Early/late variation might also be present due to slew merging effects of multiple input gates in the clock path.

To enable the software to consider multiple libraries and operating conditions, you must specify a multi-mode/multi-corner (MMMC) environment during the import design. If the MMMC environment is not specified, and you try to run timing optimization in OCV mode, the optDesign command exits with an error message.

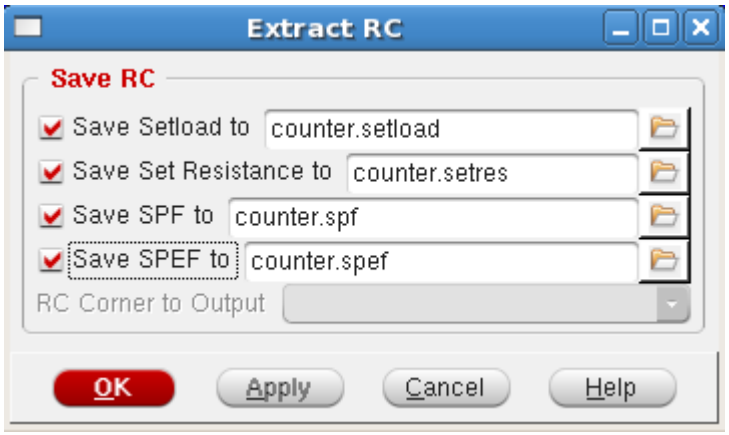
Use the following command to enable the On Chip variation (OCV) engine in the encounter terminal before Post-route timing analysis:

setAnalysisMode –analysisType onChipVariation

Perform the timing analysis again to check for violations and optimize if any.

RC Extraction

- Now do the RC extraction using **Timing →Extract RC→** save the files in different formats (counter.setload, counter.setres, counter.spf, counter.spef).



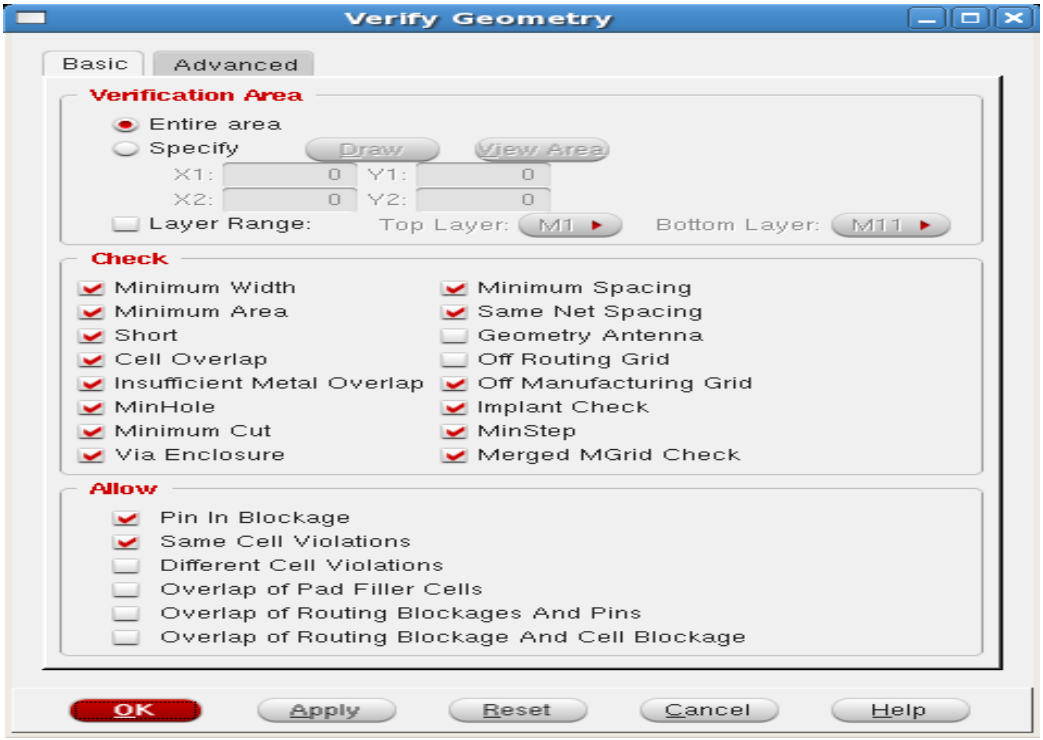
- Click OK. All the selected files will be created in the current directory.

Physical Verification

The Verify menu has a number of items to check that the design has been properly placed and routed.

Verify Geometry

- Select 'Verify' -> 'Verify Geometry' in the main menu.
- If you face any violation and check it out those violations to tools->violation browser



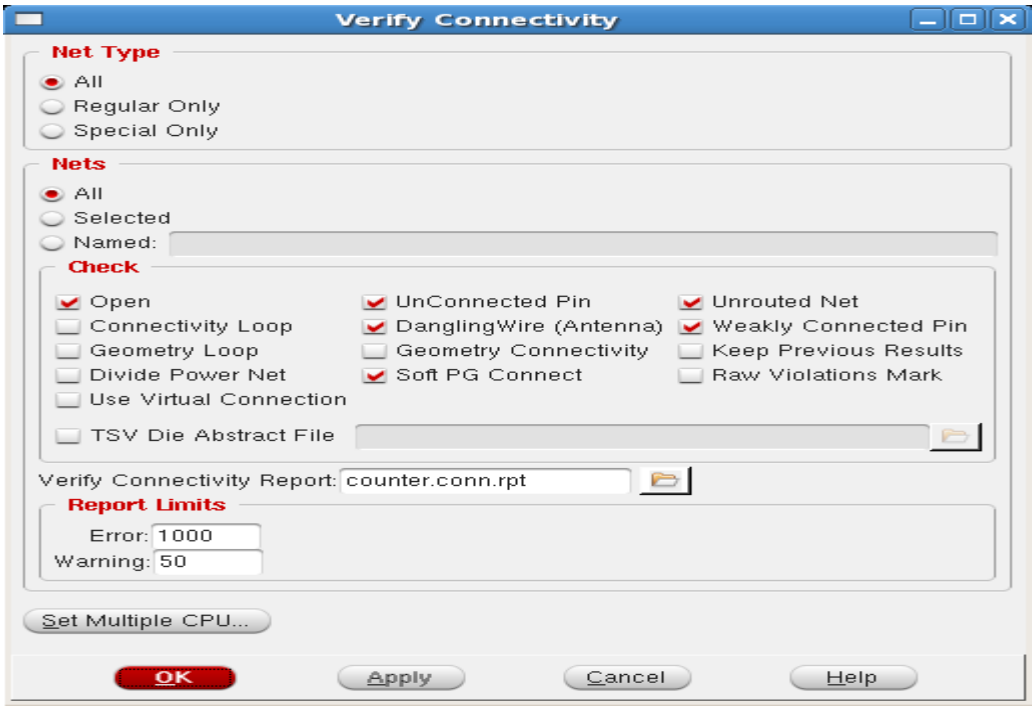
- We can see the result on the terminal window as

```
encounter 1>  *** Starting Verify Geometry (MEM: 584.1) ***

  VERIFY GEOMETRY ..... Starting Verification
  VERIFY GEOMETRY ..... Initializing
  VERIFY GEOMETRY ..... Deleting Existing Violations
  VERIFY GEOMETRY ..... Creating Sub-Areas
                  ..... bin size: 1920
  VERIFY GEOMETRY ..... SubArea : 1 of 1
  VERIFY GEOMETRY ..... Cells           :  0 Viols.
  VERIFY GEOMETRY ..... SameNet         :  0 Viols.
  VERIFY GEOMETRY ..... Wiring          :  0 Viols.
  VERIFY GEOMETRY ..... Antenna         :  0 Viols.
  VERIFY GEOMETRY ..... Sub-Area : 1 complete 0 Viols. 0 Wrngs.
VG: elapsed time: 64.00
Begin Summary ...
  Cells       : 0
  SameNet     : 0
  Wiring      : 0
  Antenna     : 0
  Short       : 0
  Overlap     : 0
End Summary

  Verification Complete : 0 Viols.  0 Wrngs.

**********End: VERIFY GEOMETRY**********
 *** verify geometry (CPU: 0:00:50.4  MEM: 173.8M)

encounter 1>
```

Verify connectivity

- Select ‘Verify’ -> ‘Verify Connectivity’ in the main menu. Click ‘OK’.

- If you face any violation and check it out those violations to tools->violation browser

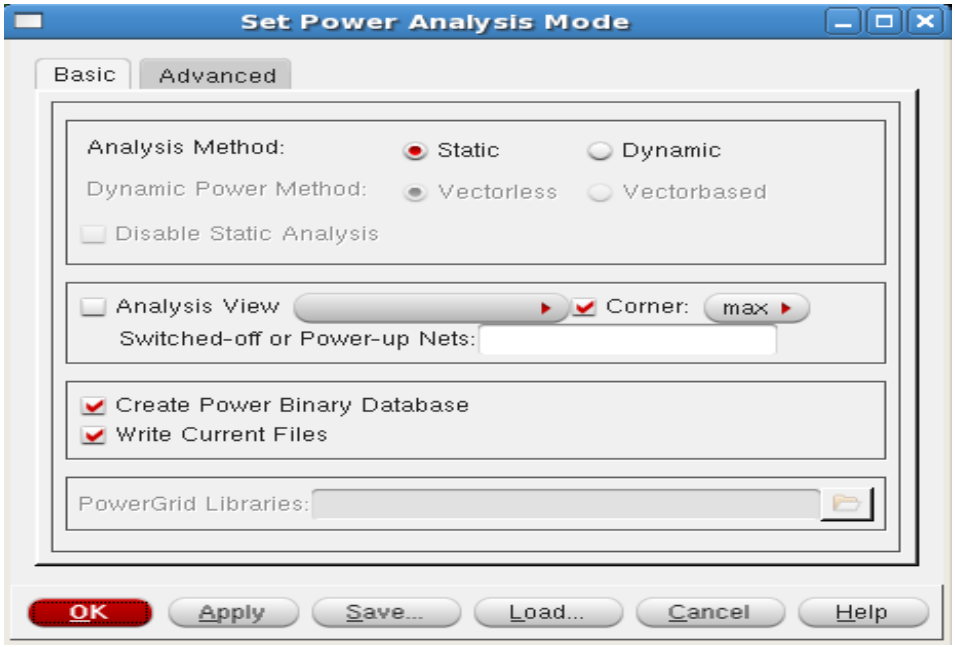


We can see the result on the terminal window as

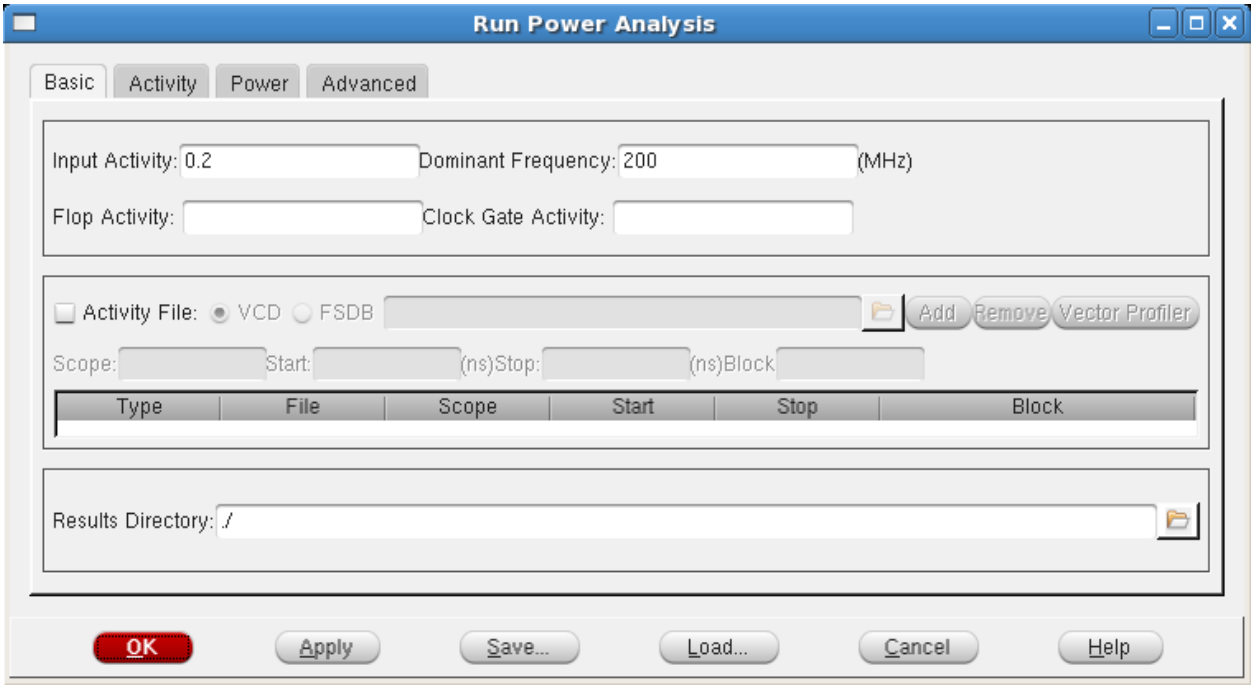
```
encounter 1> VERIFY_CONNECTIVITY use new engine.

******** Start: VERIFY CONNECTIVITY ********
Start Time: Tue Sep  9 06:55:25 2014

Design Name: counter
Database Units: 2000
Design Boundary: (0.0000, 0.0000) (17.8150, 15.4200)
Error Limit = 1000; Warning Limit = 50
Check all nets

Begin Summary
  Found no problems or warnings.
End Summary

End Time: Tue Sep  9 06:55:25 2014
Time Elapsed: 0:00:00.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.0  MEM: 0.000M)

encounter 1>
```

Power Analysis (EPS Engine integrated inside EDI)

- Now we carry out the power analysis as Select **Power→Power planning→setup**
  - Analysis Method: Static
  - Corner: max



- Then click OK to enable the Run option. Select **Power→Power planning→Run**
  - Dominant Frequency: 200MHz
  - Then click OK with all other default settings.



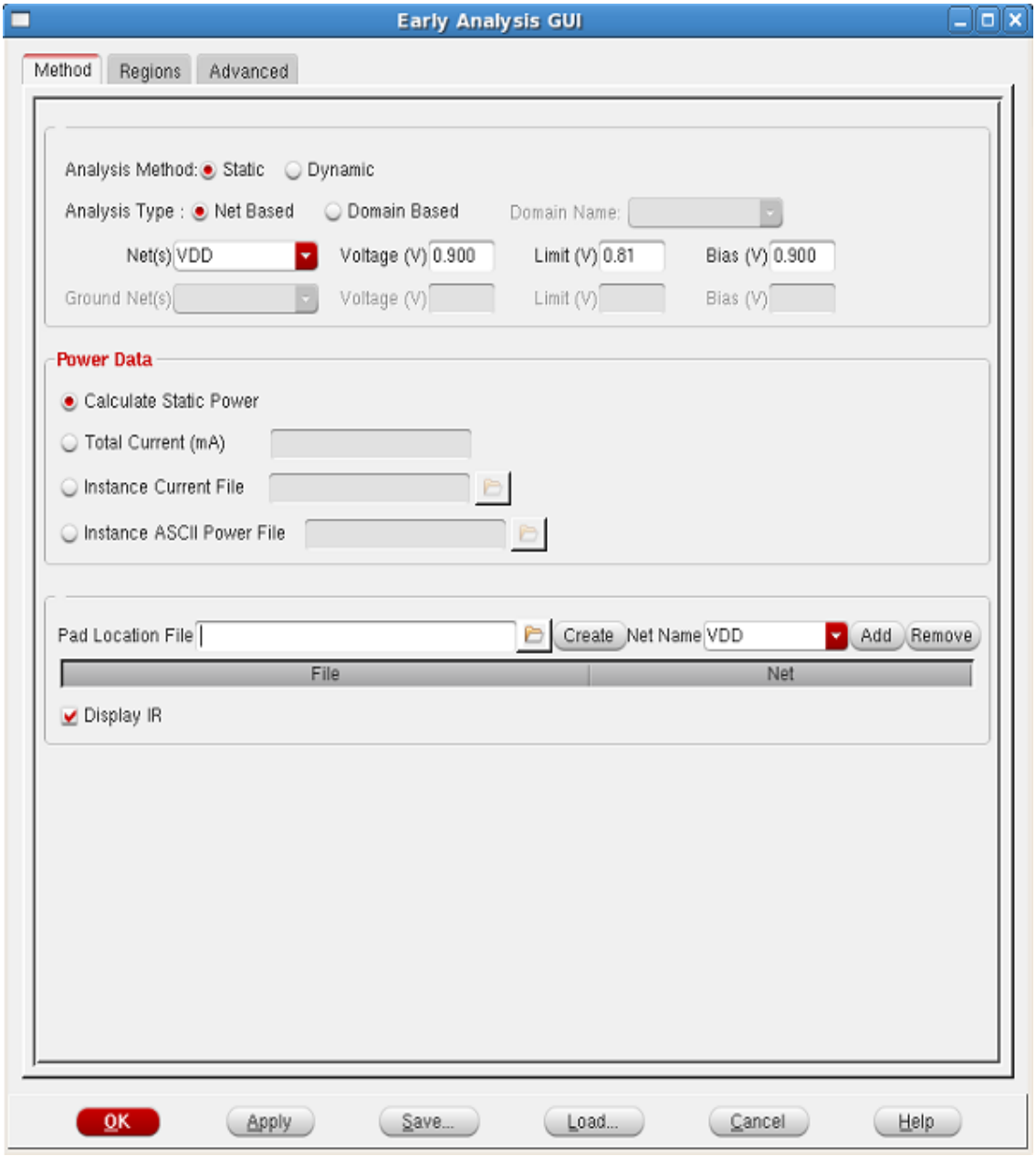
- The terminal window will come with the result as:

```
Total Power
--------------------------------------------------------------------------------
Total Internal Power:        0.00678660            79.1172%
Total Switching Power:       0.00178923            20.8586%
Total Leakage Power:         0.00000208             0.0243%
Total Power:                 0.00857791
--------------------------------------------------------------------------------
Write power database file './/power_2.db'

Output file is .//counter.rpt

encounter 14>
```

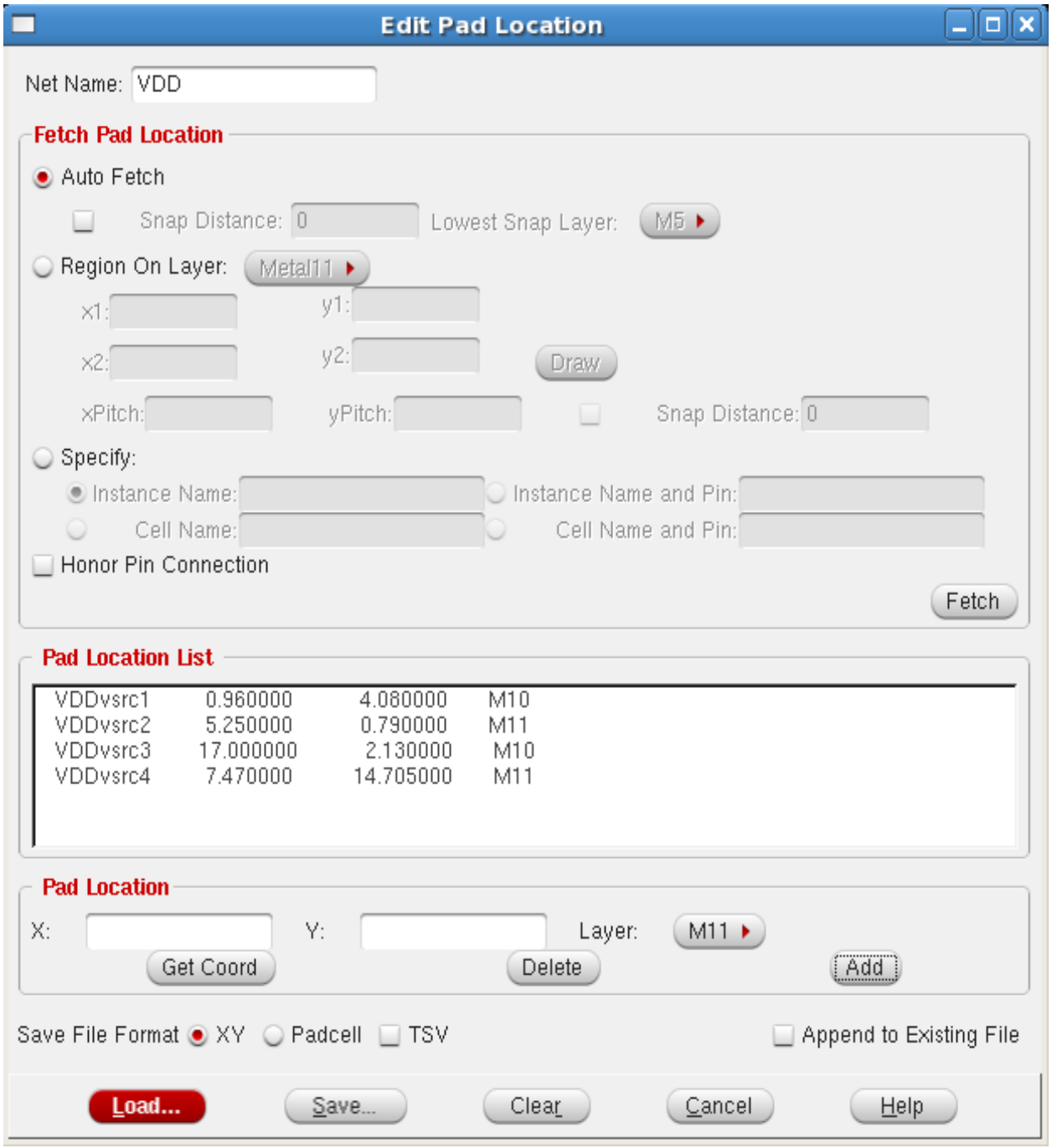
- Before going to the rail analysis we need to source the power.tcl file in encounter prompt as: source power.tcl

```
globalNetConnect VDD -type pgpin -pin VDD -inst *
globalNetConnect VSS -type pgpin -pin VSS -inst *
globalNetConnect VDD -type tiehi
globalNetConnect VSS -type tielo
globalNetConnect VDD -type tiehi -pin VDD -inst *
globalNetConnect VSS -type tielo -pin VSS -inst *
```

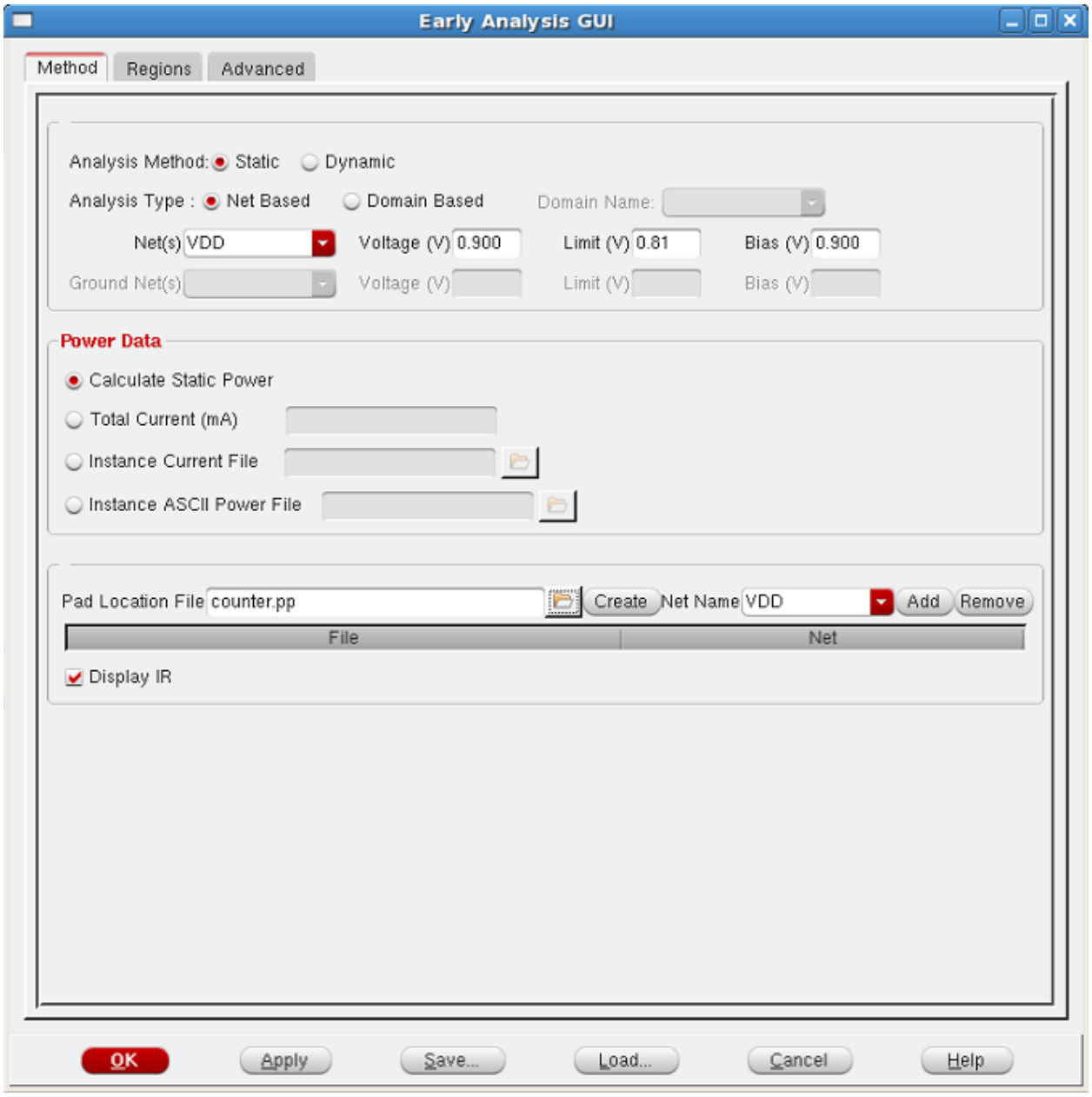
- Select **Power→Rail Analysis→Early Rail Analysis** to get the Early GUI form:
  - Select the net as VDD (0.9 V) and net name as VDD.
  - Now we have to create the pad location file for the VDD and VSS power rings. For that select **create**.



- The **Edit Pad Location** form opens up:
- Select **Get coord** in the pad location (X, Y) to know the coordinates of the VDD ring on the left side and make it **Layer: M11** then **click Add** so it will be updated in the pad location list and a bubble will appear in the GUI for the selected portion of VDD ring.
- Similarly we have to get the pad location using the **Get Coord** in all directions on the VDD ring (**top, right and bottom**). Select the M11 layer for left and right portion of VDD ring and M10 layer for the top and bottom portion of VDD ring.



- Now save the pad location file (**counter.pp**).
  - Use the pad location file as an input in the early rail analysis like **Power→Rail Analysis→Early Rail Analysis .**Here we give **counter.pp** as pad location file input and select Display IR.



- Click ok.

➢ Now see the result in terminal window.

```
Rail Analysis completed successfully.

Finished Rail Analysis at 00:10:11 09/09/2014 (cpu=0:00:18, real=0:00:31, peak mem=806.02MB)
Current Power Analysis resource usage: (total cpu=0:06:15, real=3:52:25, mem=806.02MB)
# Warnings: 0
# Errors: 0

Circuit profile:
                                                    Current   Voltage
Layer Name                    Nodes  Resistors      Sources   Sources
----- --------------------  --------  ---------  --------  --------
  0   Metal11                   18         16         0         0
  1   Via10                      0         10         0         0
  2   Metal10                   55         50         0         0
  3   Via9                       0         20         0         0
  4   Metal9                    20          0         0         0
  5   Via8                       0         20         0         0
  6   Metal8                    20          0         0         0
  7   Via7                       0         20         0         0
  8   Metal7                    20          0         0         0
  9   Via6                       0         20         0         0
 10   Metal6                    20          0         0         0
 11   Via5                       0         20         0         0
 12   Metal5                    20          0         0         0
 13   Via4                       0         20         0         0
 14   Metal4                    20          0         0         0
 15   Via3                       0         20         0         0
 16   Metal3                    20          0         0         0
 17   Via2                       0         20         0         0
 18   Metal2                    20          0         0         0
 19   Via1                       0         20         0         0
 20   Metal1                    52         48        26         0
                           --------  ---------  --------  --------
 21   Circuit total            286        304        26         0
```

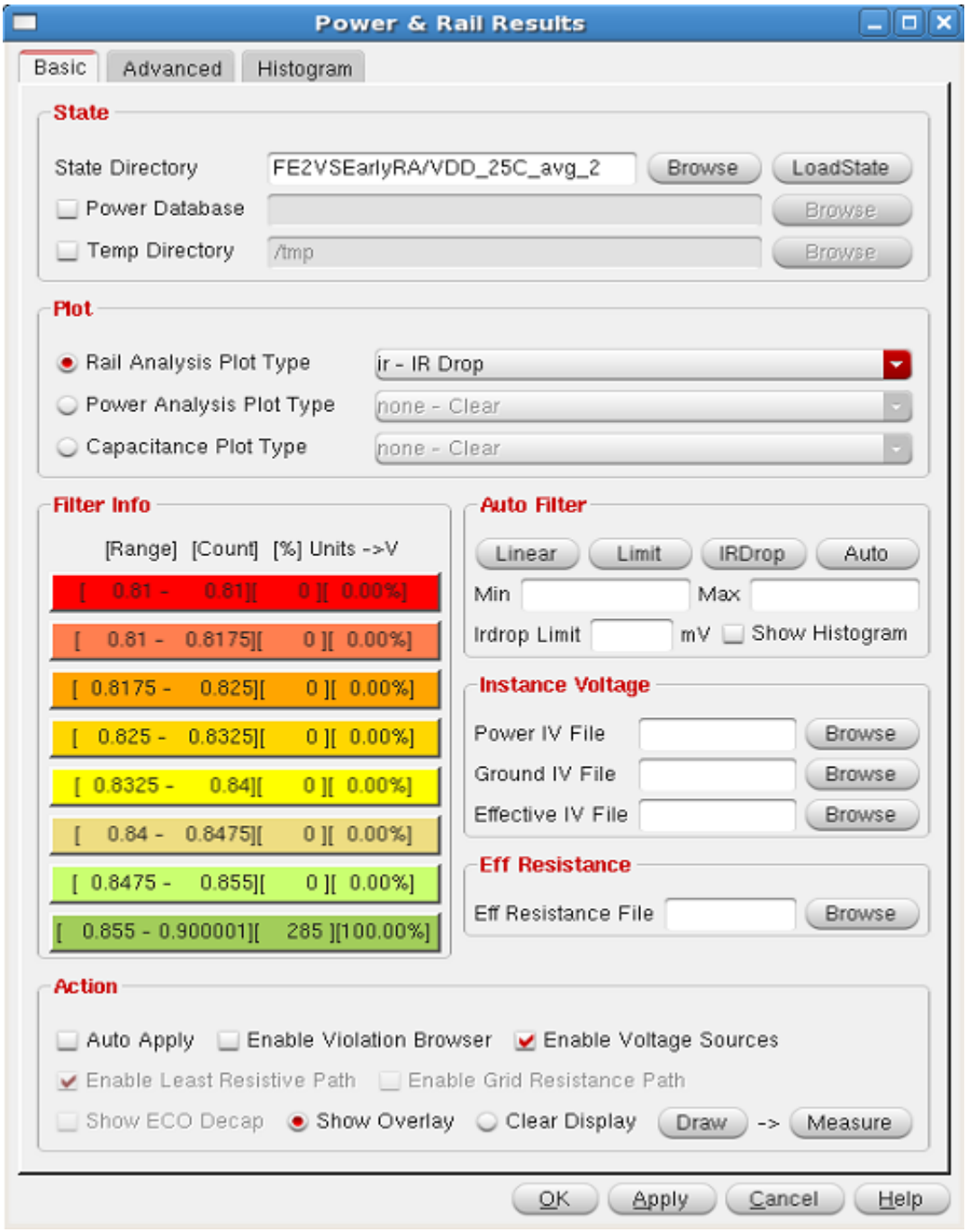
Fig: The above figure explains the successfully execution of the Rail Analysis.

```
* Note: Current sources will not be reset before loading.
Loading current source data file: FE2VSEarlyRA/VDD_25C_avg_2/VDD.tap

Beginning the loading of current source data.

Total current loaded = 9.531016e-06A

Loading of current data required:
                0.01 user, 0.00 system, and 0.01 real seconds

*Added new voltage source VDDvsrc1 (0) at node 13
*Added new voltage source VDDvsrc2 (1) at node 32
*Added new voltage source VDDvsrc3 (2) at node 5
*Added new voltage source VDDvsrc4 (3) at node 34
  Minimum, Average, Maximum IR Drop: 0.900V, 0.900V, 0.900V
    Range 1(    0.810V   -     0.810V ):         0 (   0.00%)
    Range 2(    0.810V   -     0.818V ):         0 (   0.00%)
    Range 3(    0.818V   -     0.825V ):         0 (   0.00%)
    Range 4(    0.825V   -     0.833V ):         0 (   0.00%)
    Range 5(    0.833V   -     0.840V ):         0 (   0.00%)
    Range 6(    0.840V   -     0.848V ):         0 (   0.00%)
    Range 7(    0.848V   -     0.855V ):         0 (   0.00%)
    Range 8(    0.855V   -     0.900V ):       285 (100.00%)
```

Fig: The above figure explains about the current source data.

- The whole design is divided on grid basis so there will be current and voltage sources inserted in the design for power analysis and these sources will be driving the standard cells in the design.

  - The display of IR map is done by changing the filter range from 1.59 (min) to 1.62 (max) and select the Auto button:



Generating Stream file

- Once everything has been completed next stage is generation of GDSII file. Select ‘save’-> ‘gds’.

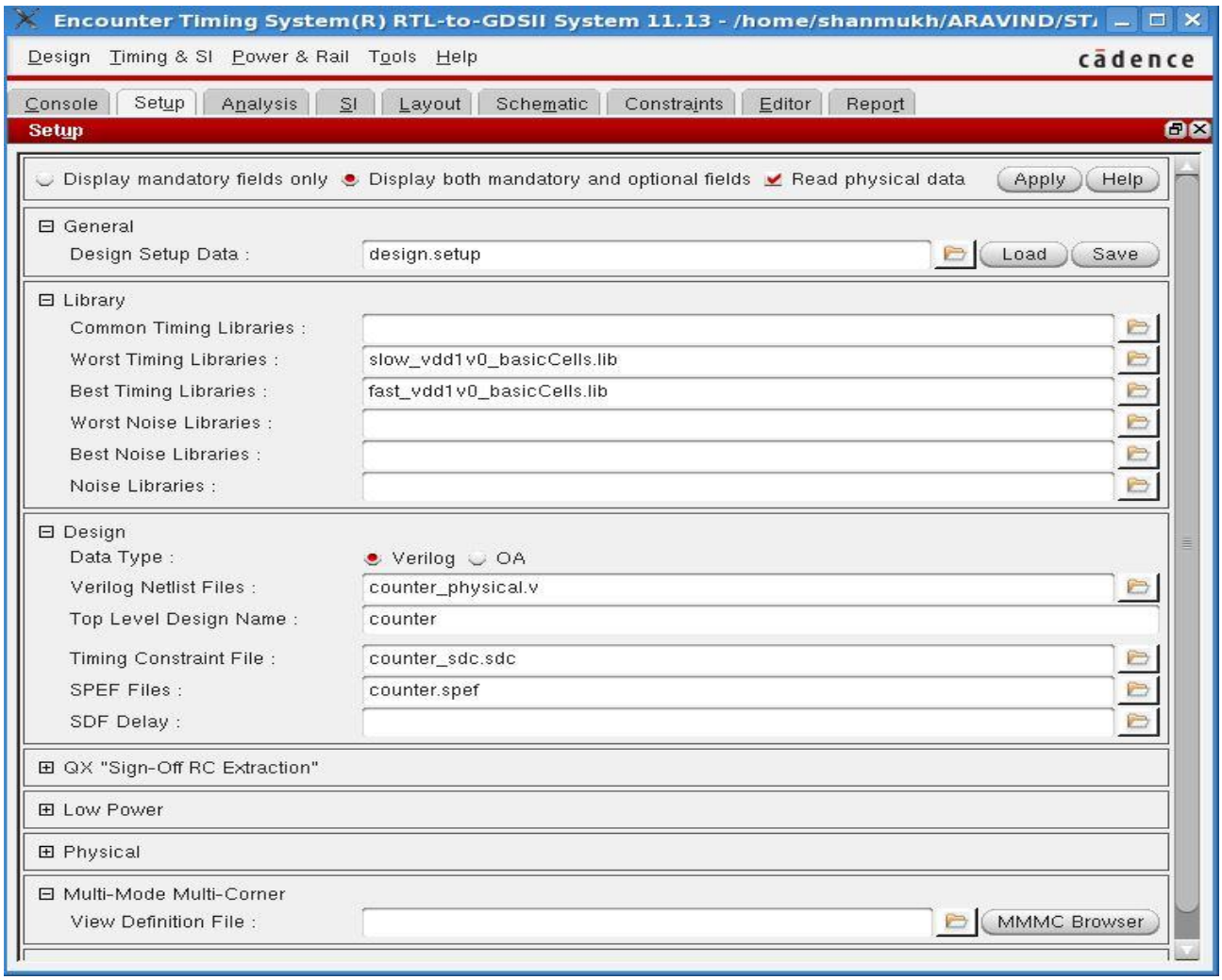
STA (Encounter Timing System)

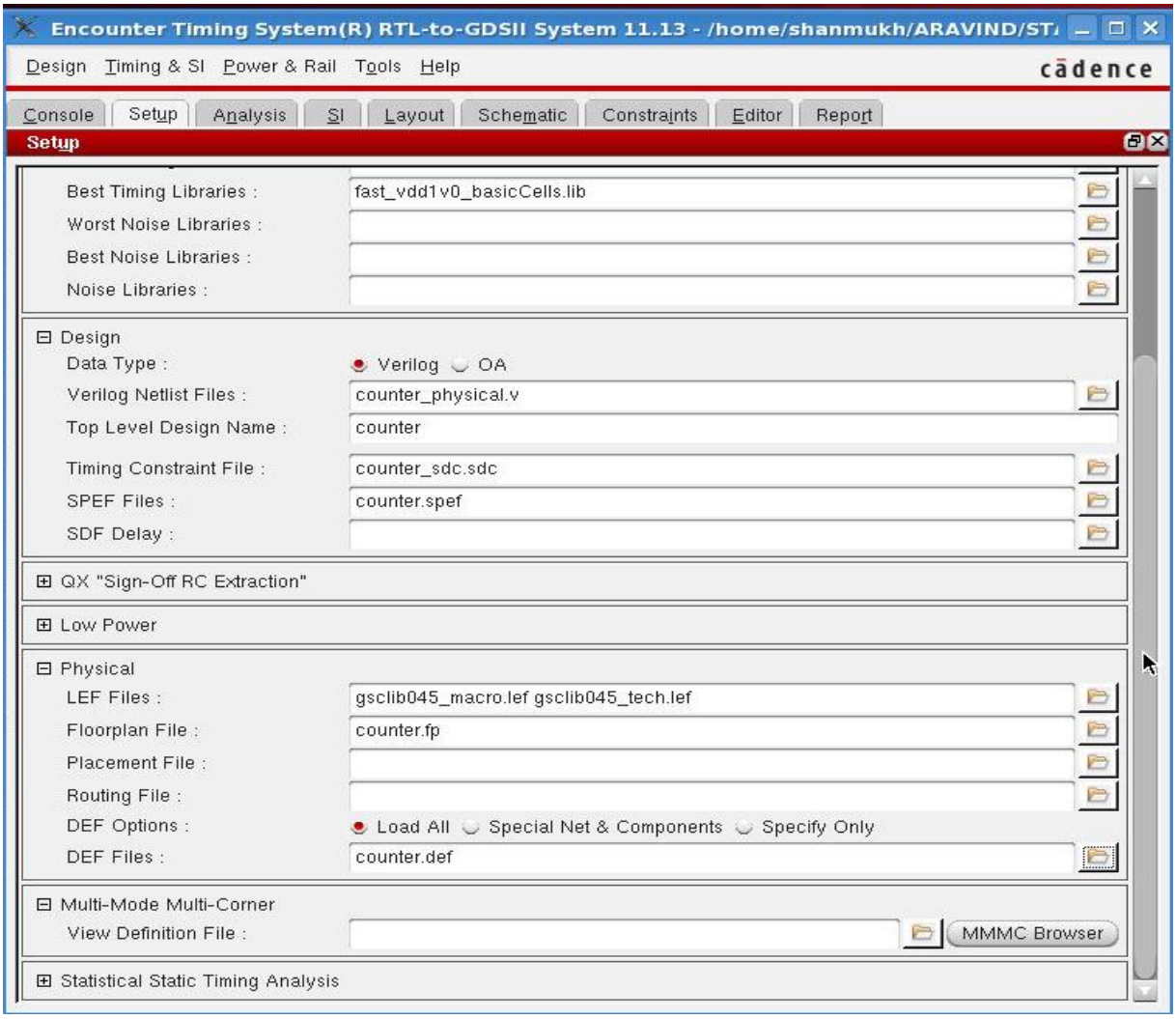
Encounter Timing System is a full-chip static timing analysis (STA) tool. Invoke the ETS tool using the command 'ets'. Invoke the tool inside STA directory.

The following inputs we are getting from the Physical design:

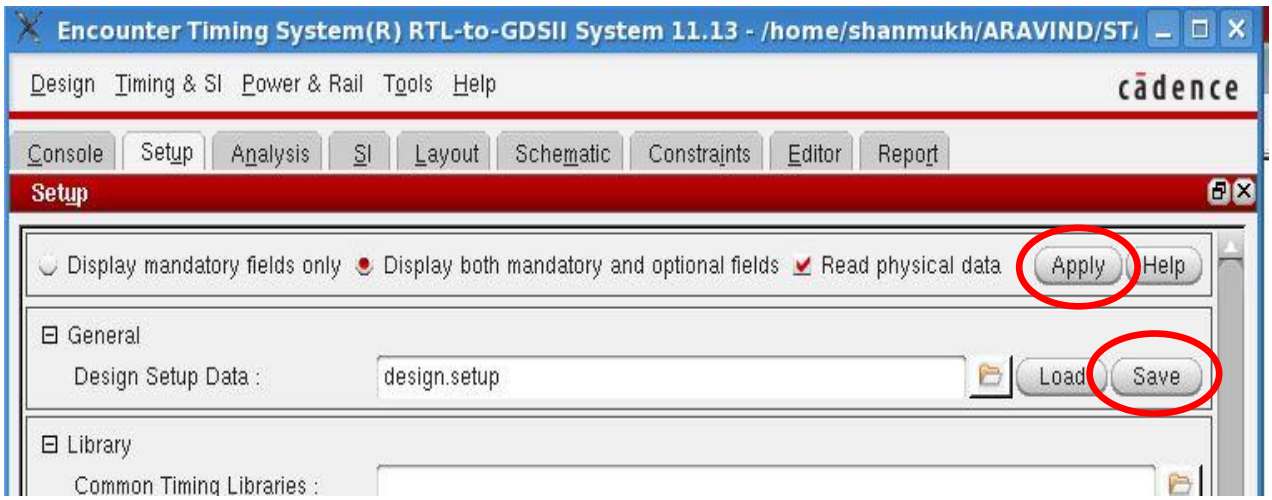
- counter_physical.v – gatelevel netlist output after physical design.
- counter_sdc.sdc – constraint file generated during synthesis.
- counter.spef – spef file generated after physical design.
- gsclib045_tech.lef, gsclib045_macro.lef – lef file used in physical design
- counter.fp – physical floorplan file
- counter.def – physical design exchange format.

➢ Browse the libraries (slow_vdd1v0_basicCells.lib and fast_vdd1v0_basicCells.lib form lib directory), netlist (counter_physical.v), constraints (counter_sdc.sdc), spef (counter.spef), physical design (counter.fp, counter.def) info as shown below.





- Save the settings and click Apply to start ets.



- Once the design is loaded take the below reports.

Generating Reports

- To Report the annotated SPEF coverage use the command “report_annotated_parasitics”

```
ets 8> report_annotated_parasitics
#########################################################################
# report_annotated_parasitics: Tue Sep  9 12:26:50 2014
#   view: default_emulate_view
#   -max_missing 1000
#########################################################################
# Summary of Annotated Parasitics:
+-------------------------------------------------------------------------+
|    Net Type     |    Count    |   Annotated (%)    | Not Annotated (%) |
+-----------------+-------------+--------------------+-------------------+
| total           |         29 |         28  96.6% |          1   3.4% |
+-----------------+-------------+--------------------+-------------------+
| 0-term:assign   |          1 |          0   0.0% |          1 100.0% |
| real net        |         28 |         28 100.0% |          0   0.0% |
+-------------------------------------------------------------------------+

+-----------------------------------------------------------------+
| Annotated |    Res (KOhm)    |    Cap (pF)     |    XCap (pF)    |
+-----------+------------------+-----------------+-----------------+
| Count     |            432   |            365   |             0   |
| Value     |        2.5397    |        0.0377    |        0.0000   |
+-----------------------------------------------------------------+
```

- To View the clock report use the command “report_clocks”

```
ets 10> report_clocks
#################################################################
#  Generated by:      Cadence Encounter Timing System 11.13-s118_1
#  OS:                Linux x86_64(Host ID nofcnl031)
#  Generated on:      Tue Sep  9 12:28:13 2014
#  Design:            counter
#  Command:           report_clocks
#################################################################
     ---------------------------------------------------------------
                                Clock Descriptions
     ---------------------------------------------------------------
                                                      Attributes
     ---------------------------------------------------------------
      Clock   Source   Period    Lead   Trail   Generated   Propagated
       Name
     ---------------------------------------------------------------
     clk     clk     10.000   0.000   5.000       n           n
     ---------------------------------------------------------------
```

- To Check the analysis coverage use the command “report_analysis_coverage”

```
ets 11> report_analysis_coverage
#################################################################
#  Generated by:      Cadence Encounter Timing System 11.13-s118_1
#  OS:                Linux x86_64(Host ID nofcnl031)
#  Generated on:      Tue Sep  9 12:29:22 2014
#  Design:            counter
#  Command:           report_analysis_coverage
#################################################################
    ------------------------------------------------------------------------------------------
                                TIMING CHECK COVERAGE SUMMARY
    ------------------------------------------------------------------------------------------
     Check Type                    No. of    Met            Violated         Untested
                                   Checks
    ------------------------------------------------------------------------------------------
    ExternalDelay (Late)          8         8 (100%)       0 (0%)           0 (0%)
    Hold                          24        6 (25%)        18 (75%)         0 (0%)
    PulseWidth                    24        16 (66%)       0 (0%)           8 (33%)
    Recovery                      8         8 (100%)       0 (0%)           0 (0%)
    Removal                       8         0 (0%)         0 (0%)           8 (100%)
    Setup                         24        24 (100%)      0 (0%)           0 (0%)
    ------------------------------------------------------------------------------------------
```

- To View the list of all constraint violations use the below command

  “report_constraint -all_violators”

```
ets 15> report_constraint -all_violators
#################################################################
#  Generated by:      Cadence Encounter Timing System 11.13-s118_1
#  OS:                Linux x86_64(Host ID nofcnl031)
#  Generated on:      Tue Sep  9 12:32:33 2014
#  Design:            counter
#  Command:           report_constraint -all_violators
#################################################################

max_delay/setup
---------------------------

No paths found

min_delay/hold
---------------------------
      ------------------------------------------------------------------------------
       End Point                                   Slack      Cause
      ------------------------------------------------------------------------------
      count_reg[7]/SE f                           -0.128    VIOLATED
      count_reg[6]/SE f                           -0.128    VIOLATED
      count_reg[5]/SE f                           -0.128    VIOLATED
      count_reg[4]/SE f                           -0.128    VIOLATED
      count_reg[3]/SE f                           -0.128    VIOLATED
      count_reg[2]/SE f                           -0.128    VIOLATED
      count_reg[1]/SE f                           -0.128    VIOLATED
      count_reg[0]/SE f                           -0.128    VIOLATED
      count_reg[0]/SI f                           -0.106    VIOLATED
      count_reg[5]/SI f                           -0.022    VIOLATED
      count_reg[7]/SI f                           -0.022    VIOLATED
      count_reg[3]/SI f                           -0.022    VIOLATED
      count_reg[6]/SI f                           -0.021    VIOLATED
```

- To report the worst slack time for setup and hold using the commands “report_timing –late” and “report_timing –early”

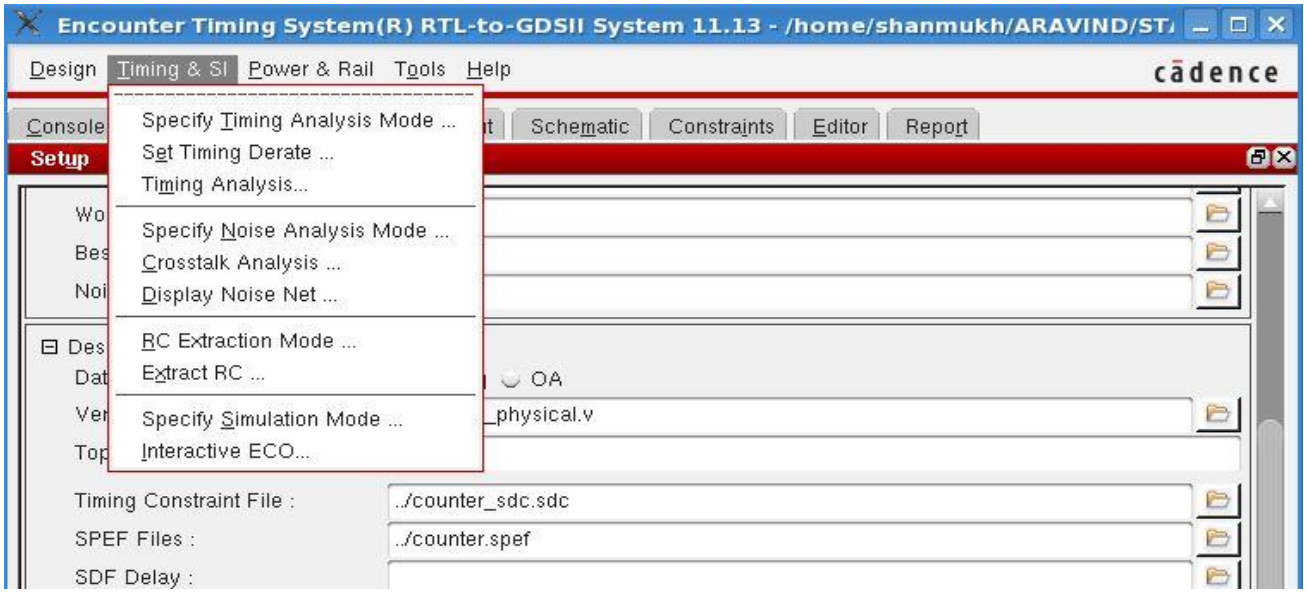
```
ets 17> report_timing -late
#====================================================================
#  Generated by:      Cadence Encounter Timing System 11.13-s118_1
#  OS:                Linux x86_64(Host ID nofcnl031)
#  Generated on:      Tue Sep  9 12:37:54 2014
#  Design:            counter
#  Command:           report_timing -late
#====================================================================
Path 1: MET Setup Check with Pin count_reg[7]/CK
Endpoint:   count_reg[7]/D (^) checked with  leading edge of 'clk'
Beginpoint: count_reg[0]/Q (^) triggered by  leading edge of 'clk'
Other End Arrival Time          0.000
- Setup                         0.181
+ Phase Shift                  10.000
- Uncertainty                   0.100
= Required Time                 9.719
- Arrival Time                  1.493
= Slack Time                    8.227
     Clock Rise Edge                 0.000
     + Clock Network Latency (Ideal) 0.000
     = Beginpoint Arrival Time       0.000
     ------------------------------------------------------------------------------------------
      Instance              Arc              Cell             Delay     Arrival    Required
                                                                        Time       Time
     ------------------------------------------------------------------------------------------
     count_reg[0]          CK ^                                        0.000      8.227
     count_reg[0]          CK ^ -> Q ^      SDFFRHQX1        0.323     0.323      8.550
     g851                  B ^ -> Y v       NAND2X1          0.120     0.442      8.669
     g849                  AN v -> Y v      NAND2BX1         0.169     0.611      8.838
     g847                  AN v -> Y v      NAND2BX1           0.181   0.792      9.019
     g845                  AN v -> Y v      NAND2BX1         0.192     0.984      9.210
     g843                  AN v -> Y v      NAND2BX1         0.171     1.154      9.381
     g841                  AN v -> Y v      NAND2BX1           0.145   1.299      9.526
     g839                  B v -> Y ^       XNOR2X1          0.194       1.493    9.719
     count_reg[7]          D ^              SDFFRHQX1        0.000       1.493    9.719
     ------------------------------------------------------------------------------------------
```

Fig: above figure shows the result for “report_timing –late”

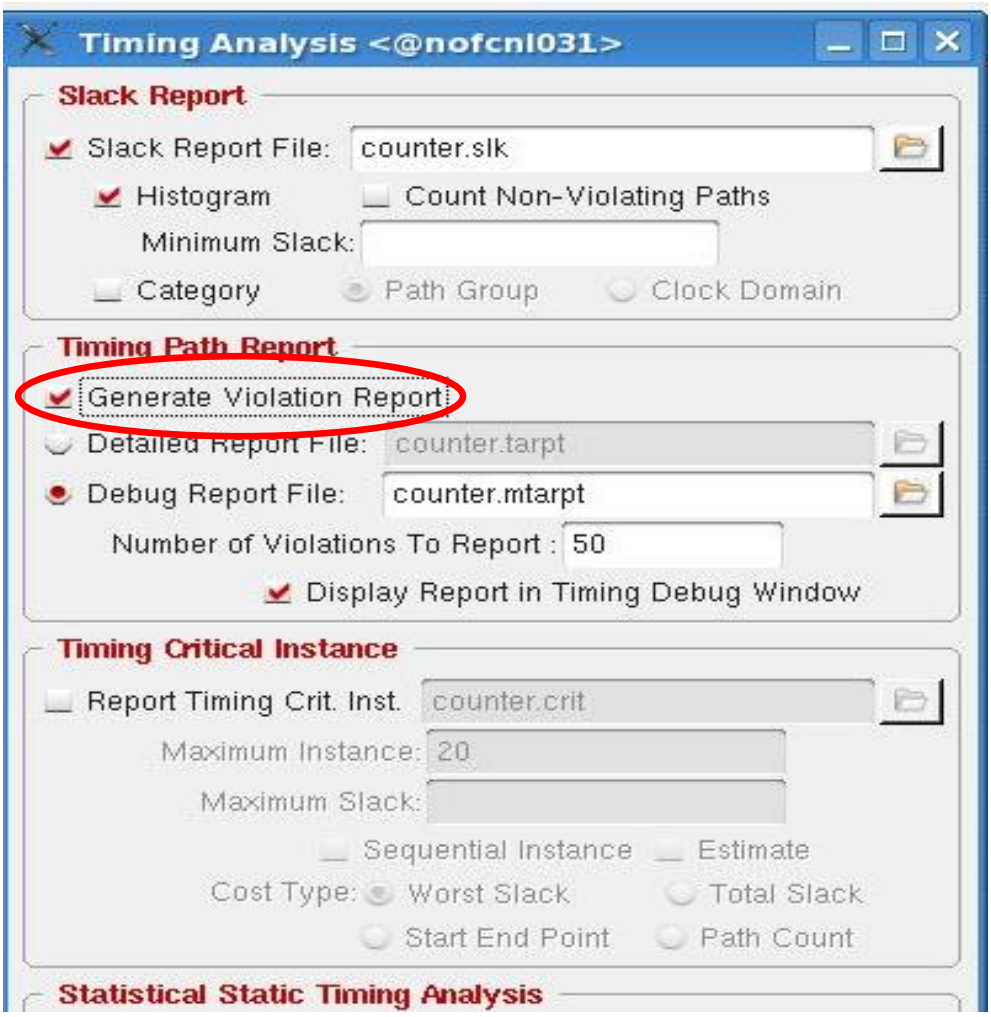
```
ets 18> report_timing -early
#############################################################
#  Generated by:      Cadence Encounter Timing System 11.13-s118_1
#  OS:                Linux x86_64(Host ID nofcnl031)
#  Generated on:      Tue Sep  9 12:38:37 2014
#  Design:            counter
#  Command:           report_timing -early
#############################################################
Path 1: VIOLATED Hold Check with Pin count_reg[0]/CK
Endpoint:   count_reg[0]/SE (v) checked with  leading edge of 'clk'
Beginpoint: SE              (v) triggered by  leading edge of '@'
Other End Arrival Time          0.000
+ Hold                         0.028
+ Phase Shift                  0.000
+ Uncertainty                  0.100
= Required Time                0.128
  Arrival Time                 0.000
  Slack Time                  -0.128
     Clock Rise Edge                      0.000
     + Input Delay                        0.000
     = Beginpoint Arrival Time            0.000
     ------------------------------------------------------------------------------------------
      Instance              Arc              Cell             Delay     Arrival    Required
                                                                        Time       Time
     ------------------------------------------------------------------------------------------
                           SE v                                          0.000      0.128
     count_reg[0]          SE v             SDFFRHQX1        0.000     0.000      0.128
     ------------------------------------------------------------------------------------------
```

Fig: Above figure shows the result for “report_timing –early”

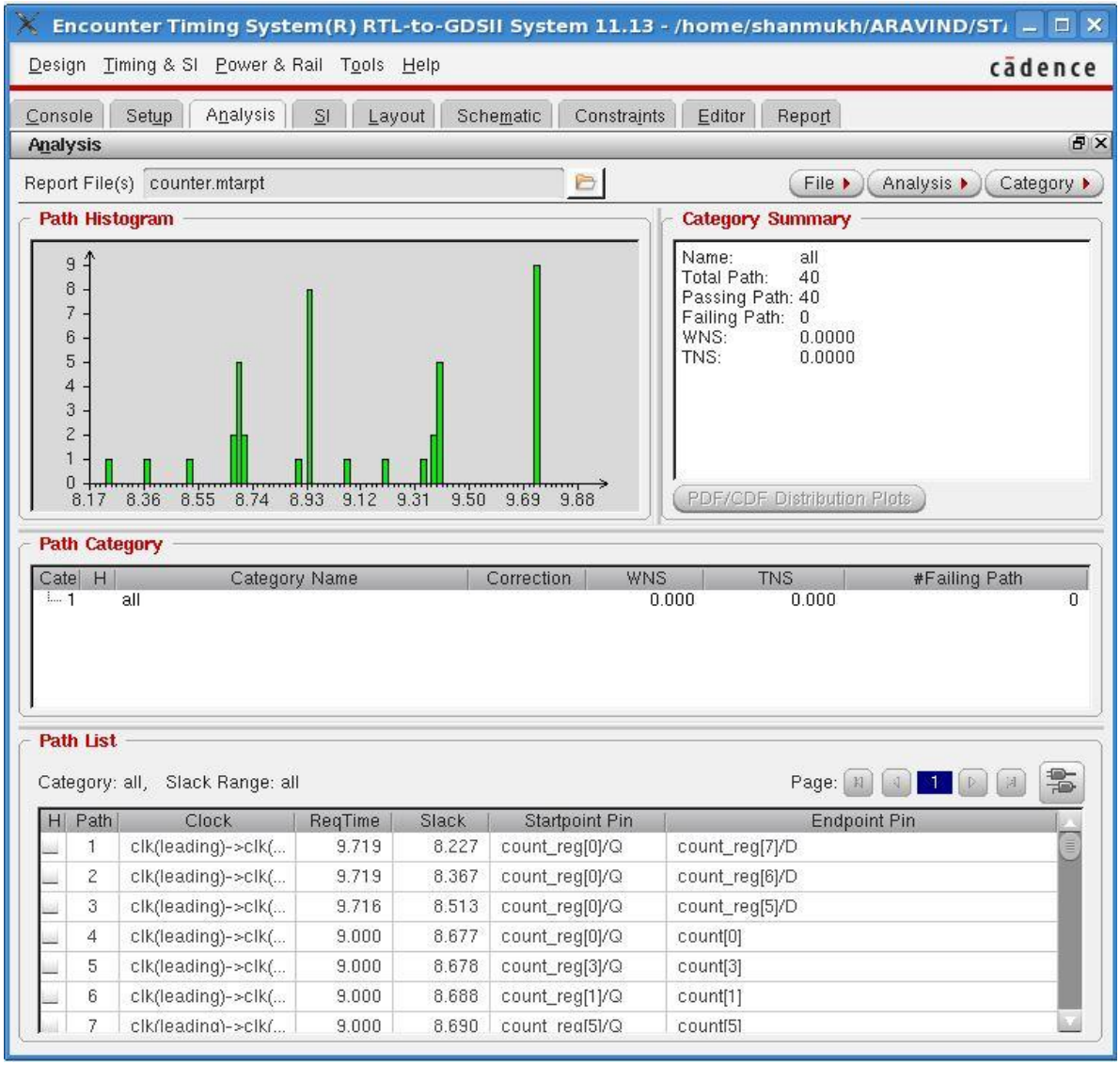
- Now to perform the timing analysis, go to click on **Timing and SI --> Timing Analysis** tab as shown in the below snapshot.



- You will get a pop up window in which you have to select the option **Generate Violation Report** from the timing path report pane.

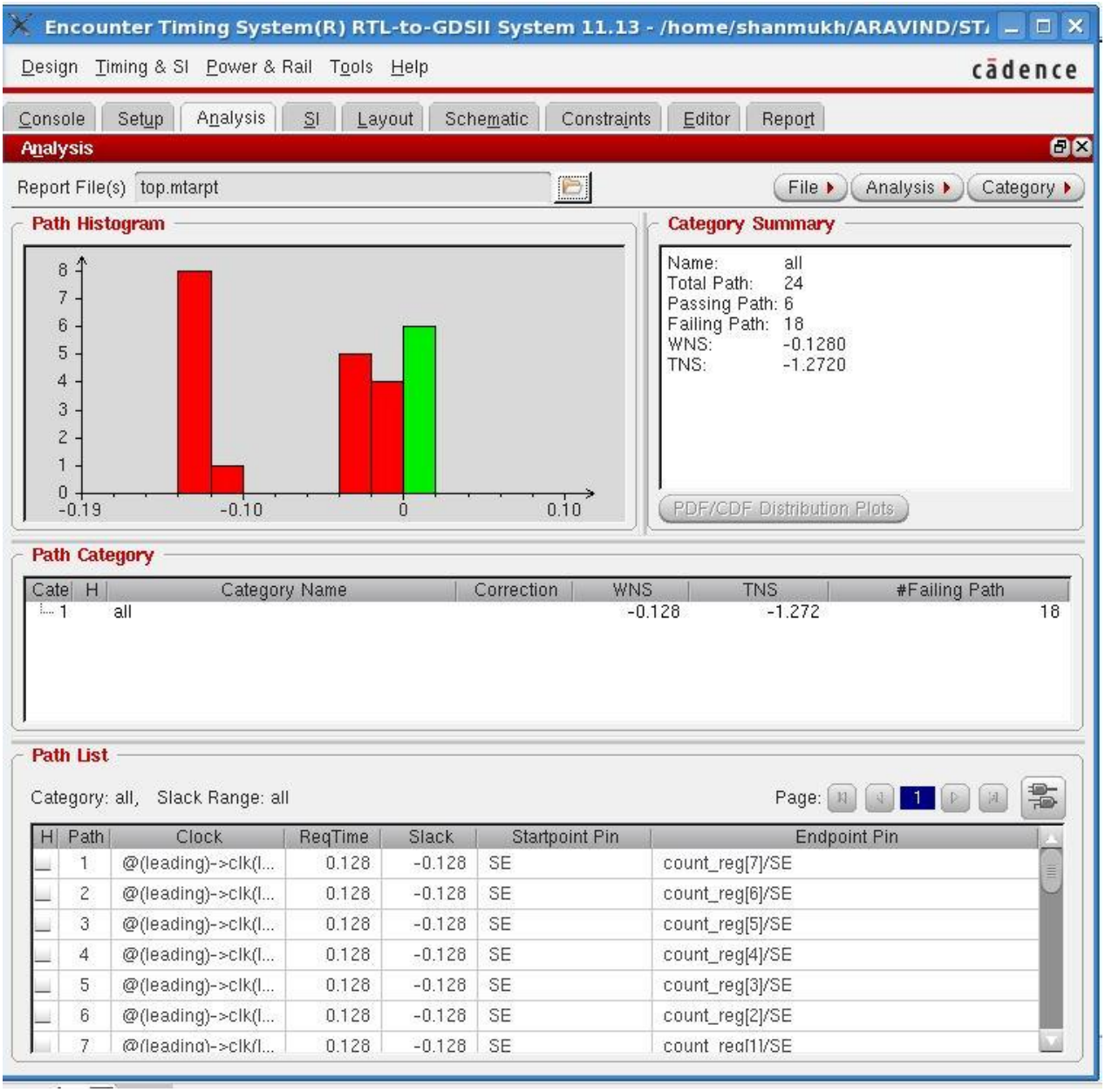


- Once timing analysis is done you can see the histogram of setup analysis in Analysis tab.

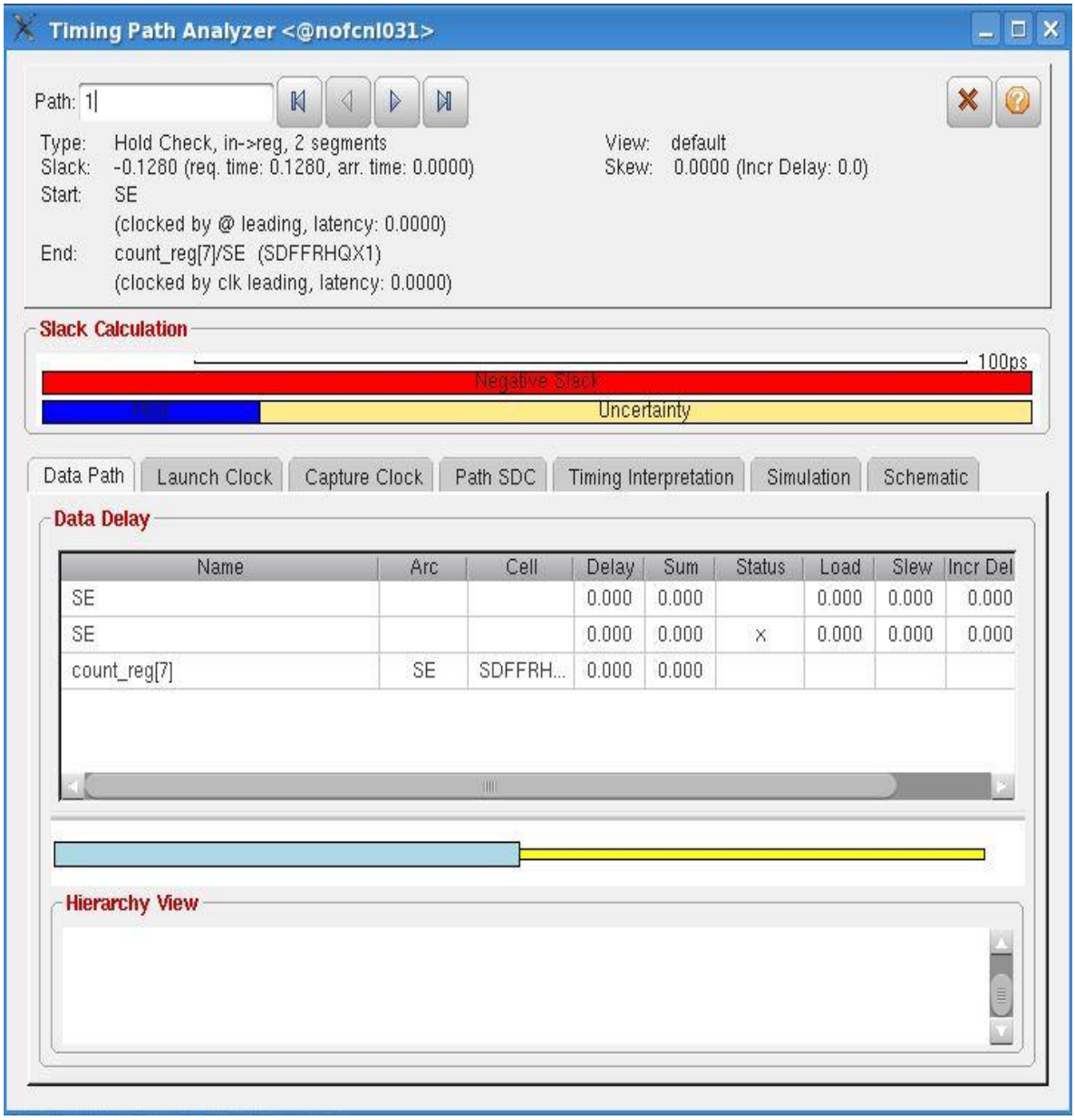


4. You can see the hold histogram in the same way by browsing your pointer to **Report File** section and change the **Check Type** to **hold** after getting the pop up window of **Display/Generate Timing Reports**.

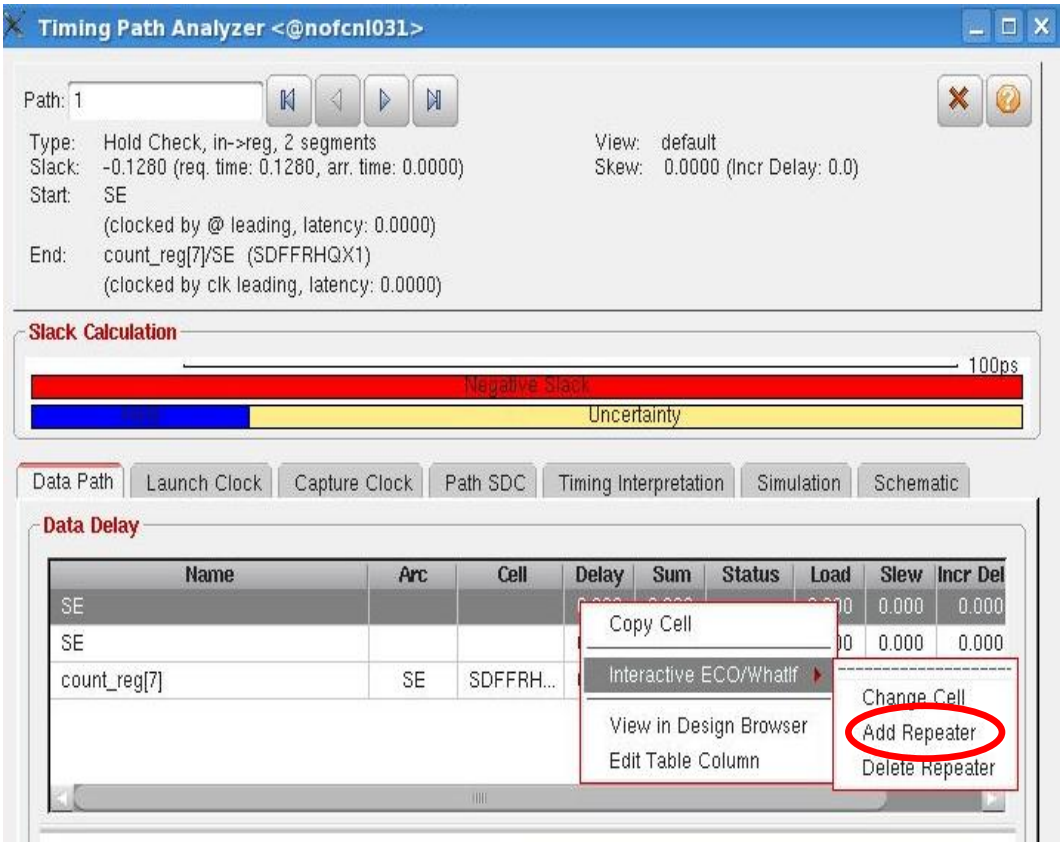
- You will get a hold Histogram accordingly, below is the snapshot for the same and you view pass paths and failing paths under **Category Summary** plane.



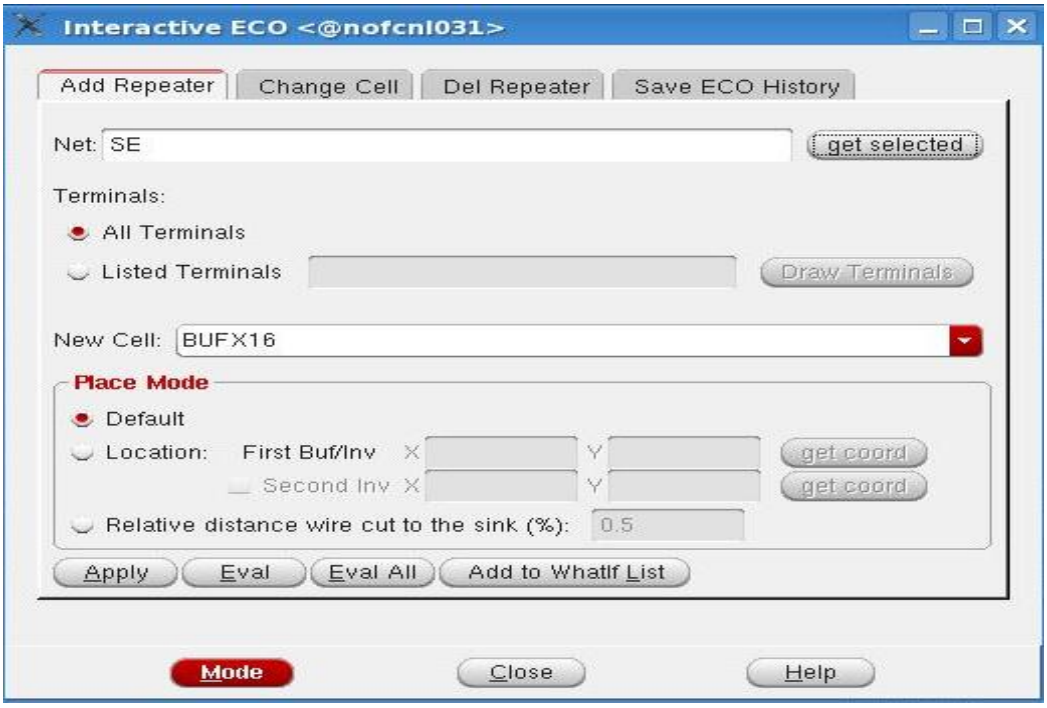
- From the **Path List**, you can right click on any path and click on the **Show Timing Path Analyzer** where you will get a pop up window, below is snapshot for the same.



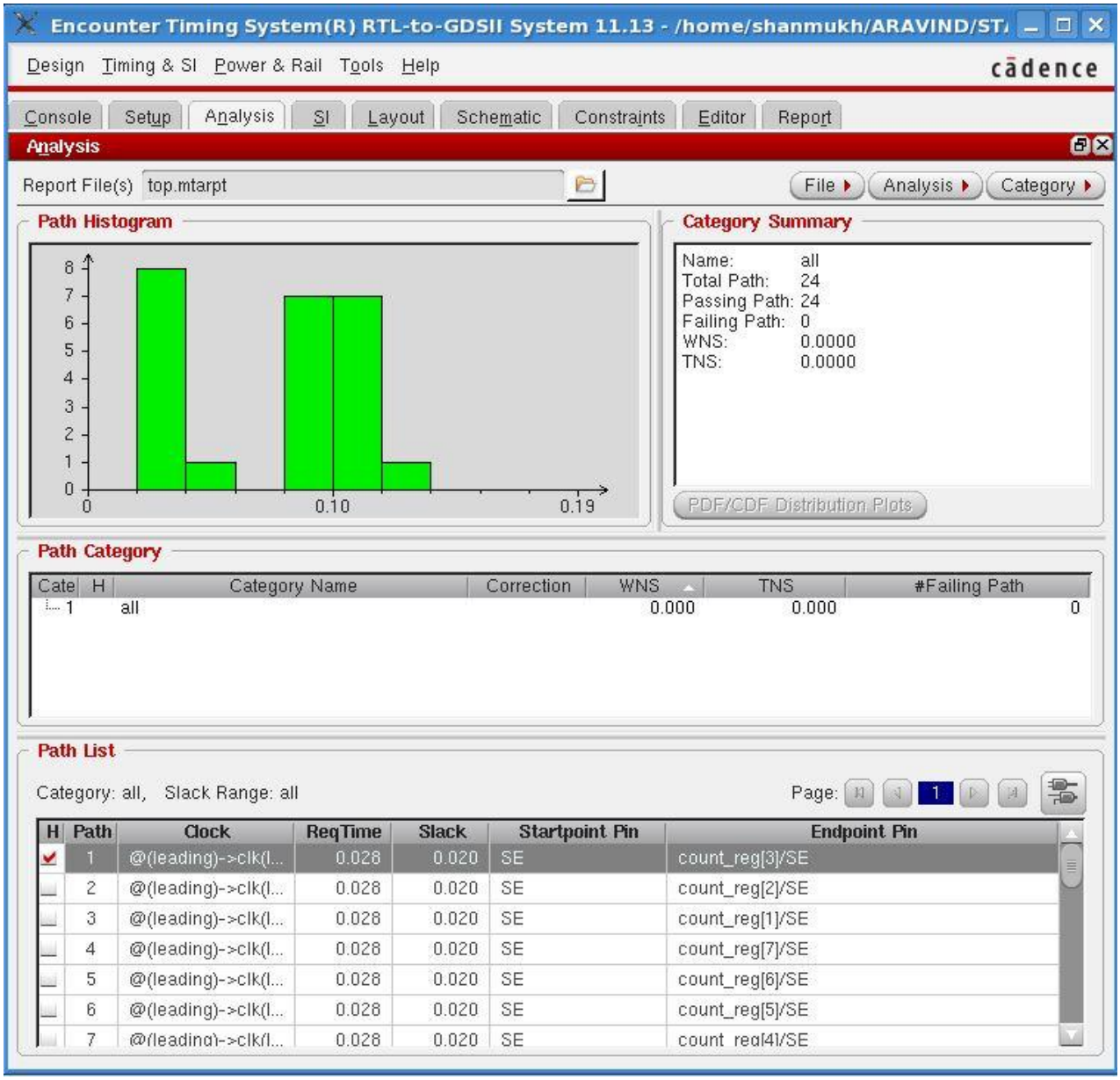
- From the **Timing Path Analyzer,** if you want to debug violating paths, you can right click on any of the signals where you will get additional options like interactive ECO etc, below is snapshot for the same.



- Click on **Add Repeater** where you will be directed to another window for adding cells or repeater, below is the snapshot for the same.



- After doing ECO, Click on evaluate and apply.
- Once the timing is debugged you will get histogram in the below format.



- You can see the layout and schematic from the tabs.

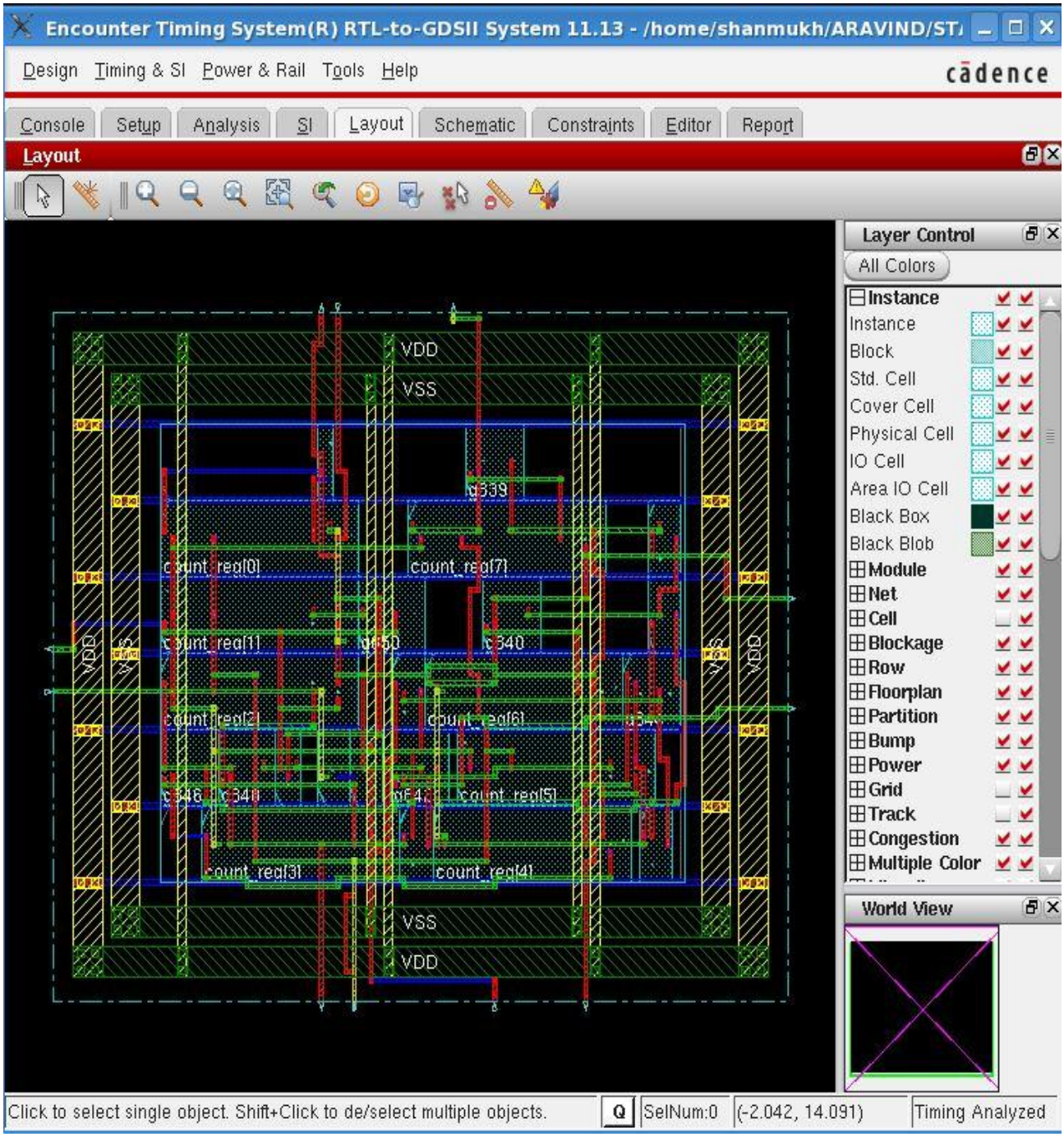


Fig: Above figure shows the layout view of the design.

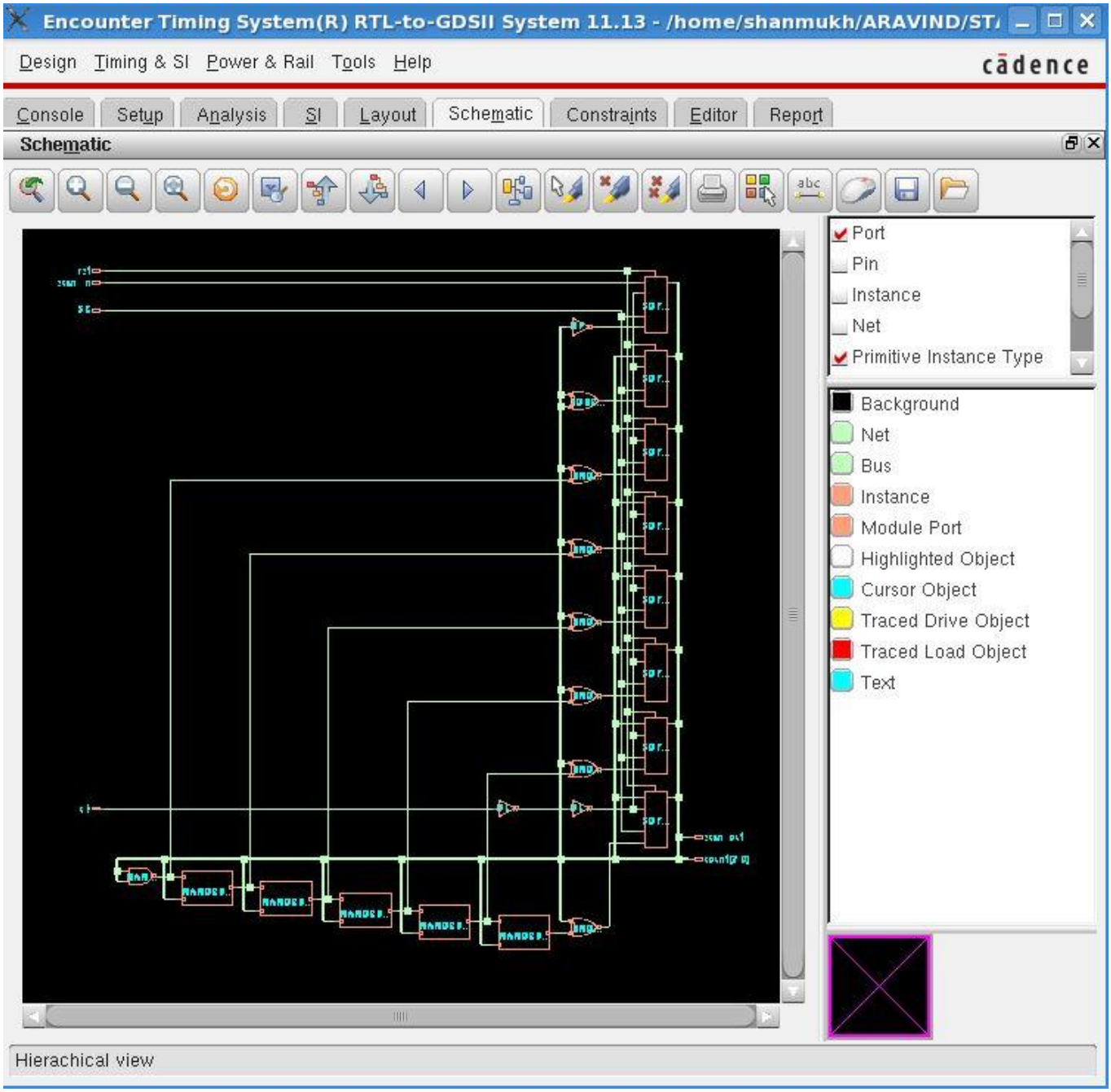
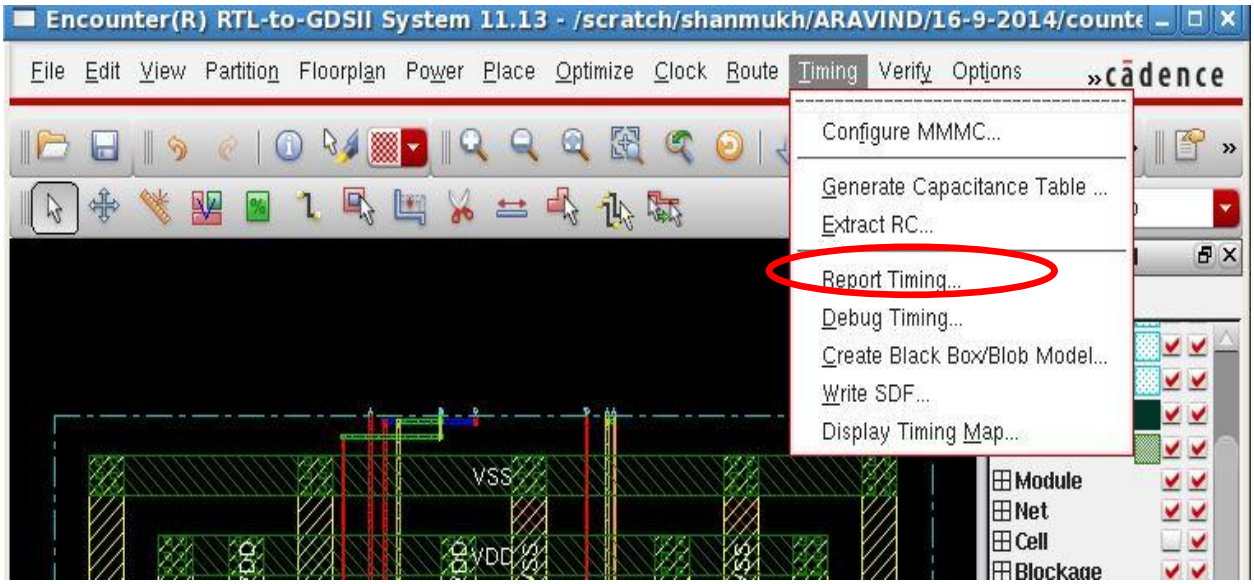


Fig: Above figure shows the schematic view of the design.

- At last you can fix these errors accordingly in the EDI flow.

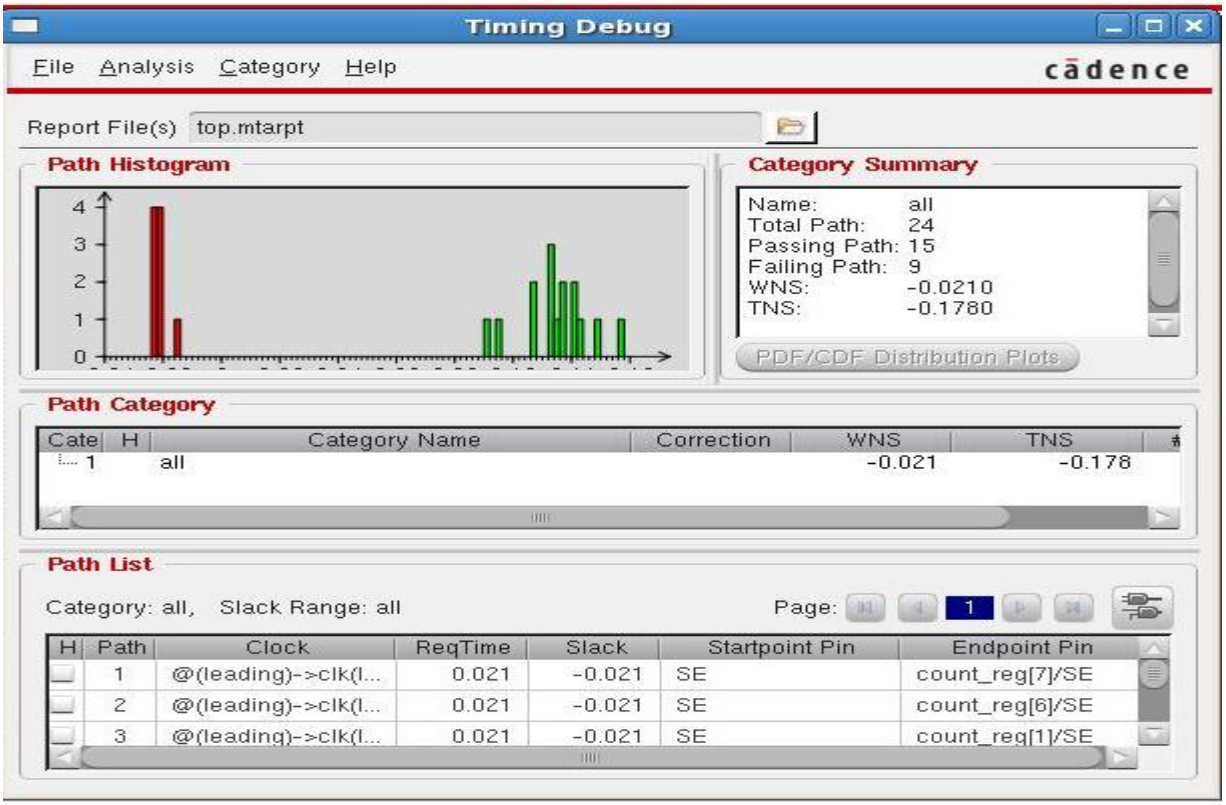
Debugging the timing violation in EDI

- Go to the Timing→ Debug Timing.

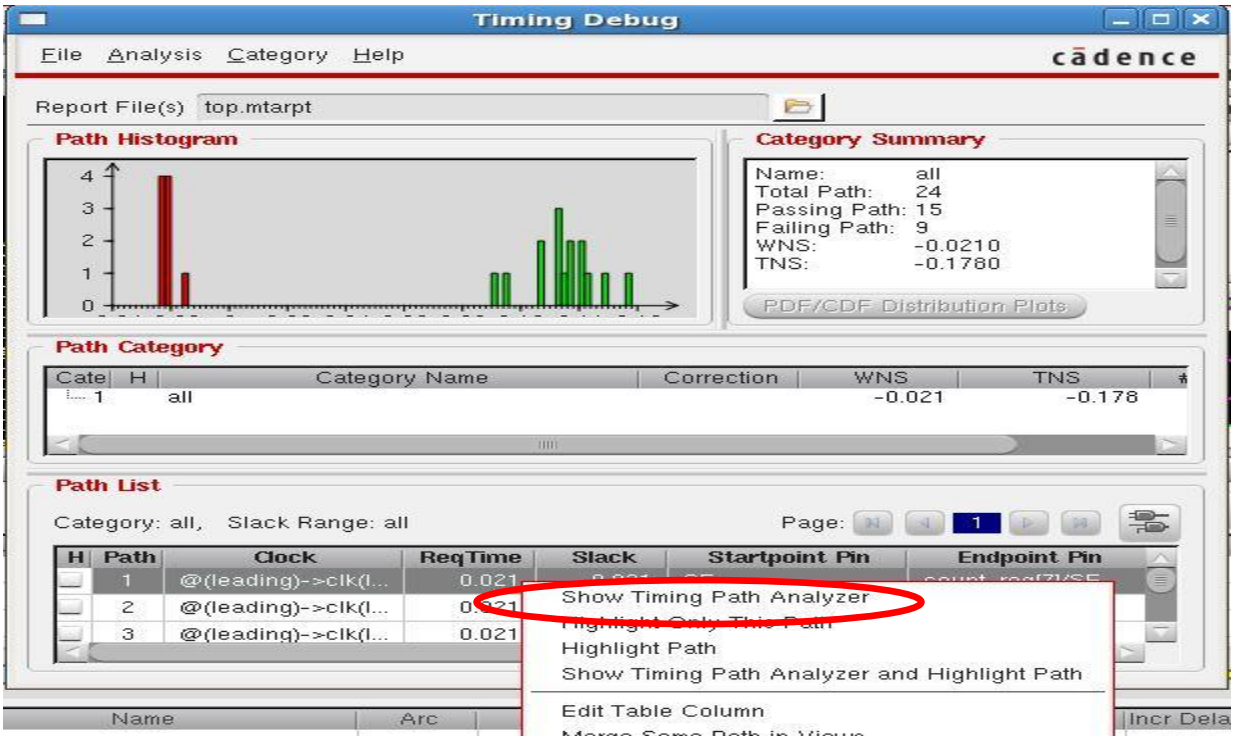


- You will get a pop-up window Display/Generate Timing Reports as shown in the below snapshot and change the check Type to hold to perform hold analysis then click OK.

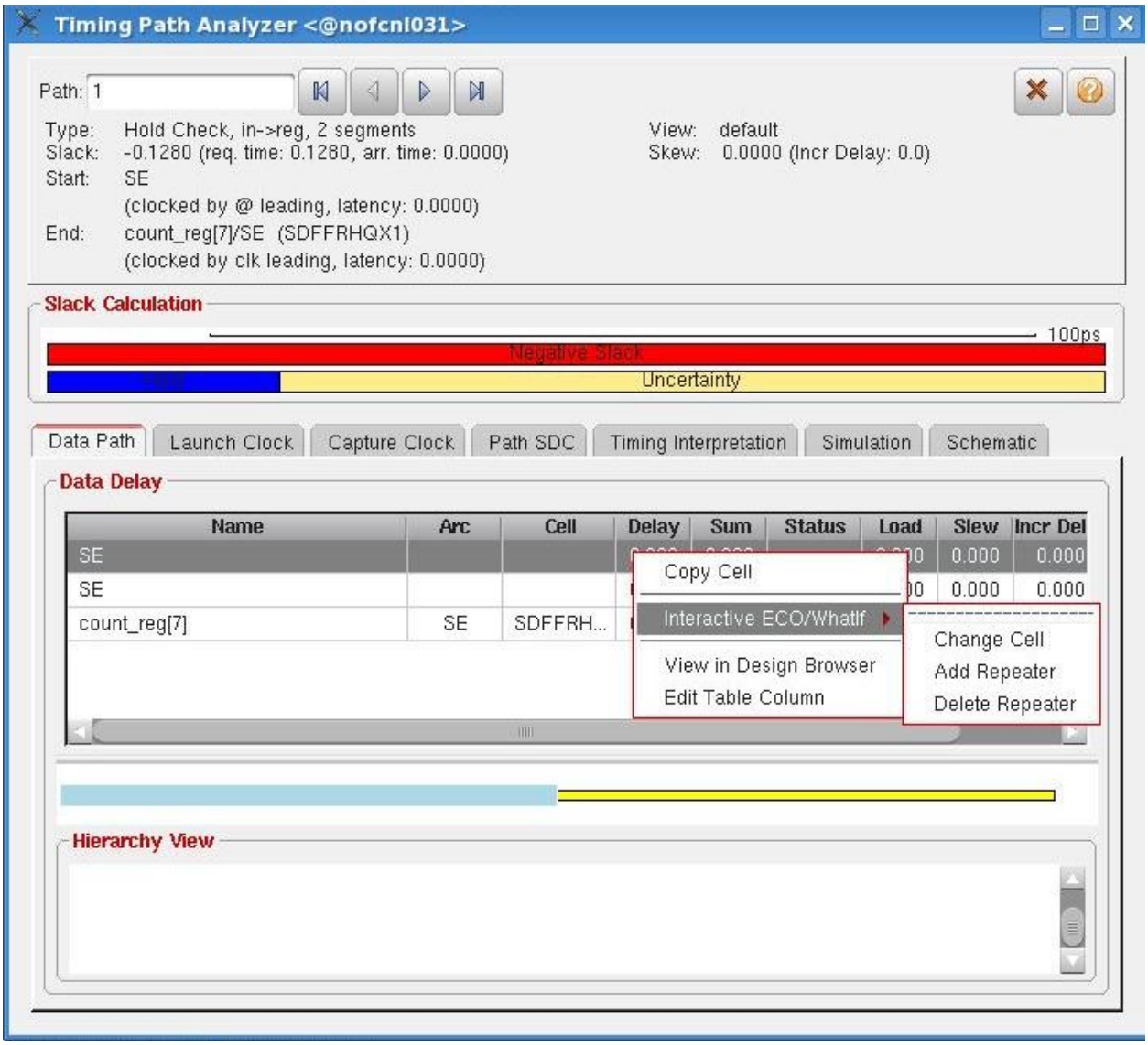
- You will get a hold Histogram in the Timing debug window accordingly, below is the snapshot for the same and you can have a watch over the pass paths and failing paths in the category pane.



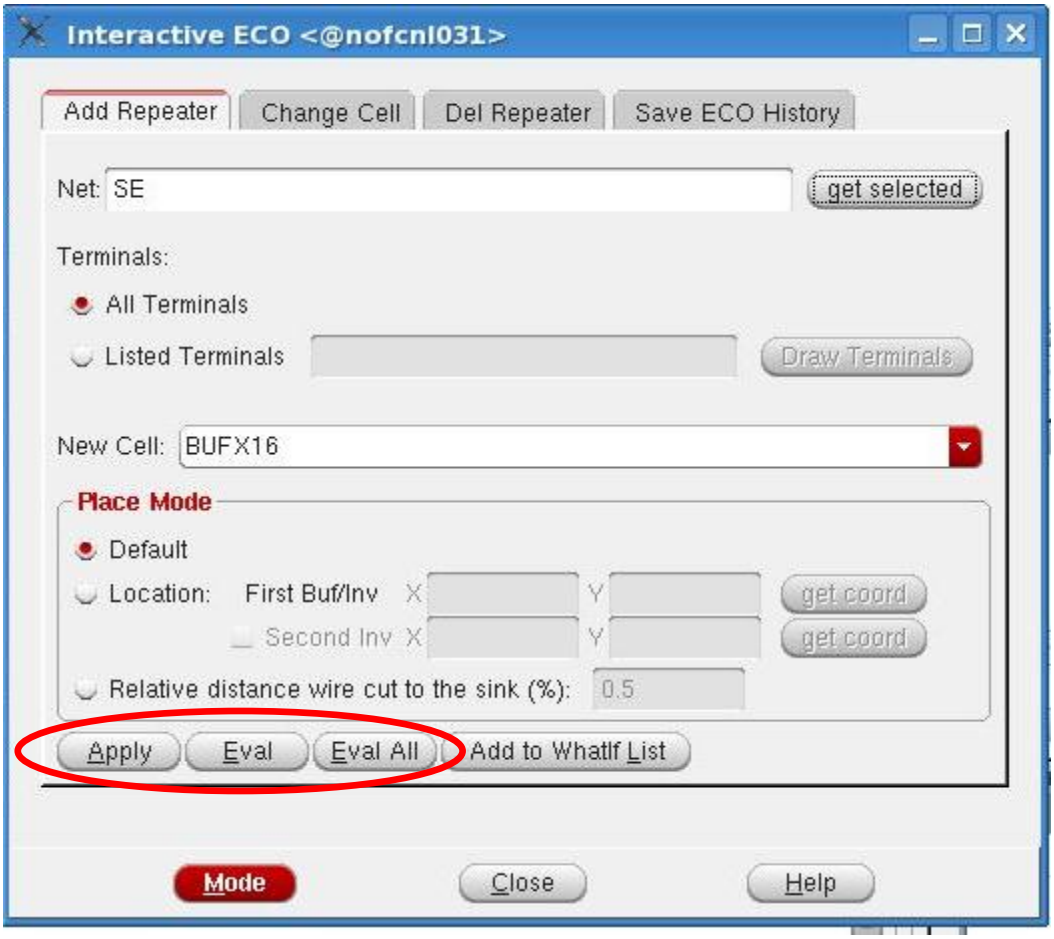
➢ From the path list you can Right click on any paths and click on the **show timing path analyzer** where you will get a pop up window, below is snapshot for the same.



- It will pop-up a **Timing Path Analyzer** window, From the Timing path analyzer if you want to debug you can right click on any of the signals where you will get additional options like interactive ECO etc. Below is snapshot for the same



- Click on **Add Repeater** where you will be directed to another window for adding cells or repeater, below is the snapshot for the same.



- Click on evaluate and apply.

Once the timing is debugged you can perform verification task of the design in EDI flow as verify connectivity and geometry as shown on the physical design flow manual.

Gate Level simulation

GLS is a step in the design flow to ensure that the design meets the functionality after Synthesis or after placement and routing. We need synthesized/post-routed netlist, Test-bench and SDF (Standard Delay Format) file. SDF will have all the delay information for the cell and the wire.

Here we will make use of the same test-bench what we have used for the functional simulation with some changes in the test-bench. That is, we have to use $sdf_annotate system task to call sdf file inside the test-bench.

We will perform Gate Level Simulation inside *'gate_level_simulation'* directory under counter_database.

Files present inside gate_level_simulation directory are

- Counter_netlist.v - netlist after synthesis with DFT. [Netlist after physical design also be used]
- Counter_test.v - testbench with "$sdf_annotate" system task to input SDF file.
- slow_vdd1v0_basicCells.v – Simulation library in '.v' format
- delays.sdf – SDF file generated during synthesis. [SDF can also be generated after physical design]

STEP 1: Modify testbench to include SDF configuration as shown below. In counter_test.v[available inside gate_level_simulation], SDF configuration system task is already included.

```
`ifdef SDF_TEST
initial
begin
$sdf_annotate("delays.sdf",counter_test.counter1,,"sdf.log","MAXIMUM",,,);
end
`endif
```

$sdf_annotate ("sdf_file"
{, module_instance}
{, "config_file"}
{, "log_file"}
{, "mtm_spec"}
{, "scale_factors"}
{, "scale_type"});

Note: We must specify the arguments to the $sdf_annotate system task in the order shown in the syntax. We can skip an argument specification, but the number of comma separators must maintain the argument sequence. For example, to specify only the first and last arguments, use the following syntax:

$sdf_annotate ("sdf_file",,,,,, "scale_type");

$sdf_annotate Arguments:

"sdf_file" The full or relative path of the SDF file. This argument is required and must be in quotation marks. We can specify the file name with the +sdf_file plus option on the command line.

module_instance Optional: Specifies the scope in which the annotation takes place. The names in the SDF file are relative paths to the module_instance with respect to the entire Verilog HDL description. The SDF Annotator uses the hierarchy level of the specified instance for running the annotation. Array indexes (module_instance[index]) are permitted in the scope. If we do not specify module_instance, the SDF Annotator uses the module containing the call to the $sdf_annotate system task as the module_instance for annotation.

"config_file" Optional: The name of the configuration file, specified in quotation marks, that the SDF Annotator reads before annotating begins. If we do not specify config_file, the SDF Annotator uses the default settings.

"log_file" Optional: The name of the log file specified in quotation marks, that the SDF Annotator generates during annotation. Also, you must specify the +sdf_verbose plus option on the command line to generate a log file. If we do not specify a log file name, but do specify the +sdf_verbose plus option, the SDF Annotator creates a default log file called sdf.log.

"mtm_spec" Optional: One of the following keywords, specified in quotation marks, indicating the delay values that are annotated to the Verilog family tool.

Keyword	Description
`MAXIMUM`	Annotates the maximum delay value.
`MINIMUM`	Annotates the minimum delay value.
`TOOL_CONTROL` (default)	Annotates the delay value that is determined by the Verilog-XL and Verifault-XL command line options (`+mindelays`, `+typdelays`, or `+maxdelays`); minimum, typical, and maximum values are always annotated to Veritime. If none of the `TOOL_CONTROL` command line options is specified, the default keyword is then `TYPICAL`.
`TYPICAL`	Annotates the typical delay value.

"scale_factors" optional: The minimum, typical, and maximum timing data values, specified in quotation marks, expressed as a set of three positive real number multipliers (min_mult:typ_mult:max_mult). For example, 1.6:1.4:1.2. If we do not specify values, the default values are 1.0:1.0:1.0 for minimum, typical, and maximum values. The SDF Annotator uses these values to scale the minimum, typical, and maximum timing data from the SDF file before they are annotated to the Verilog family tool.

"**scale_type**" Optional: One of the following keywords, specified in quotation marks, to scale the timing specifications in SDF, which are annotated to the Verilog family tool.

Keyword	Description
FROM_MAXIMUM	Scales from the maximum timing specification.
FROM_MINIMUM	Scales from the minimum timing specification.
FROM_MTM (default)	Scales from the minimum, typical, and maximum timing specifications. This is the default.
FROM_TYPICAL	Scales from the typical timing specification.

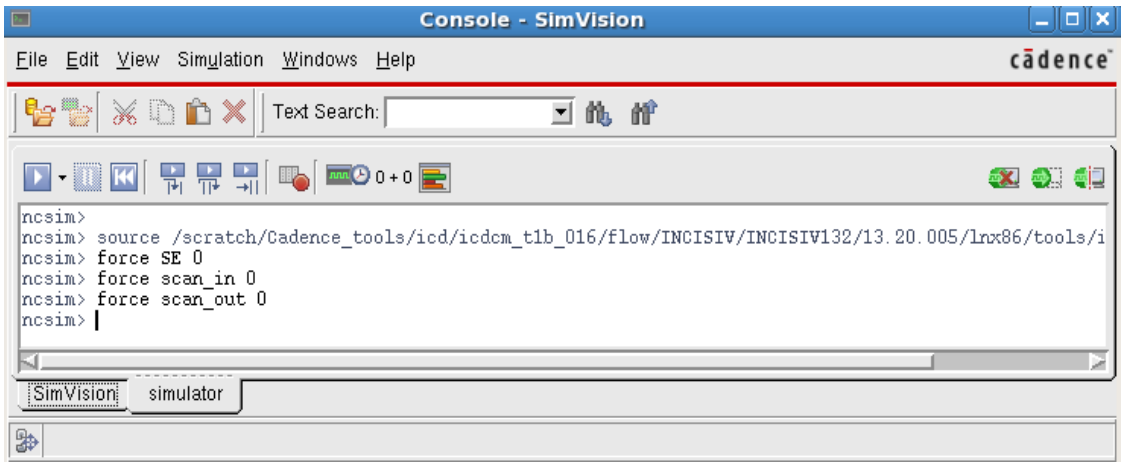
STEP 2: Simulate the netlist with the below irun command.

irun -timescale 1ns/10ps counter_netlist.v counter_test.v -v slow_vdd1v0_basicCells.v -access +rwc -define SDF_TEST -mess –gui

- -timescale: option used to mention the time unit and time precision.
- -access: this option is passed to the elaborator to provide read access to simulation objects.
- -gui: option used to invoke the irun in gui mode.
- -mess: option used to display all the messages in detail.
- -define: option is used to provide SDF definition present in the testbench.
- -v: option used to provide library in '.v' format.

STEP 3: Click on counter_test in the design browser window, and then we will get all the signals in the objects window. Force DFT signals such as SE, scan_in and scan_out signals with value 0. Execute the commands as shown like below in the console window.

- Force SE 0
- Force scan_in 0
- Force scan_out 0



STEP 4: Select all the signals and send those selected objects to waveform window by clicking 'Send selected objects to waveform window'  .

STEP 5: Click on run button  .

Below snapshot shows the counter waveform results with back annotated delays.

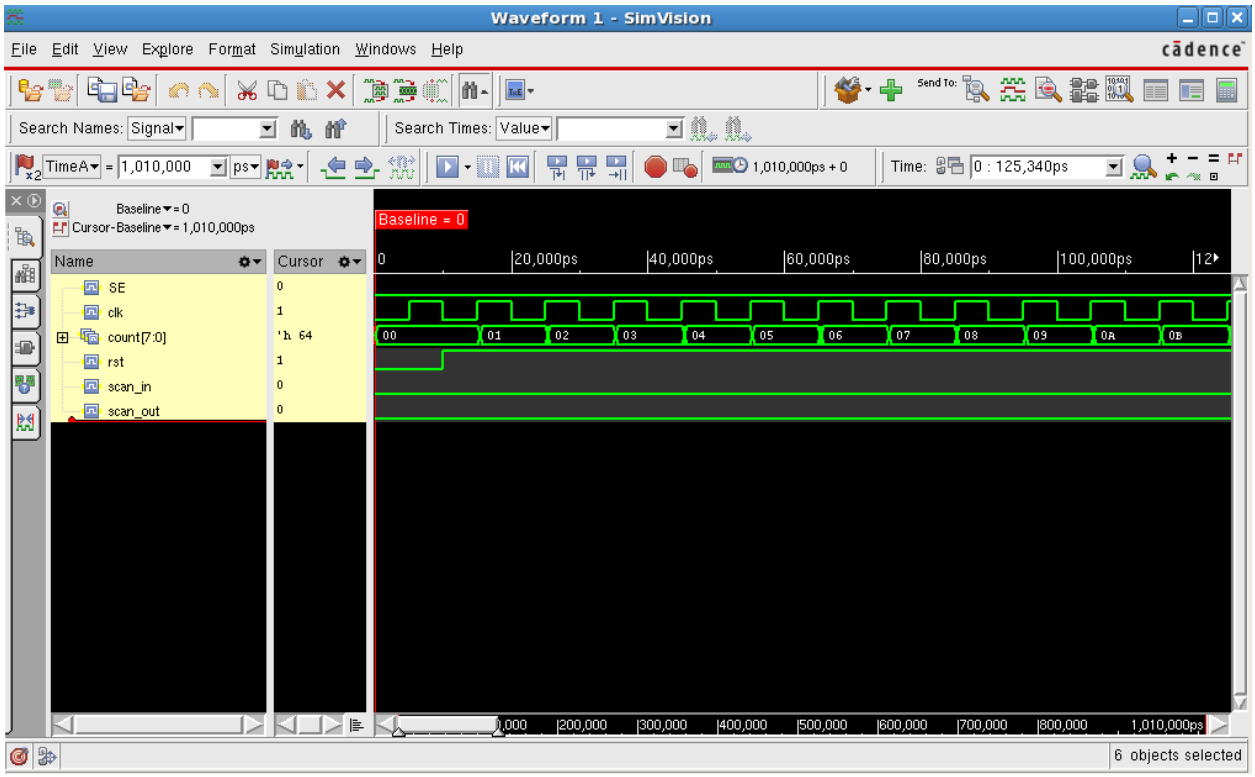


Fig: Above figure shows the functionality of counter in gate level simulation.

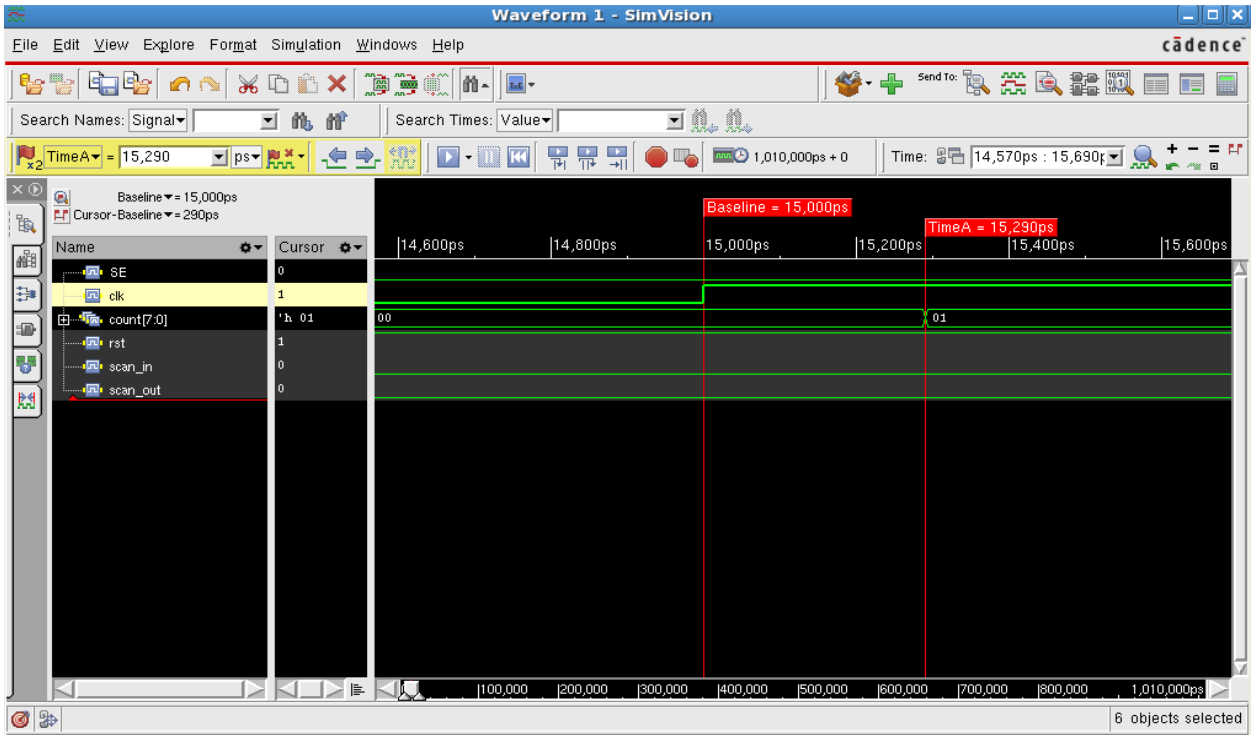


Fig: Above figure shows the back annotated delays in waveform.